

SCOPE OF APPLICATION All Project/Engineering	HYUNDAI AutoEver	SHT/SHTS 1 / 97
Responsibility: 제어SW플랫폼1팀	AUTOSAR Fbl User Manual	DOC. NO
AUTOSAR Fbl User Manual		

Document Change History				
Date (YYYY-MM-DD)	Ver.	Editor	Chap	내용(개정 전 -> 개정 후)
2019-08-13	1.0	강신일	전체	• 신규제정
2019-12-23	1.1	강신일	전체	• CAN 기능 추가 • 이중화 기능 적용시 호출되는 callout 업데이트
2020-01-09	1.2	강신일	2.2 3.2.3 4.1.3 3.2.18/19	• Release 정보 갱신 • FblPllConfig 컨테이너 설정 변경 • Public Key 정보 추가 • Category 일부 조정
2020-02-10	1.3	강신일	3.2 4.1.3 6	• PLL 설정, SPC58X 설정 추가 • Public key 정보 추가 • Open source copyright 명기
2020-07-29	1.2.0.0	전용성	3.1 3.2.1 3.2.8 3.2.18 4.2.2.1	• 1.2.0.0 change log 추가 • UTIP 설명 및 예제 추가 • aSIMS ini 설정 예제 추가 • FblMcuSpc58x 설정 추가 • FblCanTrcvSpc58x category 변경 • SPC58x DIO External WDG 추가 • MCU List에 CYT2B98x 추가 • Fbl_SelectEntryPoint 함수 설명에 CYT2B98x 내용 추가 • HSM 2.1.x 이상 버전 사용시 RTSW+FBL 통합 HSM Driver 사용 가능 사항에 대한 내용 추가 • Erase Sector num 설정 추가 • FblEthType 설정 추가
2020-08-11	1.2.1.0	전용성	2.2	• 1.2.1.0 change log 추가
2020-08-27	1.3.0.0	전용성	2.2	• 1.3.0.0 change log 추가
2020-09-10	1.3.1.0	전용성	2.2	• 1.3.1.0 change log 추가
2020-10-13	1.4.0.0	전용성	2.2	• 1.4.0.0 change log 추가
2020-10-19	1.5.0.0	전용성	2.2	• 1.5.0.0 change log 추가
2020-10-26	1.6.0.0	전용성	2.2	• 1.6.0.0 change log 추가
2020-10-28	1.7.0.0	전용성	2.2	• 1.7.0.0 change log 추가
2020-11-09	1.8.0.0	전용성	2.2	• 1.8.0.0 change log 추가 • FblRevisedSecureFlash_2_0 설정 가이드 추가 • SwSigningSkipArea 설정 가이드 추가

4th Edition Date : 2020-02-10	File Name: AUTRON_AUTOSAR_Fbl_UM.pdf	Creation 강신일	Check 윤영진	Approval 백중현
Document Management System		2020-02-10	2020-02-10	2020-02-10

2020-12-01	1.8.1.0	전용성	2.2	1.8.1.0 change log 추가 - Non-Secure flash 설정 설명 추가
2021-01-12	1.8.2.0	전용성	2.2	1.8.2.0 change log 추가
2021-01-22	1.9.0.0	전용성	2.2	1.9.0.0 change log 추가
2021-01-23	1.9.1.0	전용성	2.2	1.9.1.0 change log 추가 - SwModule 설정 변경
2021-02-04	1.9.2.0	전용성	2.2	1.9.2.0 change log 추가
2021-02-07	1.10.0.0	전용성	2.2	1.10.0.0 change log 추가 - Signature Address 설정 추가 - FblCanIfCANFDDiscreteDlcSupport 설정 추가 - API 추가 - Secure flash 적용 시 RTSW size의 ROM 의존성 제거 방법 가이드
2021-02-17	1.11.0.0	전용성	2.2	1.11.0.0 change log 추가 - FblWriteSignature 설정 추가
2021-03-19	1.12.0.0	임재현	ALL	- TV-II-B-E FBL 신규추가 - 보안 관련 설정 pdf 변경에 따른 업데이트
2021-04-07	1.13.0.0	전용성	2.2	1.13.0.0 change log 추가
2021-04-13	1.13.1.0	전용성	2.2	1.13.1.0 change log 추가
2021-05-07	1.14.0.0	전용성	2.2	1.14.0.0 change log 추가 - Secure flash ini 설정 예제 수정 - DET 설정 추가
2021-05-12	1.15.0.0	전용성	2.2	1.15.0.0 change log 추가 - FblDiagMaxNumSignArea 설정 추가 - SwSigningSkipArea 설정 삭제
2021-05-17	1.15.1.0	박순규	2.2 3.2.10	1.15.1.0 change log 추가 - Support Diagnostic Standard 설정 항목 설명 추가
2021-06-15	1.16.0.0	전용성	2.2 3.2.31 3.2.32 3.2.33 3.2.34	1.16.0.0 change log 추가 - GPIO & SPI configuration 추가 - FblCanTrcvCytxxx configuration 삭제 - Section number 재배열
2021-07-05	1.16.1.0	전용성	2.2	1.16.1.0 change log 추가
2021-07-16	1.17.0.0	박성욱	2.2 3.2.1 3.2.6 4.2.2	1.17.0.0 change log 추가 - Support MCU 설정 추가 - FblFlashConfig TC36x 내용 추가 - TC36x 내용 추가
2021-07-19	1.18.0.0	전용성	2.2 3.2.5 3.2.31	1.18.0.0 change log 추가 - Add remark on SPC58 PLL config - SPI configuration 추가 - FBL RAM 영역 관련 limitation 추가
2021-07-30	1.19.0.0	전용성	2.2 3.2.9 3.2.10 3.2.11.1 4.1.4	1.19.0.0 change log 추가 - Signature 발급처 설정 추가 - RoutineControl ID 설정 추가 - SwMemoryBlockAear 설정 추가 - Signature delimiter 관련 설정 추가
2021-09-03	1.20.0.0	박성욱	2.2 3.2.23 3.2.26 3.2.27 3.2.28	1.20.0.0 change log 추가 - CanCell, Node, RXD Changeable 로 변경 - Can MsgFuncId 설정 Changeable로 변경 - Can MsgPhysReqId 설정 Changeable로 변경 - Can MsgPhyResId 설정 Changeable로 변경
2021-10-27	1.21.0.0	전용성	2.2 3.2.4 3.2.9 3.2.10	1.21.0.0 change log 추가 - FblDiagTransferDataChunkMode 설정 추가 - Signature 발급처 설정 삭제 - FblDecompressBufSize 설정 추가

			5.9	- FbiWriteSignatureDelimiter 설정 추가 - 사용자 정의 사항 추가
2021-11-23	1.22.0.0	전용성	2.2 3.2.8 3.2.10	1.22.0.0 change log 추가 - FbiDiagMaxNumOfBlockLength 설정 추가 - FbiSecurityNumAttDelay, FbiSecurityDelayTime 설정 추가
2021-12-22	1.23.0.0	임재현	2.2 3.2 4.2 5.2	- Chanbe Log 수정 - VersionCheck (다운그레이드방지) 기능 관련 설명 추가 (especially 5.10)
2022-06-06	1.23.1.0	최진수	2.2 4.2.2.2 4.2.5 5.10.3.5	1.23.1.0 change log 추가 - MainSW 이외의 update 시 주의사항 추가 - 다운그레이드 방지 API 설명 추가 - 다운그레이드 방지 적용 시 user code 구성 필요 부분 추가

목 차

1. OVERVIEW.....	8
1.1 부트로더 정의	8
1.2 부트로더 특징	8
1.3 STARTUP.....	8
1.4 ECU HARDWARE MANAGEMENT	8
1.5 DIAGNOSIS.....	9
2. PRODUCT RELEASE NOTES	10
2.1 SCOPE OF THE RELEASE	10
2.2 CHANGE LOG.....	10
2.2.1 Version 1.0.0	10
2.2.2 Version 1.1.0	10
2.2.3 Version 1.2.0.0.....	10
2.2.4 Version 1.2.1.0.....	14
2.2.5 Version 1.3.0.0.....	14
2.2.6 Version 1.3.1.0.....	15
2.2.7 Version 1.4.0.0.....	15
2.2.8 Version 1.5.0.0.....	16
2.2.9 Version 1.6.0.0.....	16
2.2.10 Version 1.7.0.0.....	17
2.2.11 Version 1.8.0.0.....	17
2.2.12 Version 1.8.1.0.....	18
2.2.13 Version 1.8.2.0.....	18
2.2.14 Version 1.9.0.0.....	19
2.2.15 Version 1.9.1.0.....	19
2.2.16 Version 1.9.2.0.....	19
2.2.17 Version 1.10.0.0.....	20
2.2.18 Version 1.11.0.0.....	21
2.2.19 Version 1.12.0.0.....	22
2.2.20 Version 1.13.0.0.....	22
2.2.21 Version 1.13.1.0.....	23
2.2.22 Version 1.14.0.0.....	24
2.2.23 Version 1.15.0.0.....	25
2.2.24 Version 1.15.1.0.....	25
2.2.25 Version 1.16.0.0.....	25
2.2.26 Version 1.16.1.0.....	26

2.2.27	Version 1.17.0.0	26
2.2.28	Version 1.18.0.0	26
2.2.29	Version 1.19.0.0	26
2.2.30	Version 1.20.0.0	27
2.2.31	Version 1.21.0.0	27
2.2.32	Version 1.22.0.0	28
2.2.33	Version 1.23.0.0	30
2.2.34	Version 1.23.1.0	30
LIMITATIONS		34
3.	CONFIGURATION GUIDE.....	35
3.1	외부 라이브러리	35
3.2	설정	35
3.2.1	FblGeneral	35
3.2.2	FblMcuSpc58x.....	36
3.2.3	FblMcuCytxxx.....	36
3.2.4	FblOta	36
3.2.5	FblPllConfig	37
3.2.6	FblFlashConfig	38
3.2.7	FblSecurityConfig	39
3.2.8	FblSecurityAccess	39
3.2.9	FblSecureFlash	40
3.2.10	FblDiagConfig	41
3.2.11	SwModule	42
3.2.12	SwVersionInfo	46
3.2.13	FblWatchdogConfig	47
3.2.14	FblEthernetConfig	47
3.2.15	FblEcuIpAddress	48
3.2.16	FblEcuMacAddress	48
3.2.17	FblEcuTcpPort	48
3.2.18	FblEthDiagLocalAddress	48
3.2.19	FblVlan	49
3.2.20	FblEthType	50
3.2.21	FblCanConfig	50
3.2.22	FblCanIfConfig	50
3.2.23	FblCanDrvTc3xx	51
3.2.24	FblCanDrvSpc58x and FblCanDrvRh850f1km	51
3.2.25	FblCanDrvCytxxx	52

3.2.26	FblCanDiagMsgFuncConfig	52
3.2.27	FblCanDiagMsgPhysReqConfig	53
3.2.28	FblCanDiagMsgPhysResConfig	53
3.2.29	FblCanBaudRateConfig	53
3.2.30	FblCanFdBaudRateConfig	54
3.2.31	FblSpiControl	54
3.2.32	FblGpioPortPinControl	56
3.2.33	FblCanTrcvSpc58x	56
3.2.34	FblCanTrcvRh850f1km	57
4.	APPLICATION PROGRAMMING INTERFACE (API)	58
4.1	TYPE DEFINITIONS	58
4.1.1	Entry Point	58
4.1.2	압축 해제	58
4.1.3	Public Key	59
4.1.4	Delimiter	59
4.2	FUNCTIONS	60
4.2.1	External Devices	60
4.2.2	Entry Point	62
4.2.3	압축 해제	67
4.2.4	보안라이브러리	69
4.2.5	다운그레이드 방지	76
5.	APPENDIX	77
5.1	ST_MIN (SEPERATION TIME MINIMUM)	77
5.2	SECURITY KEY	77
5.3	SW VERSION INFO	77
5.4	UTIP	79
5.4.1	UTIP 설명	79
5.4.2	UTIP 예제	81
5.5	ASIMS INI 설정 예제 (SECUREFLASH)	83
5.5.1	SecureFlash 1.0	83
5.5.2	SecureFlash 2.0	83
5.6	보안 기능	84
5.6.1	HSAC	84
5.6.2	CSAC	85
5.6.3	Secure Flash	85
5.7	SPC58X DIO EXTERNAL WDG	86
5.8	SECURE FLASH 적용 시 RTSW SIZE의 ROM 의존성 제거 방법 가이드	87
5.9	사용자 정의 사항	88

5.10	SOFTWARE VERSION CHECK (다운그레이드방지) 기능 정보	89
5.10.1	Software Version Check 기능이란	89
5.10.2	Structure of the Version Block	90
5.10.3	어플리케이션 유저 Software Version Check 기능 사용 가이드 예시	91
6.	OPEN SOURCE COPYRIGHT.....	96
6.1	LWIP (LIGHT WEIGHT IP)	96

1. Overview

1.1 부트로더 정의

부트로더는 리셋시 항상 수행되는 코드

- 일반적으로 지워지지 않음
- ECU startup 정책이 구현되어 있음
- 최소한의 size 유지
- ECU 를 안전하게 업데이트 하기 위한 리프로그래밍 알고리즘을 가지고 있음

1.2 부트로더 특징

RTSW와 최소한의 interaction만 유지

- FBL와 RTSW 간에 interrupt나 RAM을 공유하지 않음
- RTSW 시작시 FBL은 항상 같은 상태를 유지

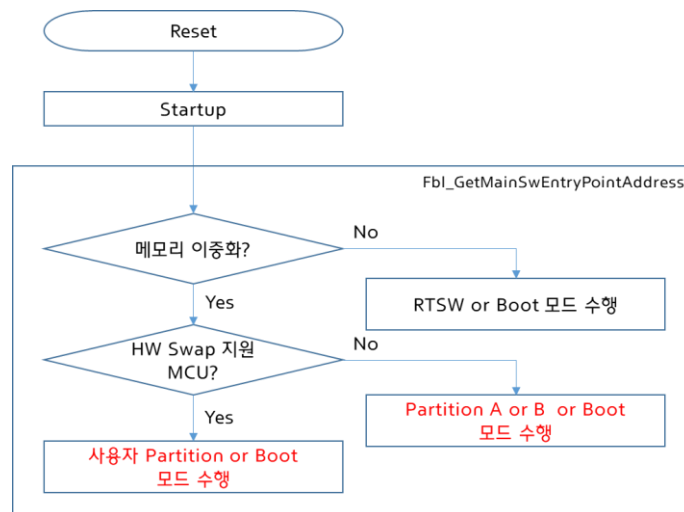
강건성

- 잘못된 부트모드 진입 감지
- 부트모드에서 프로그램의 정지 상태 감지

1.3 Startup

Startup 시 FBL 은 이중화 기능 적용시 Fbl_GetMainSwEntryPointAddress 함수가 호출되며 start address 를 리턴받아 RTSW 펌웨어(Partition A or B) 혹은 Boot 모드를 진입한다.

이중화가 미적용된 경우 FBL 은 사전에 정의된 Security Key 를 체크하여 유효한 Security Key 가 있을 경우 RTSW 진입, Security Key 가 없을 경우 Boot 모드를 진입한다.



1.4 ECU Hardware Management

Startup 시의 Default behaviour :

- IO 포트들은 초기화가 되지 않은 상태로 유지
- 내부/외부 watchdog 모두 초기화 되지 않은 상태 유지

Boot Mode 에서의 Default behaviour:

- IO 포트들은 초기화가 되지 않은 상태로 유지
- 내부 watchdog은 초기화

1.5 Diagnosis

Features

- UDS 리프로그래밍을 위한 사양을 만족

2. Product Release Notes

2.1 Scope of the release

이 문서에 대한 모든 내용은, 다음의 현대오트론 Fbl 모듈에 한정한다.

Module	Module version
Fbl	1.23.1.0

2.2 Change Log

2.2.1 Version 1.0.0

➤ 신규 기능

■ 최초 모듈 릴리즈

원인	TC3XX 타겟용 공용 FBL 모듈 릴리즈
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	API 사용 가이드 참조

2.2.2 Version 1.1.0

➤ 신규 기능

■ CAN 지원

원인	CAN 진단 지원
동작 영향	없음
설정 영향	CAN FBL 설정 필요
ASW 조치 필요 사항	없음

■ Secure Flash 2.0 지원

원인	TC3XX Secure Flash 2.0 기능 구현
동작 영향	없음
설정 영향	FblSecureFlash 설정 변경, Secure Flash 2.0 적용 펌웨어, Secure Flash 2.0 적용 리프로그래밍 툴(진단기) 필요
ASW 조치 필요 사항	FBL 프로젝트에 오토에버 HSM 2.0 펌웨어 & 드라이버(Polling 방식) 적용 필요

2.2.3 Version 1.2.0.0

➤ 신규 기능

■ Secure Access 2.0 지원

원인	Secure Access 2.0 기능 구현
----	-------------------------

동작 영향	없음
설정 영향	FblSecureAccess 설정 적용
ASW 조치 필요 사항	없음

■ SPC58x CAN 지원

원인	SPC58x CAN 지원
동작 영향	없음
설정 영향	SPC58x 설정 필요
ASW 조치 필요 사항	없음

■ RH850F1KM 모듈과 통합 작업

원인	RH850F1KM 모듈과 통합 작업 필요
동작 영향	없음
설정 영향	F1KM MCU 설정 적용
ASW 조치 필요 사항	없음

■ 서브 MCU OTA 기능 개발

원인	서브 MCU OTA 기능 개발 필요
동작 영향	서브 MCU 의 Image 들을 제어기 플래시메모리에 저장하여 서브 MCU 들을 Reprogramming 한다
설정 영향	서브 MCU 추가 시 ModuleInformation 설정
ASW 조치 필요 사항	없음

➤ 기능 개선

■ Public Key 사용자 변경 지원

원인	Secure Flash 의 서명 생성시 사용되는 key pair(aSIMS, FST) 에 따라 public key 가 변경되어야 한다. 서명 생성시에 사용된 private key 대응 되는 public key 를 사용자가 직접 적용할 수 있도록 user code 영역으로 public key 를 이동
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	FST 틀을 사용하여 서명 생성시 사용자가 integration_Code 내의 Fbl_SecData.c 파일의 public key(Sec_Gau8_ImageServPubKey) 를 변경한다.

■ CSAC 사용시 programming session 천이 오류

원인	CSAC 사용시 Dcm 의 Security Access 성공 정보의 Fbi 전달시 오류로 reprogramming 실패 발생
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ 오픈 소스 copyright 명기

원인	lwIP 오픈 소스 copyright 명기
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ SPC58X Quartz clock 설정 제약 삭제

원인	SPC58X Quartz clock 설정 제약 삭제
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ RH850F1KM HSM FLMD0 protection 정지 후 FLMD0 제어

원인	RH850F1KM HSM FLMD0 protection 정지 후 FLMD0 제어
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ SPC58X

원인	SPC58X 의 MCU Name 설정 추가
동작 영향	없음
설정 영향	SPC58X 의 MCU Name 설정 가능
ASW 조치 필요 사항	없음

■ SPC58X Transceiver 0 번 pin 설정 가능하도록 Generator 수정

원인	SPC58X Transceiver 0 번 pin 설정 가능하도록 Generator 수정
동작 영향	없음
설정 영향	SPC58X Transceiver 0 번 pin 설정 가능

ASW 조치 필요 사항	없음
--------------	----

■ SPC58X CAN Extended ID 동작 개발

원인	SPC58X CAN Extended ID 개발 필요
동작 영향	SPC58X CAN Extended ID 로 reprogramming 가능
설정 영향	SPC58X CAN Extended ID 설정 가능
ASW 조치 필요 사항	없음

■ CYT2B98x 지원

원인	CYT2B98x 지원
동작 영향	X
설정 영향	FblSupportMcu 에 CYT2B98X 로 설정 가능
ASW 조치 필요 사항	없음

■ HSM Interrupt Disable API 호출 추가

원인	Autoever HSM 2.1 이상 버전에서 RTSW 에서 FBL 로 전환 시 HSM Interrupt Disable API 호출 필수
동작 영향	X
설정 영향	X
ASW 조치 필요 사항	없음

■ MDC MDIO 포트 등 EthTrcv 관련 변경 가능하도록 구현

원인	EthSwt 적용 제어기의 경우, EthTrcv 관련로직 수행되지 않도록 옵션 적용 필요
동작 영향	없음
설정 영향	FblEthType 설정 변경
ASW 조치 필요 사항	Fbl_Applf.c 에 user code 구현 필요

■ EthFBL 1Gbit RGMII 지원

원인	RGMII 1Gbit 옵션으로 지원하도록 수정
동작 영향	없음
설정 영향	FblEthSpeed 및 register 관련 설정 변경
ASW 조치 필요 사항	없음

■ EraseMemory 시 Erase 영역의 Sector number 설정 가능하도록 수정

원인	EraseMemory 시 Erase 영역의 size 를 조절할 수 없음
----	---

동작 영향	없음
설정 영향	FblEraseSectorNum 설정 변경
ASW 조치 필요 사항	없음

■ HSAC 사용하고 Secure Flash 사용하지 않는 경우 동작 지원

원인	HSAC 사용하고 Secure Flash 사용하지 않는 경우 checkProgrammingDependencies 통과하지 못한다. FBL_USE_SECURE_FLASH 설정에서 가져오도록 변경
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ Security Access 요청 시 Pending response 만 무한 반복하는 현상 수정

원인	Security Access Seed request 에 대하여 Pending response 만 무한 반복하는 현상 발생
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ TC3xx Reprogramming 중 non cached address space 도 저장 가능하도록 수정

원인	TC3xx Reprogramming 중 non cached address space 도 저장 필요
동작 영향	non cached address space 저장 가능
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.4 Version 1.2.1.0

➤ 신규 기능

■ TC23x HSM 2.0 지원 개발

원인	TC23x HSM 2.0 지원 필요
동작 영향	TC23x HSM 2.0 지원
설정 영향	Security Access, Secure Flash 설정
ASW 조치 필요 사항	없음

2.2.5 Version 1.3.0.0

➤ 신규 기능

■ TC29x HSM 2.0 지원 개발

원인	TC29x HSM 2.0 지원 필요
동작 영향	TC29x HSM 2.0 지원
설정 영향	Security Access, Secure Flash 설정
ASW 조치 필요 사항	없음

2.2.6 Version 1.3.1.0

➤ 기능 개선

■ Fbl_SelectEntryPoint 가 2 번 호출되어 TraveoII 의 OTA SWAP 기능 동작시에 불필요한 코드 증가

원인	Fbl_SelectEntryPoint 가 2 번 호출되어 TraveoII 의 OTA SWAP 기능 동작시에 불필요한 코드 증가 및 부팅 시간 증대
동작 영향	Fbl_SelectEntryPoint 가 1 번만 호출되도록 수정
설정 영향	없음
ASW 조치 필요 사항	1 번만 호출되는 것을 고려하여 OTA SWAP 기능 구현

➤ 오류 수정

■ Chorus Sub-MCU 의 Reprogramming 할때, 특정 ID Block 삭제시 Main MCU 까지 삭제되는 문제

원인	Chorus Sub-MCU 의 Reprogramming 할때, RoutineControlOptionRecord 이용하여 특정 ID Block 삭제시 RTSW Block 까지 삭제되는 문제 발생
동작 영향	특정 ID Block 만 삭제되도록 수정
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.7 Version 1.4.0.0

➤ 오류 수정

■ TransferData 단계에서 MetaData 2byte 가 분리되어 전송되는 문제

원인	BlockLength 의 처리를 잘못하여 MetaData 2byte 가 분리되어 전송되는 문제 발생
동작 영향	Secure Flash 2.0 진행 시 Invalid Response 발생하는 오류 방지
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 신규 기능

■ ES95486-02 UDS 사양 개정에 따라 CheckProgramDependency 와 EraseTargetArea 기능 개발

원인	ES95486-02 UDS 사양 개정에 따라 CheckProgramDependency 와 EraseTargetArea 기능 개발 필요
동작 영향	CheckProgramDependency 와 EraseTargetArea 기능 동작 시

	MemoryArea 와 SWUnitMemoryMemory 을 고려하여 동작함
설정 영향	OTA Enable 과 ECUSoftwareUNIT 설정 필요
ASW 조치 필요 사항	OTA 사용 시 ES95486-02 UDS 사양 개정내용 적용 필요

➤ 신규 기능

■ TC3XX Ethernet FBL 의 Secure Flash 2.0 기능 지원 개발

원인	TC3XX Ethernet FBL 의 Secure Flash 2.0 기능 지원 개발 필요
동작 영향	TC3XX Ethernet FBL 의 Secure Flash 2.0 기능 동작함
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 기능 개선

■ SwModule 추가 시에 library 의존성 제거

원인	SwModule 추가 시에 library 의존성 제거 필요
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 기능 개선

■ ERT 진단아이디 0x1000 외에 CCU 진단아이디 0x5D1 도 동시 적용

원인	ERT 진단아이디 0x1000 외에 CCU 진단아이디 0x5D1 도 동시 적용 필요
동작 영향	ERT 진단아이디 0x1000 외에 CCU 진단아이디 0x5D1 도 동시 적용 가능
설정 영향	CCU 진단아이디 0x5D1 설정 필요
ASW 조치 필요 사항	없음

2.2.8 Version 1.5.0.0

➤ 신규 기능

■ Secure Flash Signature 전송을 RequestDownload 로 처리하도록 지원

원인	Secure Flash Signature 전송을 RequestDownload 로 처리하기위한 기능 개발 필요
동작 영향	RequestDownload 서비스로 Signature 전송 가능
설정 영향	FblDiagSignatureTxMode 설정 필요
ASW 조치 필요 사항	없음

2.2.9 Version 1.6.0.0

➤ 신규 기능

■ OTA 사양의 OTA_Ready RID 개발

원인	OTA 시에 리프로그래밍 실패로 인한 어플리케이션 소프트웨어 및 데이터가 불완전한 경우에도 RoutineControl-OTA Ready 서비스를 사용할수 있어야 한다.
동작 영향	RoutineControl-OTA Ready 서비스를 사용 가능
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ Consecutive frame 수신 시에 40ms 이 초과하면 전송 실패하는 문제

원인	Consecutive frame 수신 시에 40ms 이 초과하면 진단 Timeout 이 발생하여 전송이 실패한다.
동작 영향	Consecutive frame 수신 시에 1 초 timeout 적용됨
설정 영향	없음
ASW 조치 필요 사항	없음

■ TransferData 송신 중 실패 발생 시(Bus Off) Pending 발생하는 문제

원인	TransferData 송신 중 실패 발생 시(Bus Off) 바로 리프로그래밍 준비 상태가 되지 않고 Pending 응답을 하는 문제
동작 영향	TransferData 송신 중 실패 발생 시(Bus Off) 바로 리프로그래밍 준비 상태(Reset)로 전환된다.
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.10 Version 1.7.0.0

➤ 신규 기능

■ TC2xx 에서 사용되는 모든 CAN port 를 사용할 수 있도록 지원 개발

원인	TC2xx에서 사용되는 모든 CAN port를 사용할 수 있도록 지원 필요
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.11 Version 1.8.0.0

➤ 신규 기능

■ Secure Flash 2.0 개정 사양 적용

원인	Secure Flash 2.0 개정 사양 적용 필요
----	------------------------------

동작 영향	없음
설정 영향	FblRevisedSecureFlash_2_0 설정 필요
ASW 조치 필요 사항	없음

- Secure Flash의 signing 영역에 flashing 하는 모든 영역이 포함되도록하고 Security level 이 Seedkey 일 경우에만 CRC 검사하도록 개발

원인	Secure Flash의 signing 영역에 flashing하는 모든 영역이 포함되도록하고 Security level이 Seedkey일 경우에만 CRC 검사 필요
동작 영향	없음
설정 영향	SW signing skip area 설정 필요
ASW 조치 필요 사항	없음

➤ 기능 개선

- OTA Ready 서비스를 모든 session 과 Security level 에 상관없이 동작하도록 개선

원인	OTA Ready 서비스를 모든 session 과 Security level 에 상관없이 동작하도록 개선 필요
동작 영향	모든 session 과 Security level 에 상관없이 OTA Ready 서비스 긍정응답
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.12 Version 1.8.1.0

➤ 오류 수정

- EthFBL Retransmission 시에 50ms 이내에 pending response 를 보내도록 처리

원인	TrasferData frame 전송 시에 Tester는 이미 Packet을 다 보냈으나 제어기에서 loss가 발생할 경우 Tester가 ACK를 받지 못한 Packet에 대해 Retransmission 요청을 하게 되는데 Retransmission 요청이 되기 전 50ms이 초과하면 connection이 끊기므로 그 전에 제어기에서 pending 메시지를 보내야 한다.
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.13 Version 1.8.2.0

➤ 기능 개선

- EthFBL TransferData 수신 시 Packet loss 로 인한 시간 지연 개선

원인	TC39x EthFBL TransferData 수신 시 Packet loss로 인한 시간 지연을 개선하기 위하여 TCP Window size를 변경
동작 영향	Packet 손실로 인한 시간 지연 개선
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.14 Version 1.9.0.0

➤ 오류 수정

- RTSW block 이외 block 에 SwSigningSkipArea 설정 시, RTSW 의 Reprogramming Fail 수정

원인	SwSigningSkipArea 기능이 RTSW block만 적용하여 구현 됨
동작 영향	RTSW block 이외 block 에 SwSigningSkipArea 설정 가능
설정 영향	RTSW block 이외 block 사용 시 SwSigningSkipArea 설정 필요
ASW 조치 필요 사항	없음

➤ 오류 수정

- FBL memory swap enable 설정 시 Build error 발생

원인	FBL memory swap enable 설정 시, 변수의 extern 선언 타입이 달라서 Build error 발생
동작 영향	FBL memory swap enable 설정 시 Build 성공
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.15 Version 1.9.1.0

➤ 오류 수정

- Romtst 모듈 호환성 문제 수정

원인	Romtst 모듈 사용 시에 FBL과 호환성없이 설계됨
동작 영향	Romtst 영역을 설정하여 Rom test 정상동작
설정 영향	SwModule 설정 시 Romtst 영역 설정 필요
ASW 조치 필요 사항	RTSW block 이외의 리프로그래밍 시에 Romtst block 을 제외하고 사용필요

2.2.16 Version 1.9.2.0

➤ 오류 수정

- 하나의 Tcp Packet 에 여러개의 EthDiag Frame 이 포함되어 수신되는 경우 Trap 발생

원인	하나의 Tcp Packet에 여러개의 EthDiag Frame이 포함되어 수신되는 경우 처리하는 로직이 없어 Trap이 발생함
동작 영향	하나의 Tcp Packet 에 여러개의 EthDiag Frame 이 포함되어 수신되는 경우

	Trap 발생 수정
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.17 Version 1.10.0.0

➤ 신규 기능

■ FTD(실시간 개조 감지) 기능 지원 개발

원인	FTD(실시간 개조 감지) 기능 지원 개발
동작 영향	FTD(실시간 개조 감지) 기능 지원
설정 영향	Signature 를 write 하기 위해 Signature Address 설정과 RTSW EndAddress 변경 필요함
ASW 조치 필요 사항	Fbl_BeforeRoutineCtrlEraseInit, Fbl_AfterRoutineCtrlChkInit CallOut 을 활용하여 FTD TempStop 과 같은 기능 구현 필요함

➤ 기능 개선

■ SecureFlash 적용 상태에서 롬 크기 변경 불가능한 제약사항 개선

원인	SecureFlash 적용 상태에서 롬 크기 변경 불가능한 제약사항 개선 필요
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	Signature 위치를 RTSW 내부로 변경하기 위해서 LD 파일내에서 조정 필요함

➤ 신규 기능

■ CANFD Discrete Dlc 설정 기능 추가

원인	CANFD 64BYTE 지원을 위하여 CANFD Discrete Dlc ON/OFF설정 기능 필요
동작 영향	CANFD 64BYTE Discrete DLC 계산 지원
설정 영향	CANFD Discrete Dlc ON/OFF 설정 추가
ASW 조치 필요 사항	없음

➤ 신규 기능

■ MCU Specific 기능 초기화 직전 호출되는 Fbl_BeforeFblSpecificInit 유저함수 추가

원인	MCU Specific 기능 초기화 직전 호출되는 Fbl_BeforeFblSpecificInit 유저함수 추가 필요
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ EthDiag 진단 Frame 이 분할되서 들어오는 Packet 에 대한 처리

원인	EthDiag 진단 Frame이 분할되서 들어오는 Packet에 대한 처리가 되어 있지 않음
동작 영향	EthDiag 진단 Frame 이 분할되서 들어오는 Packet 에 대한 처리
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.18 Version 1.11.0.0

➤ 오류 수정

■ Default Session, ECU_Reset 서비스가 suppressPosRspBit 와 함께 요청될 경우 통신 중단 문제

원인	Default Session, ECU_Reset 서비스가 suppressPosRspBit 와 함께 요청될 경우 처리하는 로직이 없어서 통신이 중단됨
동작 영향	Default Session, ECU_Reset 서비스가 suppressPosRspBit 와 함께 요청될 경우 처리 가능
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ Signature 를 TXDATA 로 전송할 경우 간헐적으로 전송 실패 문제 수정

원인	Signature block인지 확인하는 로직에서 Queue의 Length가 8Byte 이상일 경우에 대한 처리가 누락됨
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 신규 기능

■ Signature 를 flash 에 저장하는 기능 On/Off 설정 추가

원인	Signature를 flash에 저장하는 기능의 On/Off 설정을 추가하여 사용자가 선택할 수 있도록 지원 필요
동작 영향	설정에 따라서 Signature 저장 선택 가능
설정 영향	FbiWriteSignature
ASW 조치 필요 사항	없음

2.2.19 Version 1.12.0.0

➤ 기능 개선

- FBL common 모듈 Crypto 라이브러리 호출 방식 변경에 따른 Crypto 인터페이스 구조 변경

원인	보안라이브러리와 FBL 모듈간 의존성 제거 및 Third Party 보안라이브러리 지원을 위해 FBL common 모듈 Crypto 라이브러리 호출 방식 변경 필요
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 기능 개선

- HSM 적용 시 난수성 개선을 위한 HSM PRNG (1 회 TRNG 이후 PRNG) 적용

원인	HSM 적용 시 난수성 개선을 위한 때 TRNG 호출에서 1회 TRNG 이후 PRNG 적용이 필요
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 신규 기능

- Cypress TV-II-B-E CAN 리프로그래밍 지원

원인	Cypress TV-II-B-E CAN 리프로그래밍 설정 및 호환 지원
동작 영향	TV-II-B-E CAN, CANFD 리프로그래밍 가능
설정 영향	FblSupportMcu, FblMcuCytxxx
ASW 조치 필요 사항	없음

2.2.20 Version 1.13.0.0

➤ 오류 수정

- Pending message 를 전송 후 BUS 가 점유될 경우 간헐적으로 Security Access Fail 발생 문제 수정

원인	FBL에서 Pending message 전송하려는 동안 우선순위가 높은 CANID의 메시지들이 BUS를 점유하여 Confirmation이 늦게 올라와서 긍정응답을 못함
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ Multi-frame 수신 중 SF(functional)를 사양과 다르게 처리하는 문제 수정

원인	Physical Multi-frame(CF) 수신 중 Functional SF 들어오면 Physical CF은 계속 수행 하고 Functional SF은 reject 하지 못함
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ Multi-frame 수신 중 SF(physical)를 사양과 다르게 처리하는 문제 수정

원인	Physical Multi-frame(CF) 수신 중 Physical SF 들어오면 Segmented Reception(CF)을 종료시키고, SF 에 대하여 수신을 처리하지 못함
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ Queue 약의 시험 중 Queue 가 clear 가 되지 않아 먹통이 되는 문제 수정

원인	Physical Multi-frame(CF)와 SF들을 무작위로 Queue에 쌓는 시험 중에 Queue 동작이 꼬여서 통신 먹통이 되는 문제
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.21 Version 1.13.1.0

➤ 오류 수정

■ Security unlock 상태에서 seed 요청 시 67 로만 응답하는 문제 수정

원인	Security unlock 상태에서 seed 요청 시 Response length를 0으로 하여서 67로만 긍정 응답함.
동작 영향	보안이 해제된 상태에서, 'request Seed' 메시지를 요청 받았을 때, 'Seed' 값의 모든 바이트를 '0' 으로 하여서 긍정 응답함.
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.22 Version 1.14.0.0

➤ 오류 수정

■ 0x7DF 로 ECU_Reset(0x11) service 요청 시 무응답하는 문제 수정

원인	ECU_Reset, SecurityAccess, RequestDownload, TransferData, RequestTransferExit, RoutineControl 서비스들은 Physical CANID 요청에만 긍정응답하도록 구현됨.
동작 영향	ECU_Reset, SecurityAccess, RequestDownload, TransferData, RequestTransferExit, RoutineControl 서비스들이 Physical, Functional CANID 요청 모두 정상동작
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ 설정되지 않은 Security Level 의 Seed 요청에도 긍정 응답하는 문제 수정

원인	설정되지 않은 Security Level의 Seed 요청에도 긍정응답하도록 구현됨.
동작 영향	설정된 Security Level 의 Seed 요청에 한해서만 긍정응답하며 설정되지 않은 Level 의 Seed 요청에 대해서는 NRC12 로 부정응답 처리.
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 신규 기능

■ 전자서명 무결성 실패 시 NRC33 에서 NRC72 로 변경

원인	사양 변경에 따라서 전자서명 무결성 실패 시에 NRC72로 부정응답해야 함
동작 영향	전자서명 무결성 실패 시에 NRC72 로 부정응답
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 신규 기능

■ FBL 디버깅을 위한 DET 추가

원인	FBL 디버깅을 위한 DET 추가
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 신규 기능

■ TC3xx CAN Node 3 번 지원 개발

원인	TC3xx CAN Node 3 번 지원 개발 필요
동작 영향	TC3xx CAN Node 3 번 사용 가능
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 신규 기능

■ CYT4BX 를 위한 CAN Node 4 번 추가

원인	CYT4BX 를 위한 CAN Node 4 번 추가 필요
동작 영향	CYT4BX CAN Node 4 번 사용 가능
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.23 Version 1.15.0.0

➤ 신규 기능

■ Calibration 영역도 전자서명 검증에 포함되도록 수정

원인	Calibration 영역도 전자서명 검증에 포함되어야 함
동작 영향	Calibration 영역도 전자서명 검증 가능
설정 영향	FblDiagMaxNumSignArea 설정 추가 SwSigningSkipArea 설정 삭제
ASW 조치 필요 사항	1. Module 업데이트 전 SwSigningSkipArea 설정 삭제 2. FblDiagMaxNumSignArea 설정

2.2.24 Version 1.15.1.0

➤ 신규 기능

■ ES95486-00 사양일 경우 ECU Programming Mode 및 Extended Diagnostic Mode 지원

원인	ECU Programming Mode 및 Extended Diagnostic Mode 지원 필요
동작 영향	ECU Programming Mode 및 Extended Diagnostic Mode Request에 대한 Response
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.25 Version 1.16.0.0

➤ 신규 기능

■ CYT SPI protocol 설정 추가

원인	CYT CAN transceiver 를 제어하기 위한 SPI protocol 설정 추가 필요
동작 영향	CYT MCU에서 SPI protocol 제어 가능
설정 영향	SPI protocol 을 제어하기 위한 공통된 parameter 들을 설정 가능
ASW 조치 필요 사항	없음

2.2.26 Version 1.16.1.0

➤ 오류 수정

■ Ethernet 리프로그래밍 TranferData 동작 중 통신 단락 시 간헐적으로 통신 불가능 현상이 발생하는 문제 수정

원인	1) TranferData 중 통신 단락되면 data을 받기 위해 Pending msg 전송함 2) Pending max에 도달하면 General Reject 부정응답 송신 3) General Reject 송신 시에 TransferData 통신에 사용된 전역변수 초기화를 누락함
동작 영향	TransferData 도중 통신 단락 이후에 리프로그래밍 재시도 성공
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.27 Version 1.17.0.0

➤ 신규 기능

■ TC36x CAN Reprogramming 및 동작 지원

원인	TC36x CAN Reprogramming 및 동작 필요
동작 영향	TC36x CAN Reprogramming 및 동작 가능
설정 영향	FblSupportMcu 에서 TC36x 설정 가능
ASW 조치 필요 사항	없음

2.2.28 Version 1.18.0.0

➤ 신규 기능

■ SPI protocol setting 개선 : SPI PortAlphabet container 추가

원인	SPC58X SPI Port 설정(alphabet) 지원 필요
동작 영향	SPC58X SPI Port 설정 가능
설정 영향	SPI configuration PortAlphabet
ASW 조치 필요 사항	없음

2.2.29 Version 1.19.0.0

➤ 신규 기능

■ ES95486-02 사양의 EraseTargetArea (ECU with/without memory redundancy) 신규 사양 적용

원인	OTA 이중화 사용 시 ES95486-02 사양의 EraseTargetArea (ECU
----	--

	with/without memory redundancy) 기능 지원 필요
동작 영향	EraseTargetArea (ECU with memory redundancy) 신규 사양 적용
설정 영향	FblOTAEnable, FblMemorySwapEnable, FblDiagEraseRID, SwMemoryBlockArea
ASW 조치 필요 사항	없음

➤ 오류 수정

■ ES95489-52 사양에 따라 전자서명 Delimiter 값의 준수 여부를 확인하지 못하는 문제 수정

원인	전자서명 Delimiter의 data 오염 여부를 확인하지 못함
동작 영향	전자서명 Delimiter의 data 오염되었을 경우 NRC72 부정응답
설정 영향	FblSignatureMadeBy
ASW 조치 필요 사항	없음

2.2.30 Version 1.20.0.0

➤ 신규 기능

■ CAN ID 및 NODE 설정을 변경하여 사용할 수 있도록 수정

원인	CAN ID 및 NODE 설정 변경 되지 않게 설계되어 있음.
동작 영향	CAN ID 및 NODE 설정 변경하여 사용할 수 있도록 수정
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.31 Version 1.21.0.0

➤ 신규 기능

■ OTA 압축 기능 개발

원인	OTA 압축된 펌웨어 리프로그래밍 지원 필요
동작 영향	압축된 펌웨어로 리프로그래밍 가능
설정 영향	압축관련 설정
ASW 조치 필요 사항	압축해제 API 작성

■ 동일한 blockSequenceCounter 를 갖는 TransferData 서비스 재시도 지원 개발

원인	TransferData 서비스 처리 실패 시 동일한 blockSequenceCounter를 갖는 TransferData를 재수신 받아서 처리 필요
동작 영향	1) 정상적으로 TransferData 수신 받았고 Server에서 긍정응답 보냈지만 Client가 긍정응답 수신 받지 못하여 Client가 동일한 blockSequenceCounter로 TransferData 재전송 처리 가능

	2) Server가 정상적으로 TransferData 수신받지 못해서 긍정응답을 하지 않고 Client가 동일한 blockSequenceCounter로 TransferData 재전송 처리 가능
설정 영향	FblDiagTransferDataChunkMode false 설정
ASW 조치 필요 사항	없음

■ 서명 저장 시에 Delimiter 영역도 저장하도록 개발

원인	서명 저장 시에 Delimiter data도 flash에 저장
동작 영향	서명 저장 시에 Delimiter data도 flash에 저장 가능
설정 영향	FblWriteSignatureDelimiter 설정
ASW 조치 필요 사항	없음

➤ 오류 수정

■ MCfg_GetLogicMemBlkInfo 함수 선언문제로 Build 실패하는 문제 수정

원인	1.19.0 버전에서 발생한 문제로 Cypress MCU에서 MCfg_GetLogicMemBlkInfo 함수 선언문제로 Build 실패하는 문제 발생
동작 영향	없음
설정 영향	없음
ASW 조치 필요 사항	없음

■ Delimiter 검증 시에 Byte order 가 Little Endian 경우가 고려되지 않아서 리프로그래밍 실패 수정

원인	1.19.0 버전에서 발생한 문제로 Little Endian을 사용하는 MCU에서 Delimiter 검증 통과하지 못하는 문제 발생
동작 영향	Little Endian을 사용하는 MCU에서 Delimiter 검증 가능
설정 영향	없음
ASW 조치 필요 사항	없음

2.2.32 Version 1.22.0.0

➤ 신규 기능

■ Security Lock 관련 사양 개정 개발

원인	- Lock Time 10s에서 180s로 변경 - False attempts 2회에서 3회로 변경
동작 영향	Lock Time과 False attempts를 설정하여 동작
설정 영향	FblSecurityNumAttDelay FblSecurityDelayTime
ASW 조치 필요 사항	없음

■ OTA 적용 CAN/CANFD 적용 제어기는 maxNumberOfBlockLength 를 514Byte 이하로 설정 가능하도록 개발

원인	Receive Buffer와 maxNumberOfBlockLength의 설정이 연동되어 있어서 Receive Buffer를 514Byte이하로 설정할 경우 Secure access의 인증서 전송이 불가능
동작 영향	OTA 적용 CAN/CANFD 적용 제어기는 maxNumberOfBlockLength를 514Byte 이하로 설정하더라도 Secure access의 인증서 전송이 가능함
설정 영향	FblDiagMaxNumOfBlockLength 설정 필요
ASW 조치 필요 사항	없음

■ Ethernet 리프로그래밍 시에 동일한 blockSequenceCounter 를 갖는 TransferData 서비스 재시도 지원 개발

원인	Ethernet 리프로그래밍 시에 동일한 blockSequenceCounter를 갖는 TransferData 서비스 재시도 지원 필요
동작 영향	1) 정상적으로 TransferData 수신 받았고 Server에서 긍정응답 보냈지만 Client가 긍정응답 수신 받지 못하여 Client가 동일한 blockSequenceCounter로 TransferData 재전송 처리 가능 2) Server가 정상적으로 TransferData 수신받지 못해서 긍정응답을 하지 않고 Client가 동일한 blockSequenceCounter로 TransferData 재전송 처리 가능
설정 영향	FblDiagTransferDataChunkMode false 설정
ASW 조치 필요 사항	없음

■ RTSW 에서 Programming session 요청에 의해 FBL 로 jump 할 경우 ISO 14229-1 사양에 따라 programming session 으로 시작하도록 개발

원인	RTSW에서 Programming session 요청에 의해 FBL로 jump 할 경우 ISO 14229-1 사양에 따라 programming session으로 시작하도록 개발 필요
동작 영향	RTSW에서 Programming session 요청에 의해 FBL로 jump 할 경우 programming session으로 시작
설정 영향	없음
ASW 조치 필요 사항	없음

■ TC2xx BlockSeqCount 재전송 지원 개발

원인	TC2xx MCU BlockSeqCount 재전송 지원 개발 필요
동작 영향	TC2xx MCU BlockSeqCount 재전송 지원
설정 영향	FblDiagTransferDataChunkMode 를 false 로 설정
ASW 조치 필요 사항	없음

2.2.33 Version 1.23.0.0

➤ 신규 기능

■ 펌웨어 다운그레이드 방지 기능 추가

원인	소프트웨어 펌웨어 다운그레이드 방지 신규 개발
동작 영향	FblSwVersionCheck 기능을 true로 설정할 시, 현재 플래시 되어있는 버전보다 더 낮은 버전의 펌웨어가 플래시되면 RoutineControl Check 단계에서 0x72 부정응답을 함
설정 영향	/AUTRON/Fbl/FblGeneral/FblSoftwareVersionCheck /AUTRON/Fbl/ModuleInformation/SwModule/BlkHeaderAddress /AUTRON/Fbl/ModuleInformation/SwModule/BlkTrailerAddress
ASW 조치 필요 사항	소프트웨어 펌웨어 다운그레이드 방지를 위해 리프로그래밍 될 S-Record는 FBL이 제안하는 펌웨어 구조로 구성해야 함. 상세하게는, RTSW 컴파일 시 버전 정보를 갖는 별도 구조의 Header / Trailer 블록을 펌웨어의 앞 뒤로 구성해주어야 함. S19 생성 및 Signed s19 생성 방법도 달라짐. (UM 5.10 절 참조)

■ 무결성검증 실패 시 펌웨어 Erase 수행하도록 기능 추가

원인	무결성 검증 실패 시 펌웨어 삭제 요구사항 반영
동작 영향	무결성 검증 실패 시 다운로드한 펌웨어가 삭제됨
설정 영향	N/A
ASW 조치 필요 사항	N/A

2.2.34 Version 1.23.1.0

➤ 오류 수정

■ CanTP 관련 Timeout 200ms 로 설정되는 문제 수정

원인	FBL CanTP 관련 timeout 이 사양 상 1000ms 이어야 하나, CanTp_Mainfunction 이 1ms Task -> 200us Task 로 옮겨지고 관련 로직이 수정되지 않아서 timeout 이 200ms 로 적용되는 문제
동작 영향	N_Cr, N_Ar, N_As, N_Bs, N-Cs timeout 값이 200ms 로 동작되던 것을 수정하여 1000ms 로 동작
설정 영향	없음
ASW 조치 필요 사항	없음

■ Client 가 긍정응답을 수신하지 못해 동일한 Sequence number 의 block 을 전송하면 reprogramming fail 되는 문제 수정

원인	Transfer Data 시, 동일한 sequence number 를 가진 block 을 전송하면 처리할 수 있어야 하나, 내부 로직에 오류가 있어 처리하지 못함
----	--

동작 영향	Transfer Data 에서 동일한 sequence number 를 갖는 block 이 전송되면 Transfer exit 단계에서 NRC 24 를 응답하던 문제를 수정
설정 영향	없음
ASW 조치 필요 사항	없음

■ StMin 을 수신받았을 때 정상적으로 처리하지 못하는 현상 개선

원인	Mainfunction Period 가 변경되었으나, StMin 을 처리하는 로직이 그에 맞게 변경되지 않음
동작 영향	FBL 이 Consecutive Frame 을 전송할 때, StMin 에 맞게 Tx 를 송신
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

■ 다운그레이드 방지 적용 시 Sector 검사 로직에서 fail 시에 NRC 0x00 을 송출하는 문제 개선

원인	NRC 를 정해주는 코드가 추가되지 않아서 다운그레이드 방지 적용 시 Sector 검사 로직에서 fail 하였을 때 NRC 0x00 을 송출
동작 영향	다운그레이드 방지 적용 시, header 와 trailer 가 같은 sector 에 있거나, header 또는 trailer 가 두 개의 sector 에 걸쳐져 있을 경우 NRC 0x72 송출
설정 영향	FblGeneral/FblSoftwareVersionCheck
ASW 조치 필요 사항	없음

➤ 오류 수정

■ Internal wdg enable 을 false 로 체크하였을 때 generated 파일에 반영되지 않는 문제 개선

원인	Bool type 의 설정을 false 로 하였을 때 else 조건이 없어서 true 로 generation 됨
동작 영향	Bool type 의 설정을 generate 시 정상적으로 generation 됨
설정 영향	FblWatchdogConfig/FblIntWdgEnable
ASW 조치 필요 사항	없음

➤ 오류 수정

■ Communication control 의 0x00, 0x01, 0x02 subfunction 에 대해 긍정응답 하지 않는 문제 개선

원인	Sub function 을 관리하는 구조체에서 해당 subfunction 이 누락되어 있었음
동작 영향	위 sub function 을 사양에 맞게 request 하였을 때 긍정응답 함
설정 영향	없음
ASW 조치 필요 사항	없음

➤ 오류 수정

- 다운그레이드 방지 기능 적용 시, header blk 다음 block transfer data 에 대한 응답 timeout 발생하는 문제 개선

원인	header blk 다음 block transfer data 에서 header 부분의 version 을 check 진행 하고, 이후 trailer sector 에 대해서 erase 수행 시 지연이 발생하여, timeout 발생
동작 영향	Transfer data 의 온전한 block 을 송부 받으면 pending 처리해주는 형태로 개선
설정 영향	FblGeneral/FblSoftwareVersionCheck
ASW 조치 필요 사항	없음

➤ 오류 수정

- 다운그레이드 방지 적용 시 header, trailer sector 판단 로직에 오류가 있는 부분 개선

원인	header, trailer sector 판단 로직 연산에서 address 와 length 간의 연산을 수행할 때 로직 오류가 있음, sector 의 끝 부분에 header, trailer 가 위치 할 경우 올바른 위치임에도 불구하고 부적합한 위치에 있다고 판단하여 erase fail
동작 영향	로직 오류를 수정하여, sector 의 끝 부분에 header, trailer 가 위치하더라도 정상적으로 리프로그래밍을 수행
설정 영향	FblGeneral/FblSoftwareVersionCheck
ASW 조치 필요 사항	없음

➤ 오류 수정

- Secure Access 2.0 CRL 의 8 번 검증 항목인 폐지된 인증서 목록을 확인하는 로직의 문제 개선

원인	Holder Reference 와 Start Sequence Number 를 통한 CRL 8번 검증 항목인 폐지된 인증서 목록을 확인하는 로직에서 사양과 맞지 않는 연산을 수행하여 CRL 검증이 정상적으로 이루어 지지 않음
동작 영향	Holder reference 와 Start Sequence Number 의 차이를 가지고 검증을 수행해야 하나, 덧셈 연산이 적용되어 정확한 CRL 검증이 되지 않았던 점을 수정하여 정상적으로 검증 수행
설정 영향	FblDiagComConfig/FblDiagSecurityAccessLevel FBL_SA_SECUREACCESS_2_0 설정 시
ASW 조치 필요 사항	없음

➤ 오류 수정

- 다운그레이드 방지 기능 사용시 RTSW Start Address 가 Blk Erase Unit 의 배수가 아닐 경우 Transfer Data 실패 문제 개선

원인	코드플래시 앞쪽이 뒤쪽보다 단위가 작기 때문에 배수가 아닌 경우가 발생한다 (ex. 0x00FE_0000 이 RTSW Start address, Blk Erase Unit은 0x40000) FBL 다운그레이드 감지 로직의 경우 header 가 포함된 sector 는 별도로 지우지 않고 Trailer 만 복구하거나 따로 지우기 때문에, RTSW Start Address 의 의존성을 없애야 함
동작 영향	두번째(Trailer 포함 영역) 삭제 영역 결정할 때 Erase unit 을 기반으로 RTSW Start Address 주소를 구하게 변경하여 Erase unit 의 배수가 되는 곳을 sector 의 start address 로 인식 함
설정 영향	FblGeneral/FblSoftwareVersionCheck
ASW 조치 필요 사항	없음

➤ 오류 수정

- MainSW 가 아닌 Sub MCU 부분을 리프로그래밍 할 때, A bank 로 jump 하는 문제 개선

원인	MainSW 가 아닌 Sub MCU 의 리프로그래밍을 위한 부분을 FBL 을 통해 전송하는 경우가 있는데, DualBank 환경에서 MainSW 가 아닌 module 에 대한 리프로그래밍을 수행하는 경우, A bank 로 jump 해주고 있는 상황이 발생
동작 영향	Fbl_WritePartitionFlag 에 DualIM 로직이 있는데, MainSW 가 아닐 경우에는 해당 로직을 수행하지 않도록 user 가 변경할 수 있게 함수에 Block_ID 를 인자로 넘겨주는 방식으로 변경
설정 영향	FblOta/FblMemorySwapEnable true 인 경우
ASW 조치 필요 사항	Fbl_WritePartitionFlag 의 인자로 BlockID 를 받는데, MainSw 의 것이 아닐 경우에는 swap 에 필요한 로직을 수행하지 않도록 user 가 변경 필요

➤ 기능 개선

- 다운그레이드 판단 로직을 유저 코드로 이동하고 기본 실패 처리 하도록 개선

원인	다운그레이드 판단 로직은 업체, 제어가 마다 관리하는 방식 등이 달라서, 판단 로직을 user 가 처리할 수 있도록 변경
동작 영향	Fbl_SvcCheckVersion 에 user 가 다운그레이드 판단 로직을 작성해야 함
설정 영향	FblGeneral/FblSoftwareVersionCheck
ASW 조치 필요 사항	Fbl_SvcCheckVersion 에 user 가 다운그레이드 판단 로직을 작성해야 함

➤ 기능 개선

- 다운그레이드 실패 시 general reject 응답하도록 개선

원인	사양 변경에 따라, 다운그레이드 실패 시 general reject 을 응답하도록 함
동작 영향	사양 변경에 따라, 다운그레이드 실패 시 general reject 을 응답하도록 함

설정 영향	FblGeneral/FblSoftwareVersionCheck
ASW 조치 필요 사항	없음

Limitations

- 현 버전의 Fbl 은 TC23X, TC29X, TC33X, TC36X, TC38X, TC39X, SPC58X, RH850F1KM, CYT2BXX, CYT4BXX MCU 만 지원한다.
- Can ID Format 은 11 bit 만 지원한다.
- Signature 를 flash 에 쓰기 위해선 Signature 를 송신 시에 RequestDownload 서비스로 송신해야 함.
- SeedKey 기반의 Security Access 는 4Byte(SEEDKEY), 8Byte(ASK) 만을 지원한다.
- FBL 단독 부팅 시, Physical Multi-frame(CF) 수신 중 Functional SF 들어오면 Physical CF 은 계속 수행 하고 Functional SF 은 reject 하는 사양은 Secure Access 서비스에서는 만족하지 못한다.
- 내부 혹은 외부 WDG 를 사용하여야 한다.
- RTSW 및 APP 에서 SW Reset 이후 초기화 되지 않기를 기대하는 영역 및 변수는 FBL 에서 사용하는 RAM 영역을 회피하여 사용하여야 한다. (FBL 40KB 사용)
- FBL 에서 사용할 변수 중에 SW Reset 시에 초기화 되지 않기를 바란다면 RAM_RST_NOCLEAR_BTL 영역을 사용한다.
- 현재 버전의 CAN ID 및 NODE 설정 변경 지원은 TC3xx MCU 만 지원한다.

3. Configuration Guide

FBL 일부 설정은 RTSW 와 공유해야 하는 정보이며 generator 에 의한 생성 파일에 설정 사항이 반영된다.

FBL generator 는 FBL 프로젝트를 위한 코드 생성과정과 RTSW 프로젝트를 위한 코드 생성과정이 분리되어 있다.

따라서 FBL 설정 변경 후 FBL 프로젝트 빌드 & RTSW 프로젝트 빌드를 모두 수행해야 설정 변경 사항이 플랫폼 전체에 반영 될 수 있다.

3.1 외부 라이브러리

Fbl 에서 사용되는 암호화 알고리즘은 외부 라이브러리를 통해 제공된다.

오토에버를 통해 제공되는 라이브러리와 지원 알고리즘 리스트는 아래와 같다.

라이브러리	지원 알고리즘	비고
HAE_ASK	오토에버 Advanced SeedKey 생성 알고리즘	차중-제어기별 variation 존재
HAE_CryptoLib_X	오토에버 SW 암호화 알고리즘	
HAE_HsmDriver	오토에버 HSM 운영스택 - 오토에버 HSM 드라이버	
HAE_SFM2	오토에버 Secure Flash 2.0 라이브러리	

필요 라이브러리는 진단 보안 레벨에 따라 결정되며 아래의 테이블을 참조하여 필요 라이브러리 리스트를 도출한다.

보안 요구사항	필요 라이브러리	
진단 보안 레벨 L1	X	
진단 보안 레벨 L9	HSM 미사용/ 오토론 HSM 운영스택 사용	HAE_ASK, HAE_CryptoLib_X
	오토에버 HSM 운영스택 사용	HAE_ASK, HAE_CryptoLib_X, HAE_HsmDriver
진단 보안 레벨 L21	HSM 미사용 / 오토론 HSM 운영스택 사용	HAE_CryptoLib_X
	오토에버 HSM 운영스택 사용	HAE_CryptoLib_X, HAE_HsmDriver
Secure Flash 2.0	오토론 HSM 운영스택 사용	HAE_SFM2
	오토에버 HSM 운영스택 사용	HAE_SFM2, HAE_HsmDriver

3.2 설정

설정관련 Category의 해석은 다음과 같다.

- **Changeable (C)** : User 에 의해서 설정 가능한 항목
- **Fixed (F)** : User 에 의한 변경이 불가능한 항목

※ User 변경 가능 항목일지라도 MCU dependent 값이 제시되어 있는 항목은 제시된 값 이외의 다른 값 사용시 정상적인 작동을 보장하지 않는다. 따라서 해당 값 변경 검토시 반드시 오토론의 가이드를 받도록 한다.

3.2.1 FblGeneral

Parameter Name	Value	Category
Short Name	User Defined	C
Support MCU ¹⁾	FBL_MCU_TC23X FBL_MCU_TC29X FBL_MCU_TC38X FBL_MCU_TC39X FBL_MCU_SPC58X FBL_MCU_RH850F1KM FBL_MCU_CYTXXX	C

Parameter Name	Value	Category
	FBL_MCU_TC36X FBL_MCU_TC33X	
Support HSM ²⁾	FBL_HSM_NONE FBL_HSM_AUTHSM_1_0 FBL_HSM_HAEHSM_2_0	C
Dev Error Detect ³⁾	true / false	F
Software Version Check ⁴⁾	True / false	C

- 1) Fbl 에서 지원하는 MCU 설정
- 2) Fbl 에서 지원하는 HSM 설정
- 3) Fbl 의 DET 설정
- 4) Fbl 의 Software Version Check 기능 설정 (다운그레이드 방지 기능)

3.2.2 FblMcuSpc58x

Parameter Name	Value	Category
Short Name	User Defined	F
FblMcuName	SPC582B50 SPC582B54 SPC582B60 SPC584B60 SPC584B64 SPC584B70 SPC584C70 SPC584C74 SPC584C80 SPC58EC70 SPC58EC74 SPC58EC80	F

3.2.3 FblMcuCytxxx

Parameter Name	Value	Category
Short Name	User Defined	F
FblMcuName	CYT2B7XX CYT2B9XX CYT2BLXX CYT3BBXX CYT4BBXX CYT4BFXX	F

- 1) Spc58x Fbl 에서 지원하는 MCU Name 설정
- 2) Cytxxx Fbl 에서 지원하는 MCU Name 설정

3.2.4 FblOta

Parameter Name	Value	Category
Short Name	User Defined	C
Memory Swap Enable ¹⁾	true / false	C

Parameter Name	Value	Category
Decompress Enable ²⁾	true / false	C
OTAEnable ³⁾	true / false	C
PartitionFlagAddress ⁵⁾	User Defined	C
PartitionFlagSize ⁶⁾	User Defined	C
FblDecompressBufSize ⁷⁾	User Defined	C

- 1) 사용자가 구현한 메모리 이중화 지원 기능 적용 여부
사용자 API (Fbl_SelectEntryPoint, Fbl_WritePartitionFlag) 를 구현한 후 Enable 적용해야 한다.
- 2) 사용자가 구현한 OTA 펌웨어 데이터 압축해제 기능의 적용 여부
사용자 API (Fbl-DecompressInit, Fbl-DecompExpandData) 를 구현한 후 Enable 적용해야 한다.
- 3) ES95486-02 UDS 사양 개정 기능의 적용 여부(EraseTargetArea, CheckProgramDependency)
- 4) 1)번 기능 사용 시 RTSW 의 무결성 검증 후 사용할 Partition Flag 의 주소
- 5) 1)번 기능 사용 시 RTSW 의 무결성 검증 후 사용할 Partition Flag 의 사이즈
- 6) 2)번 기능 사용 시 압축기능에서 사용할 Buffer 의 사이즈 (default : 2048)

3.2.5 FbIPIIConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Cpu Clock ¹⁾	80 / 160 / 300 MHz	C
Quartz Clock ²⁾	8 / 16 / 20 / 24 / 40 MHz	C
Time Prescaler ³⁾		C
CLK_TRIM_ECO_WDTRIM ⁴⁾	User Defined	C
CLK_TRIM_ECO_ATRIM ⁴⁾	User Defined	C
CLK_TRIM_ECO_FTRIM ⁴⁾	User Defined	C
CLK_TRIM_ECO_RTRIM ⁴⁾	User Defined	C
CLK_TRIM_ECO_GTRIM ⁴⁾	User Defined	C
CLK_ECO_AGC_EN ⁴⁾	User Defined	C

- 1) Fbl 에서 사용하는 CPU clock 설정
 - TC3XX : 300 MHz
 - SPC58X : 160 MHz. Limitation: SPC58X CPU Clock config only accepts 160 Mhz
 - RH850F1KM : 80 MHz
 - CYT2BXXX (TV-II-B-E 계열) : 160 Mhz
- 2) Fbl 에서 사용하는 Quartz clock 설정
 - TC3XX : 20 MHz / 40 MHz
 - SPC58X : 16 MHz / 40 MHz
 - RH850F1KM : 16 MHz / 40 MHz
 - CYT2BXXX (TV-II-B-E 계열) : 8 MHz / 16 MHz / 20 MHz / 24 MHz

3) 미사용

4) Cypress ECO TRIM 설정 (Cypress Only)

- Cypress FBL 에서 ECO Clock 을 사용함. Crystal 에 의존적인 설정이므로 유저가 직접 입력 필요. Cypress 에서 제공하는 <ECO Trim Calculator for Traveo II> 문서 참조 필요

3.2.6 FblFlashConfig

Parameter Name	Value	Category
Short Name	User Defined	C
MainSwEntryAddress1 ²⁾	0x00000000~0xFFFFFFFF	C
MainSwEntryAddress2 ³⁾	0x00000000~0xFFFFFFFF	C
Flash Alignment ¹⁾	1 ~ 256	C
Startup Command Address ⁴⁾	0x00000000~0xFFFFFFFF	C
Common Ram Address ⁵⁾	0x00000000~0xFFFFFFFF	C
Erase Sector Num ⁶⁾	1	F
FblFlashEraseUnit ⁷⁾	0x00000000~0xFFFFFFFF	C

※ HSM 사용시 Chip Lock 등의 문제가 발생할 수 있기 때문에 많은 주의가 필요하다.

HSM 관련 사용상 주의사항은 HSM 공급업체 매뉴얼을 참조

1) Main SW 첫번째 시작주소. OTA 메모리 이중화 기능 미사용시 MainSwEntryAddress1 만 설정

- TC3XX

HSM 미사용시 : 0x80020000

HSM 사용시 : 0x80048000

- SPC58X : 0x00FE0000

- RH850F1KM : 0x00010000

- CYTXXX : 0x10048000 (M0+ firmware size 에 따라 달라질 수 있음)

2) Main SW 두번째 시작주소. 사용자가 OTA 메모리 이중화 기능 구현후 사용자 API (Fbl_SelectEntryPoint,

Fbl_WritePartitionFlag) 를 구현하여 선택적으로 MainSwEntryAddress1, MainSwEntryAddress2 주소 사용

- TC3XX : 메모리 SWAP 이 HW 에서 지원되므로 설정 불필요

- SPC58X : OTA SWAP 사용시 사용자 설정 필요

- RH850F1KM : OTA SWAP 사용시 사용자 설정 필요

- CYTXXX : 메모리 SWAP 이 HW 에서 지원되므로 설정 불필요

3) Flash writing 시 주소, 크기값의 alignment 유효성 검증 단위

- TC3XX : 256

- SPC58X : 128

- RH850F1KM : 256

- CYTXXX : 32

4) Startup command 를 위한 RAM address

- TC3XX : 0x7003BF00 (TC36x 의 경우, 0x7002FFF0)
- SPC58X : 0x400A8000
- RH850F1KM : 0xFE000000
- CYT2BXXX : 0x800A0000

5) 공용 정보를 위한 RAM address

- TC3XX : 0x7003BE00 (TC36x 의 경우, 0x7002FFF4)
- SPC58X : 0x400A8004
- RH850F1KM : 0xFE000004
- CYT2BXXX : 0x800A0004

6) Routine Control 서비스의 Erase Memory 시에 Erase Memory 의 sector number

7) 해당 MCU 의 Erase 섹터의 최대 단위 (Block Header / Trailer 섹터를 지울 때 사용됨) (Software Version Check 기능이 True 일 경우에 유효한 설정)

3.2.7 FblSecurityConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Crc16BlockSize ¹⁾	FBL_CRC16_BLOCK_4096_BYTES FBL_CRC16_BLOCK_2048_BYTES FBL_CRC16_BLOCK_1024_BYTES FBL_CRC16_BLOCK_512_BYTES FBL_CRC16_BLOCK_256_BYTES FBL_CRC16_BLOCK_128_BYTES	C

1) CRC16 검증시 watchdog trigger 호출 block 크기

설정 block 크기의 data 에 대한 CRC16 계산마다 watchdog trigger 가 호출되므로, 작은 단위의 block 설정시 watchdog trigger 호출 빈도가 높아지며 CRC16 전체 수행시간이 길어진다.

- FBL_CRC16_BLOCK_4096_BYTES

3.2.8 FblSecurityAccess

Parameter Name	Value	Category
Short Name	User Defined	C
Level ¹⁾	FBL_SA_SEEDKEY FBL_SA_AdvancedAeedKey FBL_SA_SecureAccess1.0 FBL_SA_SecureAccess2.0	C
Secure Access Certificate Hash Algorithm ²⁾	FBL_SA_HASH_SHA160 FBL_SA_HASH_SHA256	C

Parameter Name	Value	Category
FblSecurityNumAttDelay ³⁾	User Defined	C
FblSecurityDelayTime ³⁾	User Defined	C

1) Security Access 보안 레벨로 진단 보안 레벨을 적용

FBL_SA_SEEDKEY: L1 (SeedKey 4Byte – Autron SeedKey Library)

FBL_SA_ADVANCED_SEEDKEY : L9 (HSAC SeedKey 8Byte – Autoever ASK Library)

FBL_SA_SECUREACCESS_1_0 : L21 (CSAC)

FBL_SA_SECUREACCESS_2_0: L21 (CSAC with CRL)

Security Access Level	난수 생성 방식
FBL_SA_SEEDKEY (난수 4byte)	FBL 자체 생성 난수
FBL_SA_ADVANCED_SEEDKEY (난수 8byte)	HSM 미사용시
FBL_SA_SECUREACCESS_1_0 (난수 8byte)	- 초기값 FBL 자체생성 Seed + HaeCryptoLib PRNG
FBL_SA_SECUREACCESS_2_0 (난수 8byte)	HSM 사용시 - HSM PRNG (1 회 TRNG 이후 PRNG)

* Random 알고리즘은 HSM 을 적용할 경우 난수 생성알고리즘으로 HSM_PRNG (HSM 내 1 회 TRNG 적용 이후 PRNG 수행) 이 적용되며 (ES95489-01), HSM 을 적용하지 않을 경우 FBL Seed 를 기반으로 한 SW PRNG 가 사용된다

2) Secure Access 인증서 검증 시 활용되는 Hash 레벨

ES95489-01 참조하여 설정 필요

3) SendKey 서비스 요청의 최대 실패 횟수(False attempts)

4) Lock Time 시간 설정

3.2.9 FblSecureFlash

Parameter Name	Value	Category
Short Name	User Defined	C
Version ¹⁾	FBL_SF_CRC FBL_SF_VER_1_0 FBL_SF_VER_2_0	C
Signature Hash Algorithm ²⁾	FBL_SF_HASH_SHA160 FBL_SF_HASH_SHA256	C
Revised Secure Flash 2.0 ³⁾	True/False	C
Write Signature ⁴⁾	True/False	C

Parameter Name	Value	Category
Write Signature Delimiter ⁵⁾	True/False	C

- Secure Flash 적용 ES95489-02 버전
FBL_SF_CRC : Firmware CRC 검사만 수행
FBL_SF_VER_1_0 : Secure Flash 1.0
FBL_SF_VER_2_0 : Secure Flash 2.0
- SecureFlash 서명검증 과정에서 사용할 Hash 알고리즘 (FBL_SF_CRC 해당 없음)
FBL_SF_HASH_SHA160 : HAE SW 암호화 알고리즘 라이브러리에서 제공하는 SHA160 사용
FBL_SF_HASH_SHA256 : HAE SW 암호화 알고리즘 라이브러리에서 제공하는 SHA256 사용
- Secure Flash 2.0 개정 사양 적용 시 True로 설정
- True/False 설정에 따라 Signature 를 flash 에 write
- True/False 설정에 따라 Signature delimiter 를 flash 에 write

3.2.10 FblDiagConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Diag Response Before Reset Enable ¹⁾	true / false	C
Diag Response Before Session Change Enable ²⁾	true / false	C
Diag Max Num Response Pending ³⁾	0~255	C
Diag Max Num Sign Area ⁴⁾	0~128	C
Diag Signature Tx Mode ⁵⁾	FBL_TXDATA/FBL_RE QDOWNLOAD	C
Support Diagnostic Standard ⁶⁾	FBL_ES95486_00/ FBL_ES95486_02	C
DiagEraseRID ⁷⁾	0xFF00/ 0x0212	C
DiagChkProgDepRID ⁸⁾	0xFF01	C
DiagOTARReadyRID ⁹⁾	0x0300	C
DiagTransferDataChunkMode ¹⁰⁾	true / false	F
Diag Max Num Of Block Length ¹¹⁾	0~65535	C

- Reset 직전에 진단 응답 메시지 송신 여부
- true
- Session 변경 직전에 진단 응답 메시지 송신 여부
- true
- Pending 응답 메시지 최대 개수
- TC38X : 25
- TC39X : 35

- TC36X : 35

- SPC58X : 35

- RH850F1KM : 35

- CYTXXX: 35

4) Signing 영역의 최대 개수

- 미설정 시 기본값 : 16

- 전자서명 생성 시 ini 파일에 설정되는 전자서명 Block 의 최대 개수를 입력한다.

Note : FBL firmware 는 제어기에서 새로 update 가 불가능하므로 Max 값 설정 시 앞으로 추가될 전자서명 Block 의 최대 개수와 메모리 영역을 고려하여 충분히 큰값으로 설정이 필요하다.

5) Secure Flash Signature 전송 모드 선택

- Default Value : FBL_TXDATA mode

6) 진단 사양

- Default Value : ES95486_00

Note : DCM 에서 설정한 서비스와 일치하는 진단사양을 선택해야 한다.

7) RoutineControl Erase ID 설정

A. EraseTargetArea (ECU with memory redundancy) : 0x0212 설정

8) RoutineControl CheckProgrammingDependency ID 설정

9) RoutineControl OTAReady ID 설정

10) Chunk Mode 가 설정되면 TransferData 수신 중 blockSequenceCounter 를 모두 수신 받기전부터 flash 에 data 를 FblProgramDataBufferSize 크기만큼씩 쓴다.

11) RequestDownload 서비스 요청 시 긍정응답으로 회신되는 data 이며 진단기(Client)가 TransferData 의 message 로 얼마의 data bytes 를 보낼지 지정

3.2.11 SwModule

Parameter Name	Value	Category
Short Name	User Defined	C
Index ¹⁾	0~255	C
Start Address ²⁾	0x00000000~0xFFFFFFFF	C
End Address ³⁾	0x00000000~0xFFFFFFFF	C
Module Info Address ⁴⁾	0x00000000~0xFFFFFFFF	C
Special Data Address ⁵⁾	0x00000000~0xFFFFFFFF	C
Crc Info Address ⁶⁾	0x00000000~0xFFFFFFFF	C
Signature Address ⁷⁾	User Defined	C
ECUSoftwareUNIT ⁸⁾	User Defined	C
Blk Header Address ⁹⁾	0x00000000~0xFFFFFFFF	C
Blk Trailer Address ¹⁰⁾	0x00000000~0xFFFFFFFF	C

구체적인 주소는 RTSW, FBL Isl(Id) 파일 참조

1) SW module 인덱스로 0 부터 순차적으로 부여

FBL : 0, Main SW : 1, 그 이외 : 2~

2) SW module 의 시작 주소

MCU	FBL	MainSW (S19 파일의 리프로그래밍 대상 시작주소와 일치해야 한다)
TC3XX	0x80000000	HSM 미사용시 : 0x80020000 HSM 사용시 : 0x80048000
SPC58X	0x00FC0000	0x00FE0000
RH850F1KM	0x00000000	0x00010000
CYTXXX	0x10028000	0x10048000

3) SW module 의 끝 주소(S19 파일의 끝주소와 일치해야 한다)

MCU	FBL	MainSW (S19 파일의 리프로그래밍 대상 끝주소와 일치해야 한다)
TC38X	0x8001FFFF	0x809FFFFFFF
TC39X	0x8001FFFF	0x80FFFFFFF
TC36x	0x8001FFFF	0x804FFFFFFF (0x8020000000-- 0x802FFFFFFF : 사용불가)
SPC58X	0x00FC7FFF	0x013BFFFFFF
RH850F1KM	0x0000FFFF	0x003D7FFF
CYTXXX	0x10047FFF	1M (2B7) Single Bank : 0x1010FFFF Dual Bank : 0x10087FFF 2M (2B9) Single Bank : 0x1020FFFF Dual Bank : 0x10107FFF 4M (2BL_3BB_4BB) Single Bank : 0x1040FFFF Dual Bank : 0x10207FFF

		8M(4BF) Single Bank : 0x1082FFFF Dual Bank : 0x10417FFF
--	--	--

4) SW module 의 정보 구조체 주소

- TC3XX : Start address + 0x20 (RTSW, FBL Isl(Is) 파일 참조)
- SPC58X, RH850F1KM, CYTXXX : FBL, BSW Id 파일 참조

MCU	FBL	MainSW
TC3XX	0x80000020	HSM 미사용시 : 0x80020020 HSM 사용시 : 0x80048020
SPC58X	0x00FC0020	0x00FE0200
RH850F1KM	0x00000200	0x00010200
CYTXXX	0x10028020	0x10048800

5) SW module 의 특별 주소

FBL : Shared information 정보

- TC3XX : Start address + 0x30 (FBL Isl(Is) 파일 참조)
- SPC58X, RH850F1KM, CYTXXX : FBL, BSW Id 파일 참조

Main SW : Security Key 정보

- TC3XX : Start address + 0x100 (RTSW Isl(Is) 파일 참조)
- SPC58X, RH850F1KM, CYTXXX : FBL, BSW Id 파일 참조

MCU	FBL	MainSW
TC3XX	0x80000030	HSM 미사용시 : 0x80020100 HSM 사용시 : 0x80048100
SPC58X	0x00FC0060	0x00FE0300
RH850F1KM	0x000002B0	0x00010300
CYTXXX	0x10028030	0x10048900

6) SW module 의 CRC 정보 주소

FBL

- TC3XX : Start address + 0x100 (FBL Isl(Is) 파일 참조)
- SPC58X, RH850F1KM, CYTXXX : FBL, BSW Id 파일 참조

Main SW

- TC3XX : Start address + 0x200 (RTSW IsI(Is) 파일 참조)
- SPC58X, RH850F1KM, CYTXXX : FBL, BSW Id 파일 참조

MCU	FBL	MainSW
TC3XX	0x80000100	HSM 미사용시 : 0x80020200 HSM 사용시 : 0x80048200
SPC58X	0x00FC0030	0x00FE0210
RH850F1KM	0x00000210	0x00010210
CYTXXX	0x10028100	0x10048810

7) SW module 의 Signature 정보 주소

Main SW

- Security Key, PartitionFlag 주소 뒤에 설정할 경우 RTSW 이미지 크기 의존성(Secure flash 설정시 발생하는)을 없앨 수 있다. 위와 같이 설정할 시에는 RTSW LD 파일에도 반영이 필요하다.
- Signature 의 size 는 256BYTE 이며 Signature Address 만 설정하면 리프로그래밍 시에 FBL 이 자동으로 256BYTE 의 Signature 를 FLASH 에 쓴다.

8) FblOta/OTAEnable 기능이 Enable 될 경우 제어기의 ECUSoftwareUNIT 설정

RomTst Code가 위치가 변경되면 아래 그림의 RomTstArea의 주소를 변경

Container Details - SwModule

Index	Short Name	Index	Start Address	End Address	Module Info A...	Special Data A...	Crc Info Address
0	FBL	0	0x00FC0000	0x00FDFFFF	0x00FC0020	0x00FC0060	0x00FC0030
1	MainSW	1	0x00FE0000	0x013BFFFF	0x00FE0200	0x00FE0300	0x00FE0210
2	RomTstArea	2	0x00FE0400	0x00FE05FF	0x00FE0400	0x00FE0420	0x00FE040C

9) BlkHeaderAddress

- 다운그레이드 방지를 위한 펌웨어 Header 버전블록의 시작 Address (Software Version Check 기능이 True 일 경우에 유효한 설정)

10) BlkTrailerAddress

- 다운그레이드 방지를 위한 펌웨어 Trailer 버전블록의 시작 Address (Software Version Check 기능이 True 일 경우에 유효한 설정)

3.2.11.1 SwMemoryBlockArea

이중화 기능을 쓰지 않는 OTA 사용 시(OTA enable(O), FbiMemorySwapEnable(X))

Memory block 설정으로 SwMemoryBlockArea를 사용한다.

8.1.4.2 EraseTargetArea (ECUs without memory redundancy)

8.1.4.2.1 Request message

Data Byte	Parameter Name	Cvt	Hex Value	Mnemonic
#1d	RoutineControl request Service ID	M	#31h	RC
#2d	Sub-function = [routineControlType]	M	#01h	LEV_RCTP_
#3d	routineIdentifier [] = [byte #1 (MSB) byte #2]	M	#FFh	RI_B1
#4d		M	#00h	B2
#5d	routineControlOptionRecord [] = [SWUnitMemoryArea byte#1(MSB) SWUnitMemoryArea byte#2]	M	#XXh	RCOR_SW UMA
#6d				
#7d	routineControlOptionRecord [] = [Memory block]	U	#XXh	RCOR_MB
#8d				

The SWUnitMemoryArea represents the memory area to be erased.
Memory block would be used as ECU specific memory index.

Table8-10. Request Message

Container Details - SwMemoryBlockArea

Index	Short Name	Sw Block Index	Start Address	End Address
0	SwMemoryBlockArea0	1	16646144	16711679
1	SwMemoryBlockArea1	2	16711680	16777215

Parameter Name	Value	Category
Short Name	User Defined	C
Sw Block Index ¹⁾	0~7	C
Start Address ²⁾	0x00000000~0xFFFFFFFF	C
End Address ³⁾	0x00000000~0xFFFFFFFF	C

1) Sw Block Index : RoutineControl EraseTargetArea 서비스 호출 시 Memory block data 값 (2 byte)

2) Start Address : Erase 시 Memory block 의 시작 주소 (4 byte)

3) End Address : Erase 시 Memory block 의 끝 주소 (4 byte)

3.2.12 SwVersionInfo

Parameter Name	Value	Category
Short Name	User Defined	C
Customer Id ¹⁾	0~255	C

Parameter Name	Value	Category
Product Id ²⁾	0~255	C
Product Variant ³⁾	0~255	C
Sw Version ⁴⁾	0~255	C
Int Sw Version ⁵⁾	0~255	C
Synchro1 ⁶⁾	0~255	C
Synchro2 ⁶⁾	0~255	C
Build Date ⁷⁾	YYYY-MM-DD	C

사용시 업체에서 직접 설정

- 4) CC: Customer ID (1 byte)
- 5) AA: Product ID (1 byte)
- 6) V_: Product variant (high digit only) from 0:(0x00) to F:(0xF0)
- 7) VV: SW version (1 byte)
- 8) ZZ: Intermediate SW version (1 byte)
- 9) Synchro bytes (1 + 1 byte)
- 10) SW build date (형식 : 년-월-일)

Secure Access 2.0 적용시 FBL 에서 CRL 유효성 검증을 기준 날짜로 사용되므로 현재보다 미래의 날짜를 사용하지 않도록 한다.

3.2.13 FblWatchdogConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Int Wdg Enable ¹⁾	true / false	C
Int Wdg Timeout Ms ²⁾	0 ~ 4294967295	C

- 1) Internal watchdog 사용 유무
- 2) Internal watchdog 의 millisecond 단위의 timeout
- 100

3.2.14 FblEthernetConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Received Data Buffer Size ¹⁾	0 ~ 65535	C
Program Data Buffer Size ²⁾	0 ~ 65535	C

- 1) 진단 메시지 수신 버퍼 크기
- TC3XX : 8191
- CYTXXX : 2050
- 2) Programming 데이터 버퍼 크기
- TC3XX : 256

- CYTXXX : 8

3.2.15 FblEcuIpAddress

Parameter Name	Value	Category
Short Name	User Defined	C
Fixed ¹⁾	. 구분자를 사용한 IPv4 형식의 주소	C
Ref ²⁾	TcplpConfig / TcplpLocalAddr / TcplpStaticIpAddressConfig	C

1) ECU 진단 IP 주소. 적용시 '.' 구분자 형식으로 표현. 설정시 Ref 는 무시됨.

ex) 10.0.0.4

2) 참조된 컨테이너의 Static Ip Address 파라미터 값을 사용

3.2.16 FblEcuMacAddress

Parameter Name	Value	Category
Short Name	User Defined	C
Fixed ¹⁾	- 혹은 : 구분자를 사용한 hex 형식의 MAC 주소	C
Ref ²⁾	EthConfigSet / EthCtrlConfig	C

1) ECU 진단 MAC 주소. 적용시 '-' 혹은 ':' 구분자 형식으로 표현. 설정시 Ref 는 무시됨.

ex) 00:03:19:00:00:01 or 00-03-19-00-00-01

2) 참조된 컨테이너의 Ctrl Phy Address 파라미터 값을 사용

3.2.17 FblEcuTcpPort

Parameter Name	Value	Category
Short Name	User Defined	C
Fixed ¹⁾	0 ~ 65535	C
Ref ²⁾	SdAdConfig / SoAdSocketConnectionGroup	C

1) ECU 진단 Tcp Port 주소. 설정시 Ref 는 무시됨.

2) 참조된 컨테이너의 Socket Local Port 파라미터 값을 사용

3.2.18 FblEthDiagLocalAddress

Parameter Name	Value	Category
Short Name	User Defined	C
Ecu Physical Local Address For Request Fixed ¹⁾	0 ~ 65535	F
Ecu Physical Local Address For Request Ref ²⁾	EthDiagConfig / EthDiagReprogramSocketConnection / EthDiagReprogramChannel / EthDiagReprogramRxNSdu	F
Ecu Physical Local Address For Response Fixed ³⁾	0 ~ 65535	F

Parameter Name	Value	Category
Ecu Physical Local Address For Response Ref ⁴⁾	EthDiagConfig / EthDiagReprogramSocketConnection / EthDiagReprogramChannel / EthDiagReprogramTxNSdu	F
Ecu Functional Local Address Fixed ⁵⁾	0 ~ 65535	F
Ecu Functional Local Address Ref ⁶⁾	EthDiagConfig / EthDiagReprogramSocketConnection / EthDiagReprogramChannel / EthDiagReprogramRxNSdu	F
Tester Physical Local Address Fixed ⁷⁾	0 ~ 65535	C
Tester Physical Local Address Ref ⁸⁾	EthDiagConfig / EthDiagReprogramSocketConnection / EthDiagReprogramChannel / EthDiagReprogramRxNSdu	C
Remote Physical Local Address Fixed ⁹⁾	0 ~ 65535	C
Remote Physical Local Address Ref ¹⁰⁾	EthDiagConfig / EthDiagReprogramSocketConnection / EthDiagReprogramChannel / EthDiagReprogramRxNSdu	C

- 1) Request 용 ECU 진단 Physical local 주소. 설정시 Ecu Physical Local Address For Request Ref 는 무시됨.
- 2) 참조된 컨테이너의 Eth Diag Local Address 파라미터 값을 사용
- 3) Response 용 ECU 진단 Physical local 주소. 설정시 Ecu Physical Local Address For Response Ref 는 무시됨.
- 4) 참조된 컨테이너의 Eth Diag Local Address 파라미터 값을 사용
- 5) ECU 진단 Functional local 주소. 설정시 Ecu Functional Local Address Ref 는 무시됨.
- 6) 참조된 컨테이너의 Eth Diag Local Address 파라미터 값을 사용
- 7) Tester Physical local 주소. 설정시 Tester Physical Local Address Ref 는 무시됨.
- 8) 참조된 컨테이너의 Eth Diag Remote Address 파라미터 값을 사용
- 9) Remote Physical local 주소(ex CCU). 설정시 Remote Physical Local Address Ref 는 무시됨.
- 10) 참조된 컨테이너의 Eth Diag Remote Address 파라미터 값을 사용

3.2.19 FbIVlan

Parameter Name	Value	Category
Short Name	User Defined	C
Enable ¹⁾	true / false	F
Id Fixed ²⁾	0 ~ 65535	F
Id Ref ³⁾	SdAdConfig / SoAdSocketConnectionGroup	F

- 1) VLAN 기능 사용 유무

- 2) VLAN ID 값. 설정시 Id Ref 는 무시됨.
- 3) 참조된 컨테이너의 EthIfVlanId 파라미터 값을 사용

3.2.20 FblEthType

Parameter Name	Value	Category
Short Name	User Defined	C
Eth Type ¹⁾	TRANCEIVER / SWITCH	F
Eth Speed ²⁾	1000M/100M	F
CCUCON0 ³⁾	0 ~ 0xFFFFFFFF	F
CCUCON1 ⁴⁾	0 ~ 0xFFFFFFFF	F
CCUCON2 ⁵⁾	0 ~ 0xFFFFFFFF	F
CCUCON5 ⁶⁾	0 ~ 0xFFFFFFFF	F
SCU_SYSPLLCON0_NDIV ⁷⁾	0~0x7F	F
SCU_SYSPLLCON0_PDIV ⁸⁾	0~0x07	F
SCU_SYSPLLCON1_K2DIV ⁹⁾	0~0x07	F
SCU_PERPLLCON0_DIVBY ¹⁰⁾	0/1	F
SCU_PERPLLCON0_NDIV ¹¹⁾	0~7F	F
ETH_BMCR_DATA ¹²⁾	0x140/0x2100	F

※ 자세한 CCU, SCU 설정은 MCU 매뉴얼을 참고

- 1) Eth Type 설정. Default 는 Tranceiver Type
- 2) Eth Speed 설정. Default 는 100M
- 3) ~11) 외부 Eth IC 를 사용하는 경우 해당 Register 값을 정의한다. 설정 안할 경우 Default value 로 설정된다.
- 12) RGMII 1Gbit 옵션으로 지원하는 경우 0x140 으로 설정. Default value 0x2100

3.2.21 FblCanConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Received Data Buffer Size ¹⁾	0 ~ 65535	C
Program Data Buffer Size ²⁾	0 ~ 65535	C
Can Type ³⁾	FBL_CAN_CLASSIC / FBL_CAN_FD	C

- 1) 진단 메시지 수신 버퍼 크기(Service ID 와 Data parameter 을 포함해야 하므로 +2 를 해줘야 함)
 - 2050
- 2) Programming 데이터 버퍼 크기
 - Other : 256
 - CYTXXX : 32 (Flash Alignment 와 설정 동일하게)
- 3) 진단 통신용 CAN Type

3.2.22 FblCanIfConfig

Parameter Name	Value	Category
Short Name	User Defined	C
CanIfCANFDDiscreteDlcSupport ¹⁾	TRUE/FALSE	F

- 1) CANFD 사용 시 DiscreteDlc 적용

3.2.23 FblCanDrvTc3xx

Parameter Name	Value	Category
Short Name	User Defined	C
Can Cell ¹⁾	FBL_CAN0 FBL_CAN1 FBL_CAN2	C
Can Node ²⁾	FBL_CAN_NODE0 FBL_CAN_NODE1 FBL_CAN_NODE2 FBL_CAN_NODE3	C
Can RXD ³⁾	FBL_CAN_RX_RXDA FBL_CAN_RX_RXDB FBL_CAN_RX_RXDC FBL_CAN_RX_RXDD FBL_CAN_RX_RXDE FBL_CAN_RX_RXDF FBL_CAN_RX_RXDG FBL_CAN_RX_RXDH	C

※ TC3xx 의 CAN 소스 클럭은 80 MHz

- 1) 진단 통신용 CAN Cell 정보
- 2) 진단 통신용 CAN Node 정보
- 3) 진단 통신용 CAN RXD Selection

3.2.24 FblCanDrvSpc58x and FblCanDrvRh850f1km

Parameter Name	Value	Category
Short Name	User Defined	C
Can Cell ¹⁾	FBL_CAN0 / FBL_CAN1 / FBL_CAN2 FBL_CAN3 / FBL_CAN4 / FBL_CAN5 FBL_CAN6 / FBL_CAN7	F
Can Clock Divider ²⁾	1~256	F
Can Potocol Clock Sel ³⁾	FBL_XOSC/ FBL_PLL	F
Can Rx Port ⁴⁾	사용 포트	F
Can Tx Port ⁴⁾	사용 포트	F

※ CAN 소스 클럭은

FBL_XOSC 선택시 외부 Oscillator clock

FBL_PLL 선택시 80 MHz

- 1) 진단 통신용 CAN Cell 정보
- 2) 진단 통신용 CAN Clock Divider 정보
- 3) 진단 통신용 CAN Protocol Clock Selection
- 4) 진단 통신용 CAN RX/TX 포트

3.2.25 FblCanDrvCytxxx

Parameter Name	Value	Category
Short Name	User Defined	C
Can Cell ¹⁾	FBL_CAN0 FBL_CAN1	F
Can Node ²⁾	FBL_CAN_NODE0 FBL_CAN_NODE1 FBL_CAN_NODE2 FBL_CAN_NODE3	F
Can RX Port ³⁾	CYTXXX_Rx_CAN0_0_P2_1 CYTXXX_Rx_CAN0_0_P8_1 CYTXXX_Rx_CAN0_1_P0_3 CYTXXX_Rx_CAN0_1_P4_4 CYTXXX_Rx_CAN0_2_P6_3 CYTXXX_Rx_CAN0_2_P12_1 CYTXXX_Rx_CAN0_3_P3_1 CYTXXX_Rx_CAN1_0_P14_1 CYTXXX_Rx_CAN1_0_P23_1 CYTXXX_Rx_CAN1_1_P17_1 CYTXXX_Rx_CAN1_1_P22_1 CYTXXX_Rx_CAN1_2_P18_7 CYTXXX_Rx_CAN1_2_P20_4 CYTXXX_Rx_CAN1_3_P15_1 CYTXXX_Rx_CAN1_3_P19_1	F
Can TX Port ⁴⁾	CYTXXX_Tx_CAN0_0_P2_0 CYTXXX_Tx_CAN0_0_P8_0 CYTXXX_Tx_CAN0_1_P0_2 CYTXXX_Tx_CAN0_1_P4_3 CYTXXX_Tx_CAN0_2_P6_2 CYTXXX_Tx_CAN0_2_P12_0 CYTXXX_Tx_CAN0_3_P3_0 CYTXXX_Tx_CAN1_0_P14_0 CYTXXX_Tx_CAN1_0_P23_0 CYTXXX_Tx_CAN1_1_P17_0 CYTXXX_Tx_CAN1_1_P22_0 CYTXXX_Tx_CAN1_2_P18_6 CYTXXX_Tx_CAN1_2_P20_3 CYTXXX_Tx_CAN1_3_P15_0 CYTXXX_Tx_CAN1_3_P19_0	F

※ CYTXXX 의 CAN 소스 클럭은 PLL 40 MHz 고정

- 1) 진단 통신용 CAN Cell 정보
- 2) 진단 통신용 CAN Node 정보
- 3) 진단 통신용 CAN RX PortPin 정보
- 4) 진단 통신용 CAN TX PortPin 정보

3.2.26 FblCanDiagMsgFuncConfig

Parameter Name	Value	Category
----------------	-------	----------

Parameter Name	Value	Category
Short Name	User Defined	C
Can Diag Msg Func Addr Format ¹⁾	FBL_CAN_ADDR_STANDARD / FBL_CAN_ADDR_EXTENDED	F
Can Diag Msg Func Id ²⁾	0x0 ~ 0x1FFFFFFF	C

1) 진단 통신용 Functional 메시지 주소 형식

2) 진단 통신용 Functional 메시지 ID

3.2.27 FblCanDiagMsgPhysReqConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Can Diag Msg Phys Req STmin ¹⁾	0 ~ 255	C
Can Diag Msg Phys Req Addr Format ²⁾	FBL_CAN_ADDR_STANDARD / FBL_CAN_ADDR_EXTENDED	F
Can Diag Msg Phys Req Id ³⁾	0x0 ~ 0x1FFFFFFF	C

1) 진단 통신용 Physical Request 메시지의 STmin

2) 진단 통신용 Physical Request 메시지 주소 형식

3) 진단 통신용 Physical Request 메시지 ID

3.2.28 FblCanDiagMsgPhysResConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Can Diag Msg Phys Res Addr Format ¹⁾	FBL_CAN_ADDR_STANDARD / FBL_CAN_ADDR_EXTENDED	F
Can Diag Msg Phys Res Id ²⁾	0x0 ~ 0x1FFFFFFF	C

1) 진단 통신용 Physical Response 메시지 주소 형식

2) 진단 통신용 Physical Response 메시지 ID

3.2.29 FblCanBaudRateConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Can Baud Rate	FBL_CAN_100KBPS / FBL_CAN_250KBPS / FBL_CAN_500KBPS	C
Can Prop Seg ¹⁾	0 ~ 255	C
Can Seg1 ²⁾	1 ~ 255	C
Can Seg2 ³⁾	0 ~ 128	C
Can Sync Jump With ⁴⁾	1 ~ 128	C
Can Prescaler ⁵⁾	1 ~ 512	C

※ 자세한 Baud rate 설정은 MCU 매뉴얼을 참고

1) 진단 통신용 CAN Propagation delay in time quantas

2) 진단 통신용 CAN Phase segment 1 in time quantas

3) 진단 통신용 CAN Phase segment 2 in time quantas

- 4) 진단 통신용 CAN Synchronization jump width for the controller in time quantas
- 5) 진단 통신용 CAN Prescaler

3.2.30 FblCanFdBaudRateConfig

Parameter Name	Value	Category
Short Name	User Defined	C
Can Fd Baud Rate	FBL_CAN_1MBPS / FBL_CAN_2MBPS	C
Can Fd Prop Seg ¹⁾	0 ~ 32	C
Can Fd Seg1 ²⁾	0 ~ 32	C
Can Fd Seg2 ³⁾	0 ~ 16	C
Can Fd Sync Jump With ⁴⁾	0 ~ 16	C
Can Fd Trcv Delay Compensation Offset ⁵⁾	0 ~ 65535	C
Can Fd Tx Bit Rate Switch ⁶⁾	true / false	C
Can Fd Prescaler ⁷⁾	1 ~ 32	C

※ 자세한 Baud rate 설정은 MCU 매뉴얼을 참고

- 1) 진단 통신용 CAN-FD Propagation delay in time quantas
- 2) 진단 통신용 CAN-FD Phase segment 1 in time quantas
- 3) 진단 통신용 CAN-FD Phase segment 2 in time quantas
- 4) 진단 통신용 CAN-FD Synchronization jump width for the controller in time quantas
- 5) 진단 통신용 CAN-FD Transceiver Delay Compensation Offset in ns
- 6) 진단 통신용 CAN-FD Bit rate switching 여부
- 7) 진단 통신용 CAN-FD Prescaler

3.2.31 FblSpiControl

Parameter Name	Value	Category
Short Name	User Defined	C
FblSpiChannel ⁽¹⁾		C
FblSpiCPOL ⁽²⁾	CPOL_0/CPOL_1	C
FblSpiCPHA ⁽³⁾	CPHA_0/CPHA_1	C
FblSpiTransferStart ⁽⁴⁾	MSB_FIRST/LSB_FIRST	C
FblSpiSelect ⁽⁵⁾	SELECT_0 /SELECT_1 /SELECT_2 /SELECT_3	C
FblSpiClock ⁽⁶⁾	5 ~ 20000000	C
FblSpiDataWidth ⁽⁷⁾	4 ~ 32	C
FblSpiNumberOfSendCommand ⁽⁸⁾	0~n	C
FblSpiSelectPolarity ⁽⁹⁾	LOW_LEVEL/HIGH_LEVEL	C
FblSpiIn/ PortNum or PortAlphabet ⁽¹⁰⁾		C
FblSpiIn/ PinNum ⁽¹¹⁾	-	C

Parameter Name	Value	Category
FblSpiOut/ PortNum or PortAlphabet ⁽¹²⁾	-	C
FblSpiOut/ PinNum ⁽¹³⁾	-	C
FblSpiClk/ PortNum or PortAlphabet ⁽¹⁴⁾	-	C
FblSpiClk/ PinNum ⁽¹⁵⁾	-	C
FblSpiCs/ PortNum or PortAlphabet ⁽¹⁶⁾	-	C
FblSpiCs/ PinNum ⁽¹⁷⁾	-	C
FblSpiSendCommand ⁽¹⁸⁾	Sub-containers	C

- 1) The parameter is used to decide which SPI channel shall be used to manipulate SPI device, determined by the specification of MCU. EX: If the configured value is 5 in Cypress, this means that SCB5 shall be configured to support SPI device
- 2) The CPOL bit sets the polarity of the lock signal during idle state. The idle state is defined as the period when CS (chip select) is high and transitioning to low at the start of the transmission and when CS is low and transitioning to high at the end of the transmission. The value of this parameter is determined in the specification of SPI device
- 3) The CPHA bit selects the clock phase. Depending on the CPHA bit, the rising clock edge is used to sample and/or shift the data. The value of this parameter is determined in the specification of SPI device
- 4) Least significant bit or most significant bit is first transmitted. The value of this parameter is determined in the specification of SPI device
- 5) This parameter is used to decide which SPI device (ChipSelect) is activated in the current channel
- 6) Clock output for SPI channel, usually 1~10 MHz
- 7) The number of bits in a single command which shall be transmitted to SPI device
- 8) The number of TX commands which shall be transmitted to SPI device in the context of current SPI channel. These commands are defined in "FblSpiSendCommand" sub-containers
- 9) The parameter is used to decide which level of ChipSelect (high/low) shall active the SPI device. The value of this parameter is determined in the specification of SPI device

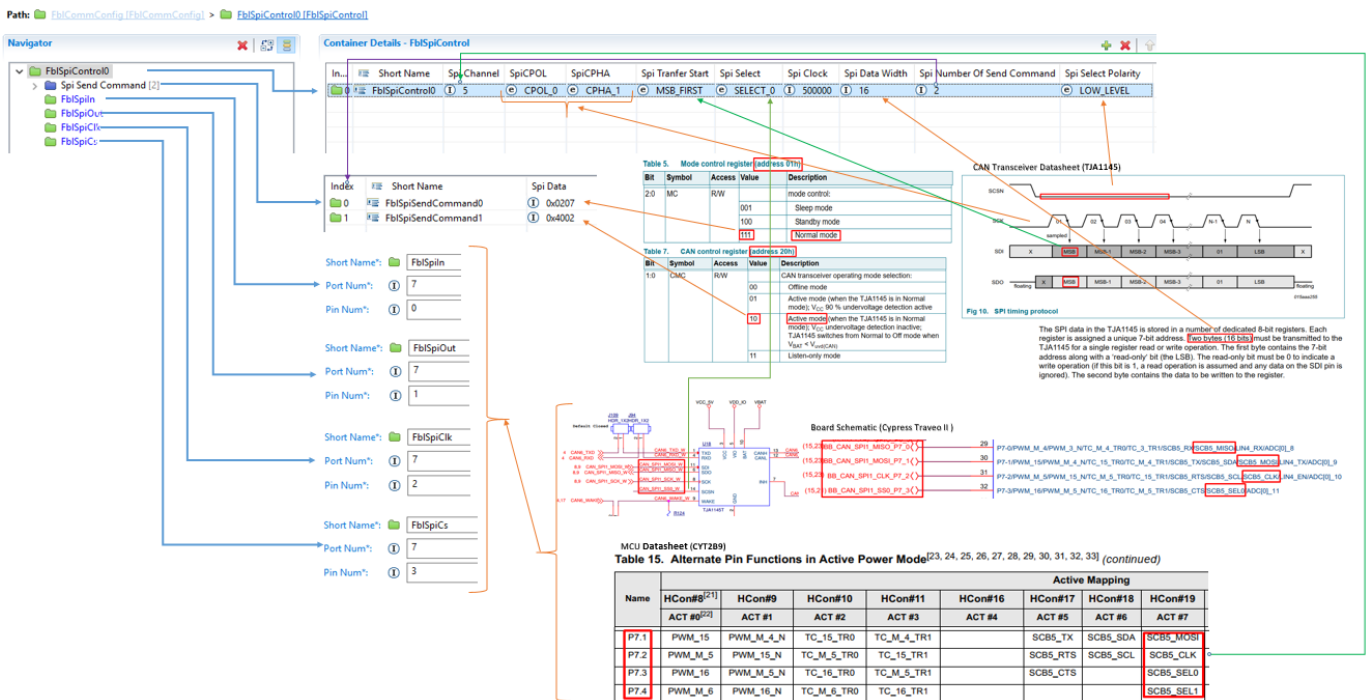
For "Hardware dependency" parameters (10, 11, 12, 13, 14, 15, 16, 17), refer to MCU manual and Hardware datasheet. User needs only input 1 type of parameter, PortNum or PortAlphabet, corresponding to current Hardware specification.

Below is “FbISpiSendCommand” sub-containers description:

Parameter Name	Value	Category
Short Name	User Defined	C
FbISpiData ¹⁾	4~32bit Hex value	C

- 1) Depend on “FbISpiDataWidth” and “FbISpiTransferStart”, SPI controller gets these data to transmit one-by-one after initialization

Below is an SPI configuration example for CAN Transceiver TJA1145 on Cypress:



3.2.32 FbIGpioPortPinControl

Parameter Name	Value	Category
Short Name	User Defined	C
PortPinIndex		C
FbIPortNum		C
FbIPinNum		C
FbIPortPinVal		C

- 1) Setting Index (0,1,2... sequential setting)
- 2) Port Number (ex. If P8_3 is Trcv Enable Pin, input value 8)
- 3) Pin Number (ex. If P8_3 is Trcv Enable Pin, input value 3)
- 4) Port Pin OutPut value (ex. High – 1, Low – 0)

3.2.33 FbICanTrcvSpc58x

Parameter Name	Value	Category
Short Name	User Defined	C
Can Trcv Enable Pin Pad ¹⁾		F
Can Trcv Enable Pin Value ²⁾		F

Parameter Name	Value	Category
Can Trcv Standby Pin Pad ³⁾		F
Can Trcv Standby Pin Value ⁴⁾		F
Can Trcv Error Pin Pad ⁵⁾		F

- 1) 진단 통신용 CAN Trcv Enable Pin Pad 설정
- 2) 진단 통신용 CAN Trcv Enable Pin Value 설정
- 3) 진단 통신용 CAN Trcv Standby Pin Pad 설정
- 4) 진단 통신용 CAN Trcv Standby Pin Value 설정
- 5) 진단 통신용 CAN Trcv Error Pin Pad 설정

3.2.34 FblCanTrcvRh850f1km

Parameter Name	Value	Category
Short Name	User Defined	C
Can Trcv Enable Pin SetToOutput ¹⁾		C
Can Trcv Enable Pin WriteDirect ²⁾		C
Can Trcv Standby Pin SetToOutput ³⁾		C
Can Trcv Standby Pin WriteDirect ⁴⁾		C
Can Trcv Error Pin SetToInput ⁵⁾		C

- 1) 진단 통신용 CAN Trcv Enable Pin **SetToOutput** 설정
- 2) 진단 통신용 CAN Trcv Enable Pin **WriteDirect** 설정
- 3) 진단 통신용 CAN Trcv Standby Pin **SetToOutput** 설정
- 4) 진단 통신용 CAN Trcv Standby Pin **WriteDirect** 설정
- 5) 진단 통신용 CAN Trcv Error Pin **SetToInput** 설정

4. Application Programming Interface (API)

4.1 Type Definitions

4.1.1 Entry Point

4.1.1.1 Fbl_SwEntryPointType

Name	Fbl_SwEntryPointType	
Type	Enumeration	
Range	FBL_SW_NO_RTSW	FBL 모드 (유효한 RTSW 펌웨어가 없음)
	FBL_SW_ENTRY_POINT_1	Partition A RTSW 모드 FblFlashConfig / MainSwEntryAddress1 파라미터 설정 값 대응 인덱스
	FBL_SW_ENTRY_POINT_2	Partition B RTSW 모드 FblFlashConfig / MainSwEntryAddress2 파라미터 설정 값 대응 인덱스
Description	이중화시 적용할 Main SW 시작 주소 인덱스	

* Fbl_Applf.h

typedef enum

```
{
    FBL_SW_NO_RTSW,
    FBL_SW_ENTRY_POINT_1,
    FBL_SW_ENTRY_POINT_2
} Fbl_SwEntryPointType;
```

4.1.2 압축 해제

4.1.2.1 Fbl_DecompReturnType

Name	Fbl_DecompReturnType	
Type	Enumeration	
Range	FBL_DECOMP_NOT_OK	압축 해제 실패
	FBL_DECOMP_BLOCK_NOT_FINISHED	압축 해제는 성공했으나 전체 데이터에 대한 압축해제가 완료되지 않음
	FBL_DECOMP_BLOCK_FINISHED	전체 데이터 압축해제 완료
Description	압축 해제 결과	

* Fbl_Dcmpr.h

typedef enum

```
{
    FBL_DECOMP_NOT_OK,
    FBL_DECOMP_BLOCK_NOT_FINISHED,
    FBL_DECOMP_BLOCK_FINISHED
} Fbl_DecompReturnType;
```

4.1.2.2 Fbl_DecompDataContext

Name	Fbl_DecompDataContext
Type	Structure

Range	boolean	bl_first	압축해제 시작 여부
	uint32	u32_totalInputDataLength	압축된 입력 데이터의 전체 길이
Description	압축해제 모듈에서 사용하는 context 구조체로 필요시 사용자가 자유롭게 정의하여 사용 가능		

* Fbl_Dcmpr.h

typedef struct

```
{
    /* Flag to determine start of decompression */
    boolean bl_first;

    /* The length of the entire compressed input data */
    uint32 u32_totalInputDataLength;
} Fbl_DecompDataContext;
```

4.1.3 Public Key

4.1.3.1 Sec_Gau8_DiagServPubKey

Name	Sec_Gau8_DiagServPubKey
Type	uint8 array
Size	256
Description	Secure Access 인증 과정에서 사용하는 RSA Public Key. 필요시 사용자가 변경 사용 가능

* Fbl_SecData.c

4.1.3.2 Sec_Gau8_ImageServPubKey

Name	Sec_Gau8_ImageServPubKey
Type	uint8 array
Size	256
Description	Secure Flash 인증 과정에서 사용하는 RSA Public Key. 필요시 사용자가 변경 사용 가능

* Fbl_SecData.c

4.1.4 Delimiter

4.1.4.1 Sec_Gau8_Delimiter

Name	Sec_Gau8_Delimiter
Type	Uint32 array
Size	aSIMS : 32 FST : 40
Description	전자서명 생성 후 부여받는 Delimiter 값. 필요시 사용자가 변경 사용 가능

* Fbl_SecData.c

4.2 Functions

4.2.1 External Devices

4.2.1.1 Fbl_BeforeCommExtInit

Function Name	Fbl_BeforeCommExtInit
Syntax	void Fbl_BeforeCommExtInit(void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Description	Fbl 디바이스 초기화 과정중 통신 드라이버 초기화 전 호출되는 API

* Fbl_Applf.c

```
void Fbl_BeforeCommExtInit(void)
```

```
{
    /* Initialize external devices before communication device initialization. */
}
```

4.2.1.2 Fbl_AfterCommExtInit

Function Name	Fbl_AfterCommExtInit
Syntax	void Fbl_AfterCommExtInit(void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Description	Fbl 디바이스 초기화 과정중 통신 드라이버 초기화 후 호출되는 API

* Fbl_Applf.c

```
void Fbl_AfterCommExtInit(void)
```

```
{
    /* Initialize external devices after communication device initialization. */
}
```

4.2.1.3 Fbl_ExtDeinit

Function Name	Fbl_ExtDeinit
Syntax	void Fbl_ExtDeinit(void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Description	Fbl 모드에서 reset 직전에 호출되는 API

* Fbl_Applf.c

```
void Fbl_ExtDeinit(void)
```

```
{
    /* Deinitialize the external devices just before the reset */
}
```

4.2.1.4 Fbl_ExtPolling

Function Name	Fbl_ExtPolling
Syntax	void Fbl_ExtPolling(void)

Sync/Async	Sync
Reentrancy	Non Re-entrant
Description	FBL내에서 주기적으로 호출되는 API Fbl 에서 while문을 통해 주기 적으로 호출됨. Fbl 은 task 가 없으므로 빠르면 수 us 에서 늦으면 수 ms 주기로 가변적으로 호출. RAM에서 수행 되는 코드이므로 추가적인 function call 없이 구현되어야 함.

```

* Fbl_Applf.c
void Fbl_ExtPolling(void)
{
    /* Performed aperiodically */
    /* Since the code is executed in RAM, it should be implemented
       without additional function call */
}

```

4.2.1.5 Fbl_BeforeFblSpecificInit

Function Name	Fbl_BeforeFblSpecificInit
Syntax	void Fbl_BeforeFblSpecificInit (void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Description	FBL MODE에서 PLL 초기화 전에 APP에서 처리가 필요할 때 사용

```

* Fbl_Applf.c
void Fbl_BeforeFblSpecificInit(void)
{
    /* Initialize before FBL specific initialization
       This function is called Before FBL PLL initalization */
}

```

4.2.1.6 Fbl_BeforeRoutineCtrlEraseInit

Function Name	Fbl_BeforeRoutineCtrlEraseInit
Syntax	uint32 Fbl_BeforeRoutineCtrlEraseInit (void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Description	Reprogramming의 RoutineCtrl Erase 서비스 시행 중 Hsm_TempStop이 호출 되기 전에 APP에서 처리가 필요할 때 사용

```

* Fbl_Applf.c
/*
    Function called before Hsm_TempStop during RoutineControlErase service.
*/
uint32 Fbl_BeforeRoutineCtrlEraseInit(void)
{
    uint32 Ldt_RetValue;

    Ldt_RetValue = E_OK;

    return Ldt_RetValue;
}

```

4.2.1.7 Fbl_AfterRoutineCtrlChkInit

Function Name	Fbl_AfterRoutineCtrlChkInit
Syntax	uint32 Fbl_AfterRoutineCtrlChkInit (void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Description	Reprogramming의 RoutineCtrl Dependency check 서비스 시행 중 PartitionFlag 또는 Security Key를 FLASH에 Write한 이후 APP에서 처리가 필요할 때 사용

* Fbl_Applf.c

/*

Function called after writing PartitionFlag during RoutineCtrlCheck service.

*/

uint32 Fbl_AfterRoutineCtrlChkInit(void)

```
{
    uint32 Ldt_RetValue;
```

```
    Ldt_RetValue = E_OK;
```

```
    return Ldt_RetValue;
```

```
}
```

4.2.2 Entry Point

4.2.2.1 Fbl_SelectEntryPoint

Function Name	Fbl_SelectEntryPoint
Syntax	Fbl_SwEntryPointType Fbl_SelectEntryPoint(void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Return Value	Fbl_SwEntryPointType Entry point index(Fbl_SwEntryPointType 설명 참조)
Description	Fbl_GetMainSwEntryPointAddress 에 의해 호출되는 함수로 Main SW 시작 주소 선택을 위해 호출되는 API 메모리 이중화시 사용자가 구현한 조건에 따라 선택된 entry point index 를 리턴 유효한 RTSW 펌웨어가 없을 경우 FBL 모드 index 를 리턴

* Fbl_Applf.c

Fbl_SwEntryPointType Fbl_SelectEntryPoint(void)

```
{
    Fbl_SwEntryPointType Ldt_SwEntryPoint;

    #if (FBL_MEMORY_SWAP_ENABLE == STD_ON)
    /*****
    /*                      메모리 이중화 기능 적용                      */
    /*****
    #if ((FBL_SUPPORT_MCU == FBL_MCU_TC38X) || (FBL_SUPPORT_MCU == FBL_MCU_TC39X) || \
    (FBL_SUPPORT_MCU == FBL_MCU_TC36X) || (FBL_SUPPORT_MCU == FBL_MCU_TC37X)
    )
    /*****
```

```

* HW MEMORY SWAP 기능이 지원되는 MCU 는
* partition A, B swap 에 따른 memory mapping 을 MCU 에서 해주므로
* partition 에 따른 별도의 start address 제어가 필요없다.
* Fbl_SelectEntryPoint callout 내에서
* 정상적인 RTSW 펌웨어 리프로그래밍 성공 여부만 판단하면 된다.
*****/

/*
    정상적인 RTSW 펌웨어가 있는지 판단 로직 구현
    ex) 사전에 정해놓은 플래시 영역에 특정 패턴이 있는지 검사
        패턴은 OTA 과정에서 펌웨어 flashing 성공시 writing(Fbl_WritePartitionFlag callout)
*/
if (0)
{
    /* RTSW 모드 */
    Ldt_SwEntryPoint = FBL_SW_ENTRY_POINT_1;
}
else
{
    /* FBL 모드 */
    Ldt_SwEntryPoint = FBL_SW_NO_RTSW;
}

#elif (FBL_SUPPORT_MCU == FBL_MCU_CYT2B98X)
/*****
* HW MEMORY SWAP 기능이 지원되는 MCU 는
* partition A, B swap 에 따른 memory mapping 을 MCU 에서 해주므로
* partition 에 따른 별도의 start address 제어가 필요없다.
* Fbl_SelectEntryPoint callout 내에서
* 정상적인 RTSW 펌웨어 리프로그래밍 성공 여부만 판단하면 된다.
* FBL_MCU_CYT2B98X의 경우는 HW 특성상 Reset 이후 Dual Memory Enable
* & Memory Swap 진행이 되어야 한다.
*****/

/*
    1) Dual Memory 기능 활성화

    2) 정상적인 RTSW 펌웨어가 있는지 판단 로직 구현 & Active Bank 선정
        ex) 사전에 정해놓은 플래시 영역에 특정 패턴이 있는지 검사
            패턴은 OTA 과정에서 펌웨어 flashing 성공시 writing(Fbl_WritePartitionFlag callout)
            이 결과를 통해 Active Bank를 선정

    3) Memory Swap
        ex) 선정된 Bank 를 Active Bank 로 적용
*/
if (0)
{
    /* RTSW 모드 */
    Ldt_SwEntryPoint = FBL_SW_ENTRY_POINT_1;
}

```

```

else
{
    /* FBL 모드 */
    Ldt_SwEntryPoint = FBL_SW_NO_RTSM;
}

#else
/*****
* HW MEMORY SWAP 기능이 지원되지 MCU 계열은
* 부팅시 partition A, B swap 에 따른 start address 제어가 필요하다.
* Fbl_SelectEntryPoint callout 내에서
* 어떤 partition 의 start address 를 수행할지를 결정해야 한다.
*****/

/*
    Partition A or Partition B 에 정상적인 펌웨어가 있는지 판단 로직 구현
    ex) 사전에 정해놓은 플래시 영역에 어떤 partition RTSM 를 수행할지 특정 패턴을 확인
        패턴은 OTA 과정에서 펌웨어 flashing 성공시 writing(Fbl_WritePartitionFlag callout)
*/
if (0)
{
    /* Partition A RTSM 모드 */
    Ldt_SwEntryPoint = FBL_SW_ENTRY_POINT_1;
}
else if (0)
{
    /* Partition B RTSM 모드 */
    Ldt_SwEntryPoint = FBL_SW_ENTRY_POINT_2;
}
else
{
    /* FBL 모드 */
    Ldt_SwEntryPoint = FBL_SW_NO_RTSM;
}
#endif

#else

/*****
/*                      메모리 이중화 기능 미적용                      */
*****/
if ((*(const uint32*)MAIN_SW_SECURITY_KEY_ADDR) == FBL_SECURITY_KEY_VALUE)
{
    /* RTSM 모드 */
    Ldt_SwEntryPoint = FBL_SW_ENTRY_POINT_1;
}
else
{
    /* FBL 모드 */
    Ldt_SwEntryPoint = FBL_SW_NO_RTSM;
}

```


#endif

return Ldt_SwEntryPoint;

}

4.2.2.2 Fbl_WritePartitionFlag

Function Name	Fbl_WritePartitionFlag
Syntax	uint32 Fbl_WritePartitionFlag(void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Return Value	uint32 Partition flag writing 성공/실패 여부
Description	이중화시 부팅과정에서 Fbl 이 어떤 Partition 을 실행해야 하는지 판단할 근거가 되는 Partition Flag 를 writing 하는 callout. RoutineControl checkProgrammingDependencies 서비스 성공시 Fbl 에 의해 호출된다.
Precondition	OTA 과정이 아닌 진단기를 통한 직접 리프로그래밍

* Fbl_Applf.c

uint32 Fbl_WritePartitionFlag(void)

```
{
    uint32 Lu32_Ret;

    #if (FBL_MEMORY_SWAP_ENABLE == STD_ON)
    /*****
    /*                      메모리 이중화 기능 적용                      */
    *****/

    #if ((FBL_SUPPORT_MCU == FBL_MCU_TC38X) || (FBL_SUPPORT_MCU == FBL_MCU_TC39X) || \
        (FBL_SUPPORT_MCU == FBL_MCU_TC36X) || (FBL_SUPPORT_MCU == FBL_MCU_TC37X)
        )
    /*****
    * HW MEMORY SWAP 기능이 지원되는 MCU 이므로
    * Fbl_SelectEntryPoint callout 에서 정상적인 RTSW 펌웨어가 있는지 판단할 수 있는
    * 사용자만의 PartitionFlag 를 writing 해야 한다.
    *****/

    /*
    RTSW 펌웨어가 정상적으로 flashing(Secure Flash 절차 성공)
    되었는지에 대한 PartitionFlag writing 로직 구현
    writing 성공시 E_OK 리턴, writing 실패시 E_NOT_OK 리턴
    */
    if (/* 사용자 펌웨어 Partition Flag Writing 결과 확인 */)
    {
        Lu32_Ret = E_OK;
    }
    else
    {
        Lu32_Ret = E_NOT_OK;
    }
    #else
```

```

/*****
* HW MEMORY SWAP 기능이 지원되지 MCU 계열이므로
* 부팅시 어떤 partition 을 수행해야 할지에 대한 정보가 필요하다.
* Fbl_SelectEntryPoint callout 에서 어떤 partition 의 RTSW 펌웨어가 수행되어야 하는지
* 사용자만의 PartitionFlag 를 writing 해야 한다.
*****/

/*
  어떤 partition 의 RTSW 펌웨어가 정상적으로 flashing(Secure Flash 절차 성공)
  되었는지에 대한 PartitionFlag writing 로직 구현
  writing 성공시 E_OK 리턴, writing 실패시 E_NOT_OK 리턴
*/
if (/* 사용자 Partition A or B 펌웨어 Partition Flag Writing 결과 확인 */)
{
    Lu32_Ret = E_OK;
}
else
{
    Lu32_Ret = E_NOT_OK;
}
#endif

#else
/*****
/*          메모리 이중화 기능 미적용          */
*****/
/*****
* FBL 에서 RTSW 펌웨어 리프로그래밍 성공 후 자체적으로
* MAIN_SW_SECURITY_KEY_ADDR 에 FBL_SECURITY_KEY_VALUE 패턴을 기록하므로 별도의 로직 필요없음
*****/
Lu32_Ret = E_OK;

#endif

return Lu32_Ret;
}

```

***note** Block Id 값을 인자로 갖는데, MainSW 가 아닌 경우에는 위 logic 이 타지 않도록 처리해야 할 필요가 있는 경우, user 가 이를 위 함수에 구현해 주어야 한다.

4.2.2.3 Fbl_GetActivePartitionBlkAddress

Function Name	Fbl_GetActivePartitionBlkAddress	
Syntax	void Fbl_GetActivePartitionBlkAddress(uint32* headerAddr, uint32* trailerAddr)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
In-Out Parameter	uint32*	FBL에 설정된 BlkHeaderAddress
	uint32*	FBL에 설정된 BlkTrailerAddress

Return Value	N/A	N/A
Description	OTA 이중화 제어기이면서 VersionCheck 기능 (다운그레이드방지) 사용시에 해당하는 API이다. FBL 은 현재 Active Partition이 무엇인지 알 수 없어 현재 버전을 어디서 읽어올지를 결정할 수 없다. Active Partition은 User 로직으로 인해 알 수 있기 때문에, 이 함수를 통해 Active Partition을 감지하고 감지된 Partition의 headerAddress 와 trailer address를 In-Out 파라미터로 전달한다. 전달된 Address는 현재 버전을 읽어올 때에만 (Read) 사용되고 나머지는 사용되지 않는다. 일반적인 경우, 전달된 In-Out 파라미터에 Offset을 더하는 식으로 구현할 수 있다. 결론적으로, B 파티션 Active Partition인 경우에는 수정된 주소를 headerAddr, trailerAddr 에 저장한다.	
Precondition	메모리 이중화를 적용하는 경우에만 해당한다. 이 함수는 RoutineControl-Erase 서비스 수행 시에 불린다.	

4.2.3 압축 해제

4.2.3.1 Fbl_DecompressInit

Function Name	Fbl_DecompressInit	
Syntax	void Fbl_DecompressInit(uint32 Lu32_TotalInputDataLength)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	Lu32_TotalInputDataLength	압축된 입력 데이터의 전체 길이
Description	Decompress 기능이 설정되어 있는 경우 RequestDownload 서비스 수행시 호출되는 함수로 decompress 모듈의 초기화를 구현한다.	

* Fbl_Dcmpr.c

```
void Fbl_DecompressInit(void)
```

```
{
    Fbl_GstDecompressContext.bl_first = TRUE;
    Fbl_GstDecompressContext.u32_totalInputDataLength = Lu32_TotalInputDataLength;
}
```

4.2.3.2 Fbl_DeompExpandData

Function Name	Fbl_DeompExpandData	
Syntax	Fbl_DeompReturnType Fbl_DeompExpandData(uint16* inbuffBytesLeft, const uint8* inBuffer, uint16* outbuffBytesLeft, uint8* outBuffer)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	inBuffer	압축된 데이터 입력 버퍼
Parameters (out)	outBuffer	압축해제한 데이터 출력 버퍼
Parameters (inout)	inbuffBytesLeft	압축된 데이터 버퍼 중 사용 가능한 버퍼의 크기로 사용 후 미사용한 버퍼의 크기로 업데이트한다. ex) 20 바이트 크기의 사용 가능한 버퍼 중 12 바이트 사용시 해당 변수를 8 로 업데이트
	outbuffBytesLeft	압축해제한 데이터 출력 버퍼중 채울 수 있는 버퍼의 크기로 채운 후 미사용한 버퍼의 크기로 업데이트한다.

		ex) 20 바이트 크기의 사용 가능한 버퍼 중 12 바이트 사용시 해당 변수를 8 로 업데이트
Return Value	Fbl_DecompReturnType	함수 수행 결과
Description	압축 데이터 버퍼(inBuffer)와 크기(inbuffBytesLeft), 출력 버퍼(outBuffer)와 크기(outbuffBytesLeft)를 받아 압축 데이터를 압축해제하여 outBuffer 에 채운 후, 사용 후 남은 inBuffer 의 크기를 업데이트하고 데이터를 채운 outBuffer 의 크기를 업데이트하여 적절한 결과(Fbl_DecompReturnType 설명 참조)를 리턴한다.	

* Fbl_Dcmpr.c

Fbl_DecompReturnType Fbl_DecompExpandData(

```
uint16* inbuffBytesLeft, /* 입력 데이터(압축된 데이터) 의 사용 가능한 크기 & 미사용 크기 */
const uint8* inBuffer, /* 입력 데이터(압축된 데이터) 버퍼 포인터 */
uint16* outbuffBytesLeft, /* 출력 데이터(압축해제된 데이터) 의 사용 가능한 크기 & 미사용 크기 */
uint8* outBuffer /* 출력 데이터(압축해제된 데이터) 버퍼 포인터 */
)
```

```
{
  #if (FBL_DECOMPRESS_ENABLE == STD_ON)
  Fbl_DecompReturnType Ldt_RetValue;
```

```
if (Fbl_GstDecompressContext.bl_first == TRUE)
{
  /* 압축해제 최초 시작시 수행할 코드 */
  Fbl_GstDecompressContext.bl_first = FALSE;
}
else
{
}
```

```
/* 압축해제 수행 */
```

```
/*
```

inBuffer 로부터 최대 inbuffBytesLeft 크기만큼 입력 데이터를 사용하여 압축해제 진행
Decompress 과정 중 압축해제한 데이터가 outbuffBytesLeft 와 일치하지 않을 가능성이 많으므로
사용자가 관리하는 버퍼내에 압축해제 데이터를 저장

```
*/
```

```
/* 압축해제 결과 업데이트 */
```

```
/*
```

1) outBuffer 에 최대 outbuffBytesLeft 만큼 복사하도록 한다

2) inbuffBytesLeft & outbuffBytesLeft 업데이트를 해준다

ex) inbuffBytesLeft & outbuffBytesLeft 업데이트 예제

Decompress 진행 전

inbuffBytesLeft : 100, outbuffBytesLeft : 200

Decompress 진행 후

ex1) inBuffer 에서 100 byte 사용, outBuffer 에 128 byte 채웠을 경우

inbuffBytesLeft : 0, outbuffBytesLeft : 72

ex2) inBuffer 에서 100 byte 사용, outBuffer 에 200 byte 채웠을 경우

inbuffBytesLeft : 0, outbuffBytesLeft : 0

ex3) inBuffer 에서 40 byte 사용, outBuffer 에 128 byte 채웠을 경우

inbuffBytesLeft : 60, outbuffBytesLeft : 72

ex4) inBuffer 에서 40 byte 사용, outBuffer 에 200 byte 채웠을 경우
inbuffBytesLeft : 60, outbuffBytesLeft : 0

3) 압축해제 결과에 따른 리턴값(Fbl_DecompReturnType 설명 참조)을 설정한다

ex1) 압축 해제는 성공했으나 전체 데이터에 대한 압축해제가 완료되지 않음

Ldt_RetValue = FBL_DECOMP_BLOCK_NOT_FINISHED;

ex2) 전체 데이터 압축해제 완료

Ldt_RetValue = FBL_DECOMP_BLOCK_FINISHED;

ex3) 압축 해제 실패

Ldt_RetValue = FBL_DECOMP_NOT_OK;

*/

return Ldt_RetValue;

#else

return FBL_DECOMP_NOT_OK;

#endif

}

4.2.4 보안라이브러리

본 FBL은 HKMC 보안라이브러리 (HAE_CryptoLib, ASK, HSM2.0, SecureFlash2.0) 을 사용하고 있다. FBL은 이러한 보안라이브러리들을 직접적으로 호출하지 않고 User Interface Layer를 통해 최종적으로 보안라이브러리를 호출하도록 구성되어 있다.

4.2.4.1 Crylf_PrngInit

Function Name	Crylf_PrngInit
Syntax	void Crylf_PrngInit(void)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Parameters (in)	void
Description	SW 보안라이브러리로 구현되는 PRNG 사용이 필요한 경우 (HSM 미사용의 경우) PRNG 모듈의 초기화를 구현한다.

* Fbl_Crylf.c

void Crylf_PrngInit(void)

{
}

4.2.4.2 Crylf_PrngReseed

Function Name	Crylf_PrngReseed
Syntax	void Crylf_PrngReseed(uint8* entropy)
Sync/Async	Sync
Reentrancy	Non Re-entrant
Parameters (in)	uint8* entropy 16Byte 엔트로피 (랜덤변수)
Description	SW 보안라이브러리로 구현되는 PRNG 사용이 필요한 경우 PRNG의 초기값으로 쓰일 엔트로피를 입력받아 SW 보안라이브러리의 PRNG 모듈의 Seed를 넣어준다. 전달받은 Entropy 대신 유저가 직접 생성한 Entropy 또한 사용 가능하다.

```
* Fbl_Crylf.c
void Crylf_PrngReseed(uint8* entropy)
{
}
```

4.2.4.3 Crylf_PrngGetRand

Function Name	Crylf_PrngGetRand	
Syntax	boolean Crylf_PrngGetRand(uint8* randomNumberBuffer, const uint32 randomNumberLength)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	const uint32 randomNumberLength	요청 난수 길이
Parameters (out)	uint8* randomNumberBuffer	난수 저장 포인터
Description	FBL 로부터 난수 생성 요청을 받아 난수를 반환한다.	

```
* Fbl_Crylf.c
boolean Crylf_PrngGetRand(uint8* randomNumberBuffer,
const uint32 randomNumberLength)
{
}
```

4.2.4.4 Crylf_KeyGenerate

Function Name	Crylf_KeyGenerate	
Syntax	boolean Crylf_KeyGenerate(uint8* seed, uint8* key)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint8* seed	입력 Seed
Parameters (out)	uint8* key	입력 Seed에 의해 생성되는 Key
Return Value	boolean	함수 수행 결과
Description	FBL 로부터 Seed를 입력받아 Key를 반환해준다. Seed & Key 의 Length는 FBL에 Seed-Key 설정 시 4Byte, ASK 설정 시 8byte로 고정 이다.	

```
* Fbl_Crylf.c
boolean Crylf_KeyGenerate (uint8* seed, uint8* key)
{
}
```

4.2.4.5 Crylf_Sha

Function Name	Crylf_Sha	
Syntax	boolean Crylf_Sha(uint32 hashAlgorithm, const uint8* data, uint32 dataLength, uint8* digest)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint32 hashAlgorithm	0 : SHA160 1 : SHA256

	const uint8* data	입력 Data
	uint32 dataLength	입력 Data Length
Parameters (out)	uint8* digest	Hash 결과
Return Value	boolean	함수 수행 결과
Description	입력 hashAlgorithm, data, dataLength를 통해 보안라이브러리 활용하여 digest 를 생성한다. 본 함수는 Hae_CryptoLib의 Hmg_Sha160(), Hmg_Sha256() 함수와 대응된다.	

* Fbl_Crylf.c

```
boolean Crylf_Sha(uint32 hashAlgorithm, const uint8* data,
    uint32 dataLength, uint8* digest)
{
}
```

4.2.4.6 Crylf_ShaStart

Function Name	Crylf_ShaStart	
Syntax	boolean Crylf_ShaStart(uint32 hashAlgorithm)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint32 hashAlgorithm	0 : SHA160 1 : SHA256
Return Value	boolean	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 Hash 알고리즘 호출에 대응되는 함수이다. 입력 hashAlgorithm 을 활용하여 Hash 모듈을 초기화한다. Hae_CryptoLib의 Hmg_Sha160Start, Hmg_Sha256Start 에 대응된다.	

* Fbl_Crylf.c

```
boolean Crylf_ShaStart(uint32 hashAlgorithm)
{
}
```

4.2.4.7 Crylf_ShaUpdate

Function Name	Crylf_ShaUpdate	
Syntax	boolean Crylf_ShaUpdate(uint32 hashAlgorithm, const uint8* data, uint32 dataLength)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint32 hashAlgorithm	0 : SHA160 1 : SHA256
	const uint8* data	입력 데이터 포인터
	uint32 dataLength	입력 데이터 길이
Return Value	boolean	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 Hash 알고리즘 호출에 대응되는 함수이다. 입력 hashAlgorithm 을 활용하여 Hash Update를 수행한다. Hae_CryptoLib의 Hmg_Sha160Update, Hmg_Sha256Update 에 대응된다.	

* Fbl_Crylf.c

```
boolean Crylf_ShaUpdate(uint32 hashAlgorithm,
```

```
const uint8* data, uint32 dataLength)
{
}
```

4.2.4.8 Crylf_ShaFinish

Function Name	Crylf_ShaFinish	
Syntax	boolean Crylf_ShaFinish(uint32 hashAlgorithm, uint8* digest)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint32 hashAlgorithm	0 : SHA160 1 : SHA256
	uint8* digest	Hash 결과 포인터
Return Value	boolean	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 Hash 알고리즘 호출에 대응되는 함수이다. 입력 hashAlgorithm 을 활용하여 Hash Finish를 수행한다. Hae_CryptoLib의 Hmg_Sha160Finish, Hmg_Sha256Finish 에 대응된다.	

* Fbl_Crylf.c

```
boolean Crylf_ShaFinish(uint32 hashAlgorithm, uint8* digest)
{
}
```

4.2.4.9 Crylf_RsaPkcs1v15VerifyStart

Function Name	Crylf_RsaPkcs1v15VerifyStart	
Syntax	boolean Crylf_RsaPkcs1v15VerifyStart(const uint8* signature, const uint8* modulus, uint32 exponent)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	const uint8* signature	입력 signature 포인터
	const uint8* modulus	입력 modulus 포인터
	uint32 exponent	입력 exponent
Return Value	boolean	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 RsaPkcs1v15 알고리즘 호출에 대응되는 함수이다. Hae_CryptoLib의 Hmg_RsaPkcs1v15VerifyStart 함수와 대응된다.	

* Fbl_Crylf.c

```
boolean Crypto_Hae_RsaPkcs1v15VerifyStart(const uint8* signature,
const uint8* modulus, uint32 exponent)
{
}
```

4.2.4.10 Crylf_RsaPkcs1v15VerifyUpdate

Function Name	Crylf_RsaPkcs1v15VerifyUpdate
Syntax	boolean Crylf_RsaPkcs1v15VerifyUpdate(void)

Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	boolean	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 RsaPkcs1v15 알고리즘 호출에 대응되는 함수이다. Hae_CryptoLib의 Hmg_RsaPkcs1v15VerifyUpdate 함수와 대응된다.	

* Fbl_Crylf.c

```
boolean Crylf_RsaPkcs1v15VerifyUpdate(void)
{
}
```

4.2.4.11 Crylf_RsaPkcs1v15VerifyFinish

Function Name	Crylf_RsaPkcs1v15VerifyFinish	
Syntax	boolean Crylf_RsaPkcs1v15VerifyFinish(uint32 hashAlgorithm, const uint8* digset)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint32 hashAlgorithm	0 : SHA160 1 : SHA256
	uint8* digset	Hash된 Message의 값 포인터
Return Value	boolean	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 RsaPkcs1v15 알고리즘 호출에 대응되는 함수이다. Hae_CryptoLib의 Hmg_RsaPkcs1v15VerifyFinish 함수와 대응된다.	

* Fbl_Crylf.c

```
boolean Crylf_RsaPkcs1v15VerifyFinish(uint32 hashAlgorithm, const uint8* digset)
{
}
```

4.2.4.12 Crylf_SFDecryptStart

Function Name	Crylf_SFDecryptStart	
Syntax	Std_ReturnType Crylf_SFDecryptStart(uint8* iv, const uint8* decKey, const uint8 decKeyLen, const uint8* macOfDecKey, const uint8* macKey, const uint8 macKeyLen)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint8* iv	초기 벡터 값
	const uint8* decKey	평웨어 키
	const uint8 decKeyLen	평웨어 키 길이
	const uint8* macOfDecKey	평웨어키의 MAC
	const uint8* macKey	MAC을 계산하기 위한 키

	const uint8 macKeyLen	MAC을 계산하기 위한 키 길이
Return Value	Std_ReturnType	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 SecureFlash2.0 Block Decrypt Start 알고리즘 호출에 대응되는 함수이다. SecureFlash 2.0 Library의 의 SecureFlash_DecryptStart 함수와 대응된다.	

* Fbl_Crylf.c

```
Std_ReturnType Crylf_SFDecryptStart(uint8* iv, const uint8* decKey,
    const uint8 decKeyLen, const uint8* macOfDecKey, const uint8* macKey,
    const uint8 macKeyLen)
{
}
```

4.2.4.13 Crylf_SFDecryptUpdate

Function Name	Crylf_SFDecryptUpdate	
Syntax	Std_ReturnType Crylf_SFDecryptUpdate(uint8* out, const uint8* in, const uint32 inLen)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	const uint8* in	복호화 대상 Data
	const uint32 inLen	복호화 대상 Data 길이
Parameters (out)	uint8* out	복호화된 Data
Return Value	Std_ReturnType	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 SecureFlash2.0 Block Decrypt Update 알고리즘 호출에 대응되는 함수이다. SecureFlash 2.0 Library의 의 SecureFlash_DecryptUpdate 함수와 대응된다.	

* Fbl_Crylf.c

```
Std_ReturnType Crylf_SFDecryptUpdate(uint8* out, const uint8* in,
    const uint32 inLen)
{
}
```

4.2.4.14 Crylf_SFDecryptFinish

Function Name	Crylf_SFDecryptFinish	
Syntax	Std_ReturnType Crylf_SFDecryptFinish(uint32* totalLen)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (out)	uint32* totalLen	복호화 된 Data 총 크기
Return Value	Std_ReturnType	함수 수행 결과
Description	본 함수는 Hash 알고리즘의 Streaming Call 방식의 SecureFlash2.0 Block Decrypt Finish 알고리즘 호출에 대응되는 함수이다. SecureFlash 2.0 Library의 의 SecureFlash_DecryptFinish 함수와 대응된다.	

* Fbl_Crylf.c

```
Std_ReturnType Crylf_SFDecryptFinish(uint32* totalLen)
{
}
```

4.2.4.15 Crylf_HsmInit

Function Name	Crylf_SFDecryptFinish	
Syntax	Std_ReturnType Crylf_HsmInit(void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Std_ReturnType	함수 수행 결과
Description	본 함수는 HSM init 을 수행하기위해 부팅 초기에 호출된다. FBL은 인터럽트를 사용하지 않기 때문에 Polling 방식으로 HSM이 사용되도록 Initialize 해야 한다.	

* Fbl_Crylf.c

```
Std_ReturnType Crylf_HsmInit(void)
{
}
```

4.2.4.16 Crylf_HsmTempStop

Function Name	Crylf_HsmTempStop	
Syntax	Std_ReturnType Crylf_HsmTempStop(void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Std_ReturnType	함수 수행 결과
Description	본 함수는 Flash Memory 접근 전 호출된다. 본 함수에서 HSM이 Flash를 Read 하지 못하도록 조치해야 한다 (방식은 HSM 마다 다름)	

* Fbl_Crylf.c

```
Std_ReturnType Crylf_HsmTempStop(void)
{
}
```

4.2.4.17 Crylf_HsmRestart

Function Name	Crylf_HsmRestart	
Syntax	Std_ReturnType Crylf_HsmRestart(void)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Return Value	Std_ReturnType	함수 수행 결과
Description	본 함수는 HSM TempStop 이후 Flash Memory 접근 후 호출된다. 본 함수에서 HSM이 정상적으로 수행될 수 있도록 조치해야한다.	

* Fbl_Crylf.c

```
Std_ReturnType Crylf_HsmRestart(void)
{
}
```

4.2.4.18 Crylf_DeriveKey

Function Name	Crylf_DeriveKey	
Syntax	Std_ReturnType Crylf_DeriveKey(uint16 pskIndex, uint32 dkLen, const uint8* password, uint32 pLen, const uint8* salt, uint32 sLen, uint32 icount, uint8* dk)	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint16 pskIndex	Password로 사용할 키 번호
	const uint8* password	패스워드 포인터
	uint32 pLen	패스워드 길이
	const uint8* salt	Salt(난수) 값 포인터
	uint32 sLen	Salt(난수) 길이
	uint32 icount	내부 연산 반복 횟수
Parameters (out)	uint32 dkLen	유도 키 길이
	uint8* dk	유도 키 포인터
Return Value	Std_ReturnType	함수 수행 결과
Description	본 함수는 PBKDF2 수행을 위한 함수이다. 입력 Parameter를 통해 HSM 인터페이스에 맞도록 Key를 Derive 한다.	

* Fbl_Crylf.c

```
Std_ReturnType Crylf_DeriveKey(uint16 pskIndex, uint32 dkLen,
    const uint8* password, uint32 pLen, const uint8* salt, uint32 sLen,
    uint32 icount, uint8* dk)
{
}
```

4.2.5 다운그레이드 방지

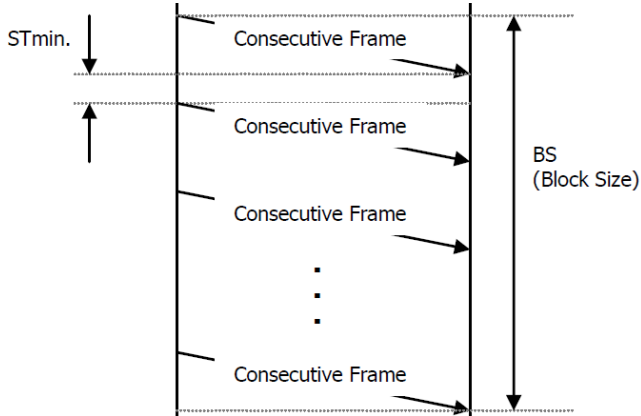
4.2.5.1 Fbl_SvcCheckVersion

Function Name	Fbl_SvcCheckVersion	
Syntax	Std_ReturnType Fbl_SvcCheckVersion (uint32 currentVersion, uint32 newVer))	
Sync/Async	Sync	
Reentrancy	Non Re-entrant	
Parameters (in)	uint32 currentVersion	Programming session 진입 시의 RTSW 버전
	uint32 newVer	Re-programming 대상 이미지의 RTSW 버전
Return Value	Std_ReturnType	함수 수행 결과
Description	다운그레이드 방지를 위해 User 가 기입한 버전 정보를 받아서 처리하는 함수이다. 현재의 버전이 currentVersion 이고, 리프로그래밍을 대상 이미지의 버전이 newVer 이다. User 가 다운그레이드 판단 로직을 작성해야 한다.	

5. Appendix

5.1 ST_Min (Seperation Time Minimum)

ST_Min은 아래 그림처럼 Consecutive Frame 사이의 시간 간격을 의미함



- ECU와 Tester 사이에 gateway 가 **없는** 경우 ST_Min 은 0 으로 recommended
- ECU와 Tester 사이에 gateway 가 **있는** 경우 ST_Min 은 2 이상으로 recommended

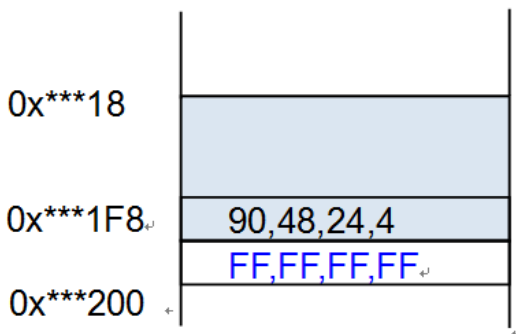
5.2 Security Key

이중화 기능이 미적용된 제어기의 경우만 해당

Security Key 주소는 8-byte align되어야 하며, 나머지 4-byte 는 flash erased value 로 채워져야 함

* MPC560, RH850, SPC58X, CYPRESS: 0xFF

* TC2XX, TC3XX : 0x00



5.3 SW version info

Ecud_Fbl.arxml 의 SwVersionInfo 설정에 따라 Fbl_Cfg.c / Fbl_SharedCfg.c 에 자동 생성됨

Path: Fbl [Fbl] > ModuleInformation [ModuleInformation] > SwVersionInfo [SwVersionInfo]

Navigator

- Fbl
 - FblGeneral
 - FblPllConfig
 - FblFlashConfig
 - FblSecurityConfig
 - FblDiagConfig
 - ModuleInformation (2)
 - SwVersionInfo**
 - FblWatchdogConfig
 - FblCommConfig

Container Details - SwVersionInfo

Short Name*: SwVersionInfo Edit

CustomerID*: ① 02 Convert... unset

ProductID*: ① 00 Convert... unset

Product Variant*: ① 00 Convert... unset

Sw Version*: ① 01 Convert... unset

Int Sw Version*: ① 00 Convert... unset

Synchro1*: ① 00 Convert... unset

Synchro2*: ① 00 Convert... unset

Build Date*: ⑧ 2019-08-23 unset

```
const Fbl_SwModuleInfo Fbl_Gdt_SwModuleInfoFbl =
{
    CUSTOMER_ID,
    PRODUCT_ID,
    PRODUCT_VARIANT,
    SW_VERSION,
    INTERMEDIATE_SW_VERSION,
    SYNCHRO1,
    SYNCHRO2,
    0xFFU,
    BUILD_DAY,
    BUILD_MONTH,
    BUILD_YEAR,
    0xFFFFU,
    0xFFFFU
};
```

Fbl_Cfg.c

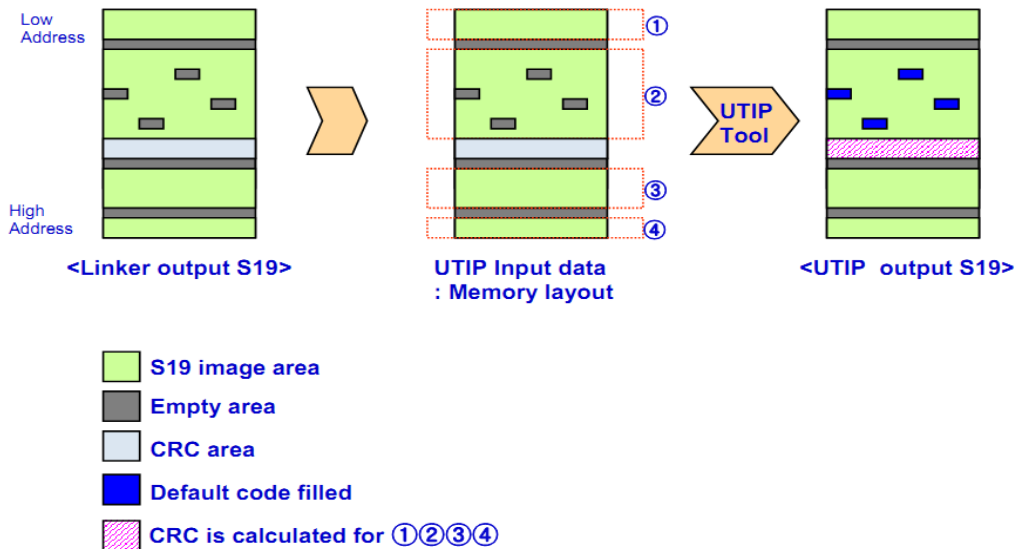
```
const Fbl_SwModuleInfo Fbl_Gdt_SwModuleInfoMainSW =
{
    CUSTOMER_ID,
    PRODUCT_ID,
    PRODUCT_VARIANT,
    SW_VERSION,
    INTERMEDIATE_SW_VERSION,
    SYNCHRO1,
    SYNCHRO2,
    0xFFU,
    BUILD_DAY,
    BUILD_MONTH,
    BUILD_YEAR,
    0xFFFFU,
    0xFFFFU
};
```

Fbl_SharedCfg.c

5.4 UTIP

5.4.1 UTIP 설명

- checksum or CRC sum 계산
- 메모리의 빈 공간 채움
- ECUD_FBL 에 설정된 SWModule 의 index 1(RTSW)의 Address 와 동일하게 utip 를 설정해야 한다.



• UTIP Setting Example for UTIP tool

- INIT operation (리프로그래밍을 하려는 영역의 시작주소와 끝주소)

▼ MEMORY INIT: Begin ~ End contains the source s19 file area

	Begin	End
1	0x8000	0x7FFFF

- FILL operation (설정된 주소공간의 빈영역을 Parameter 로 채움)

* MPC560, RH850, SPC58X, CYPRESS: 0xFF

* TC2XX, TC3XX : 0x00

▼ FILL: Begin(0x00A00) ~ End(0x00AFF) Operation(skip) Parameter(0xFF:value to the empty area)

	Begin	End	Operation	Parameter
1	0x8000	0x7FFFF	skip	0xFF
2				

- SECTION operation (CRC 계산과 저장을 위해 SECTION 을 나눔)

▼ SECTION: SK, CRC, ASW(Begin0/End0:ASW, Begin1/End1:ASW,...)

	Begin	End	Name
1	0x8000	0x801B	ASW
2	0x8170	0xBFFF	ASW
3	0xC000	0xFFFF	ASW
4	0x10000	0x17FFF	ASW
5	0x18000	0x1FFFF	ASW
6	0x20000	0x3FFFF	ASW
7	0x40000	0x5FFFF	ASW
8	0x60000	0x7FFFF	ASW
9	0x801C	0x80BF	CRC_ASW
10	0x80C0	0x816F	SK_ASW
11			

- ENCRYPTION operation
ENCRYPTION is not supported now.
- CRCSUM operation (SECTION 에서 지정한 ASW 영역에 대한 CRC 계산)
Init Address 는 Write Address - 0x9
Endian은 MPC560(M), SPC58X(M), AURIX(I), RH850(I), CYPRESS(I)

▼ CRC: SECTION(Name of source section:ASW) Init Address(CRC init. value location)

	SECTION	Init Address	Write Address	Endian	Default Value
1	ASW	0x8013	0x801C	M	0x0
2					

- SAVE operation (ASW, CRC_ASW 영역을 저장) (Security Key 제외)

▼ SAVE

	S19 Save File	Format	Sections
1	S71_768_test_utp.s19	MOTO32	ASW, CRC_ASW
2			

- UTP 파일 포맷 예제 (순서를 뒤로)


```
[step710~rtuf2_s12x_otp.otp#5HPWy+DXtYhaYtEmBuj1dg]
# Initialize range of ROM to cover for CRC, 0x0FFDFFF(?)
INIT -r=0x0004000-0x0FF8FFF
```

```
# Load S19 file
LOAD %INFILE%
```

```
fill skip -r=0x004000-0x007FFF -op=0x18,0xA7
fill skip -r=0x00C000-0x00CFFF -op=0x18,0xA7
fill skip -r=0x00CFFC-0x00CFFF -op=0xFF
fill skip -r=0xFE8000-0xFEBFFF -op=0x18,0xA7
fill skip -r=0xFC8000-0xFCBFFF -op=0x18,0xA7
fill skip -r=0xF08800-0xF09FFF -op=0x18,0xA7
```

```
SECTION 'ASW' -r=0x00004000-0x00005FFF
SECTION 'ASW' -r=0x0000C000-0x0000CF53
SECTION 'SK' -r=0x0000CFF8-0x0000CFFF
SECTION 'ASW' -r=0xFE8000-0xFEBFFF
SECTION 'ASW' -r=0xFC8000-0xFCBFFF
SECTION 'ASW' -r=0xF08800-0xF09FFF
SECTION 'CRC' -r=0x0000CF54-0x0000CFFF
```

```
# calculate CRC on -w address with init value -i
.CRCSUM.CRC16.M -r=ASW -w=0x0000CF54 -i=0x0000CF4B
save 'S71_rtuf2_s12x_otp' MOTO24 -r=ASW, CRC, SK
exit
```

Input S19 file: S19 address (paged address)
0x004000~0x005B5A (0xFD8000~0xFD9B5A)
0x00C000~0x00CF2A (0xFF8000~0xFF8F2A)
0x00CF48~0x00CFFF(0xFF8F48~0xFF8FFB)
0xF08800~0xF091D5
0xFC8000~0xFCBA2D
0xFE8000~0xFEBFFF
So, memory range 0x004000~0xFF8FFF

Load S19 file generated by linker into memory range of 0x004000~0xFF8FFF.

Do some filter operations when loading S19 file into memory range of 0x004000~0xFF8FFF.
skip => if there is content in S19 file, skip the address bytes, otherwise fills the address bytes with 0x18,0x17 pattern.

Define some memory section from the filtered and loaded memory.

Calculate CRC from the ASW sections with its initial value of -i=0x0000CF4B (S/W version).
The calculated CRC value and number of block (5 <= ASW x 5) is written into -w=0x0000CF54.
For its detail memory layout, refer to the PDR or PRM or *.ld file and zdappld.c/h.

Make the final S19 file which CRC information is embedded and memory image is arranged.

5.4.2 UTIP 예제

아래 예제는 SPC58x MCU 리프로그래밍을 위한 s19 생성 예제이다.

1) RTSW의 Start Address와 End Address를 입력한다

- EcuFbl 설정과 반드시 일치하여야 한다.
- EcuFbl 설정은 반드시 RTSW의 실제 Memory Map과 일치하여야 한다.

2) CRC, SK 영역을 구분하여 UTIP를 생성한다.

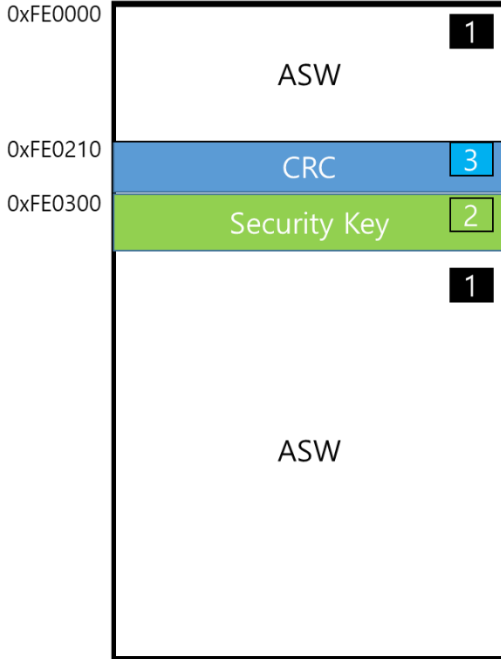
- EcuFbl 설정과 반드시 일치하여야 한다.
- EcuFbl 설정은 반드시 RTSW의 실제 Memory Map과 일치하여야 한다.
- SK 영역은 FBL이 무결성 검증이 끝나면 Write할 Block이므로, 영역의 넓이가 Write 최소 단위가 넘는지 반드시 확인한다.

3) CRC

- CRC할 대상 영역으로 ASW 영역을 입력한다.
- CRC는 Routine Control Check 때 ASW 영역의 무결성을 검증하기 위해 필요하다.(L1 설정 시 사용)
- Romtst 영역으로도 사용된다.

4) SAVE

- SK를 제외한 모든 영역을 Section으로 지정한다. SK는 실제 RoutineControl Check에서 무결성검사가 완료된 후에 FBL 자체적으로 해당 영역에 Write하여 FBL이 RTSW로 Jump할지 말지를 결정하는 Data이기 때문에, **만들어질 s19 파일에는 넣지 않도록 한다 (중요).**



Container Details - SwModule

Short Name*: MainSW

Index*: 1

Start Address*: 0x0FE0000 (1)

End Address*: 0x13BFFFF (1)

Module Info Address*: 0x0FE0200

Special Data Address*: 0x0FE0300 (2)

Crc Info Address*: 0x0FE0210 (3)

```
# This is the UTIP Configuration File

# Init Area
INIT -r=0x0FE0000-0x13BFFFF (1)

# Load file
LOAD D:\workspace\RS4_IAU_21812\ART_20200226\RS4_IAU_v220\rs4_iau_asr_swp.sre

# Fill Area
FILL skip -r=0x0FE0000-0x13BFFFF -op=0xFF

# Section Area
SECTION 'ASW' -r=0x0FE0000-0x0FE020F (1)
SECTION 'ASW' -r=0x0FE0400-0x13BFFFF
SECTION 'ASW_CRC' -r=0x0FE0210-0x0FE02FF (3)
SECTION 'ASW_SK' -r=0x0FE0300-0x0FE03FF (2)

# ENCRYPT Information

# CRCSUM Information (1)
CRCSUM CRC16 M [-r=ASW] -w=0x0FE0210 -i=0x0FE0207

# Save Information
save 'RS4_IAU_v220_13BFFFF.s19' MOTO32 -r=ASW, ASW_CRC (1, 3)

exit
```

Security Key 제외하여 s19 생성

[INIT] S19 FILE MEMORY ADDRESS AREA

	Begin	End
1	0xFE0000	0x13BFFFF
2		
3		

[FILL] MEMORY FILL PATTERN DEFINITION

	Begin	End	Operation	Parameter
1	0xFE0000	0x13BFFFF	skip	0xFF
2				
3				
4				
5				
6				
7				
8				
9				

[SECTION] MEMORY SECTION DEFINITION

	Begin	End	Name
1	0xFE0000	0xFE020F	ASW
2	0xFE0400	0x13BFFFF	ASW
3	0xFE0210	0xFE02FF	ASW_CRC
4	0xFE0300	0xFE03FF	ASW_SK
5			
6			
7			
8			
9			

[CRC] SECTION INCLUSION FOR CRC CALCULATION

	SECTION	Init Address	Write Address	Endian	Default Value
1	ASW	0xFE0207	0xFE0210	M	0x0
2					
3					

[SAVE] SAVE OUTPUT TO S19 FILE

	S19 Save File	Format	Sections
1	RS4_IAU_v220_13BFFFF	MOTO32	ASW, ASW_CRC
2			
3			

COMMAND

Input S19 File: RS4_IAU_21812\ART_20200226\RS4_IAU_v220\rs4_iau_asr_swp.sre

Load Config Save Config Execute Compare V850ES

5.5 aSIMS ini 설정 예제 (SecureFlash)

SecureFlash를 위해 aSIMS에서 s19를 Signed s19 로 변환한다.

Ini 관련된 자세한 설명은 aSIMS 홈페이지에서 Manual을 다운받을 수 있다.

이를 기반으로 Autron FBL 사용시 ini 설정 예제는 하기와 같다.

5.5.1 SecureFlash 1.0

SecureFlash 1.0은 Signature 생성을 위한 정보를 입력하면 된다.

하기와 같이 Signature 생성 대상 영역을 지정한다.

Autron FBL에서 Signature 대상 영역은 CRC 대상 영역과 동일한 ASW영역이다.

```
# This is the UTIP Configuration File
# Init Area
INIT -r=0xFE0000-0x13BFFFFF

# Load file
LOAD D:\workspace\RS4_IAU_21812\ART_20200226\RS4_IAU_v220\e_rs4_iau_asr_swp.sre

# Fill Area
FILL skip -r=0xFE0000-0x13BFFFFF -op=0xFF

# Section Area
SECTION 'ASW' -r=0xFE0000-0xFE020F
SECTION 'ASW' -r=0xFE0400-0x13BFFFFF
SECTION 'ASW_CRC' -r=0xFE0210-0xFE02FF
SECTION 'ASW_SK' -r=0xFE0300-0xFE03FF

# ENCRYPT Information

# CRCSUM Information
CRCSUM CRC16 M -r=ASW -w=0xFE0210 -i=0xFE0207

# Save Information
save 'RS4_IAU_v220_13BFFFF.s19' MOTO32 -r=ASW, ASW_CRC

exit
```

```
1 [Firmware Signing Tool Configuration]
2 [CONFIG_START]
3 [MANDATORY_CONFIG_START]
4 Firmware_INFO.FileType=Srecord
5 Firmware_INFO.FirmwareFileName=RS4_IAU_v220_13BFFFF.s19
6 [SIGNATURE_START]
7 SignatureCount=1
8 Signature.1=0xFE0000--0xFE020F, 0xFE0400--0x13BFFFFF
9 [SIGNATURE_END]
10
11 [MANDATORY_CONFIG_END]
12
13 [USER_OPTION_CONFIG_START]
14 ECU_INFO.Company=AUTRON
15 ECU_INFO.Name=IAU
16 ECU_INFO.SerialNumber=123456789
17 ECU_INFO.ModelNumber=123456789
18 ECU_INFO.HwVersion=123456789
19 ECU_INFO.SwVersion=123456789
20 DEVELOPER_INFO.DeveloperName=developer
21 DEVELOPER_INFO.Contact=name@company.com
22 Firmware_INFO.UseAutronHeader=No
23 Firmware_INFO.AutronHeaderAddress=
24 SignatureStartOffsetCount =
25 [USER_OPTION_CONFIG_END]
```

* FBL_common 1.8.0.0 이상 적용 시부터 CRC 영역도 Signature 생성 영역에 포함

```
[SIGNATURE_START]
SignatureCount=1
Signature.1=0xFE0000--0xFE02FF, 0xFE0400--0x13BFFFFF
[SIGNATURE_END]
```

Firmware_INFO.UseAutronHeader=No

Firmware_INFO.AutronHeaderAddress=

Autron ini 설정은 위와 같다.

5.5.2 SecureFlash 2.0

SecureFlash 2.0은 SecureFlash 1.0에 Encryption 정보를 추가하여야 한다.

Signature 대상 영역은 CRC 대상영역과 동일하지만, Encryption 대상 영역은 전 영역이다 (SK 제외, SK는 기존 s19 만들때에 제외되었으므로) 자세한 사항은 아래 예제를 참고 하면 된다.

```
# This is the UTIP Configuration File

# Init Area
INIT -r=0xFE0000-0x13BFFFF

# Load file
LOAD D:\workspace\RS4_IAU_21812\ART_20200226\RS4_IAU_v220\rs4_iau_asr_swp.sre

# Fill Area
FILL skip -r=0xFE0000-0x13BFFFF -op=0xFF

# Section Area
SECTION 'ASW' -r=0xFE0000-0xFE020F
SECTION 'ASW' -r=0xFE0400-0x13BFFFF
SECTION 'ASW_CRC' -r=0xFE0210-0xFE02FF
SECTION 'ASW_SK' -r=0xFE0300-0xFE03FF

# ENCRYPT Information

# CRCSUM Information
CRCSUM CRC16 M -r=ASW -w=0xFE0210 -i=0xFE0207

# Save Information
save 'RS4_IAU_v220_13BFFFF.s19' MOTO32 -r=ASW, ASW_CRC

exit
```

```
1 [Firmware Signing Tool Configuration]
2 [CONFIG_START]
3 [MANDATORY_CONFIG_START]
4 Firmware_INFO.FileType=Srecord
5 Firmware_INFO.FirmwareFileName=RS4_IAU_v220_13BFFFF.s19
6 [SIGNATURE_START]
7 SignatureCount=1
8 Signature.1=0xFE0000--0xFE020F, 0xFE0400--0x13BFFFF
9 [SIGNATURE_END]
10 [ENCRYPTION_START]
11 AutoEncryptionSetting=NO
12 EncryptionCount=1
13 Encryption.1=0xFE0000--0xFE02FF, 0xFE0400--0x13BFFFF
14 [ENCRYPTION_END]
15 [OTA_FKMETADATA_START]
16 FK_META.StartOffset=0x00001000
17 FK_META.FirmwareId=37110-1EA07
18 FK_META.FirmwareVersion=0x00343132
19 [OTA_FKMETADATA_END]
20 [MANDATORY_CONFIG_END]
21 [USER_OPTION_CONFIG_START]
22 ECU_INFO.Company=AUTRON
23 ECU_INFO.Name=IAU
24 ECU_INFO.SerialNumber=123456789
25 ECU_INFO.ModelNumber=123456789
26 ECU_INFO.HwVersion=123456789
27 ECU_INFO.SwVersion=123456789
28 DEVELOPER_INFO.DeveloperName=develooper
29 DEVELOPER_INFO.Contact=name@company.com
30 Firmware_INFO.UseAutronHeader=No
31 Firmware_INFO.AutronHeaderAddress=
32 SignatureStartOffsetCount =
33 [USER_OPTION_CONFIG_END]
```

ASW 영역

ASW, ASW_CRC 영역

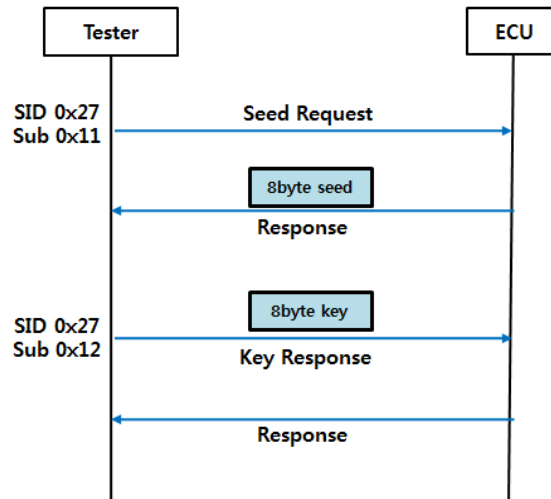
* FBL_common 1.8.0.0 이상 적용 시부터 CRC 영역도 Signature 생성 영역에 포함
FK_META.MemoryAddressingType=64 (메타정보 필드 길이 상향 조정)

```
[Firmware Signing Tool Configuration]
[CONFIG_START]
[MANDATORY_CONFIG_START]
Firmware_INFO.FileType=Srecord
Firmware_INFO.FirmwareFileName=chorus_small_FE0000_FFFFFFFFskip_sk.s19
[SIGNATURE_START]
SignatureCount=1
Signature.1=0xFE0000--0xFE02FF, 0xFE0400--0xFEFFFF
[SIGNATURE_END]
[ENCRYPTION_START]
AutoEncryptionSetting=YES
[ENCRYPTION_END]
[OTA_FKMETADATA_START]
FK_META.StartOffset=0x00001000
FK_META.FirmwareId=37110-1EA07
FK_META.FirmwareVersion=0x00343132
FK_META.MemoryAddressingType=64
[OTA_FKMETADATA_END]
[MANDATORY_CONFIG_END]
[USER_OPTION_CONFIG_START]
ECU_INFO.Company=Autron
ECU_INFO.Name=Dev
ECU_INFO.SerialNumber=123456789
ECU_INFO.ModelNumber=123456789
ECU_INFO.HwVersion=123456789
ECU_INFO.SwVersion=123456789
DEVELOPER_INFO.DeveloperName=Yongseong
DEVELOPER_INFO.Contact=yongseong.jeon@hyundai-autron.com
Firmware_INFO.UseAutronHeader=No
Firmware_INFO.AutronHeaderAddress=
SignatureStartOffsetCount =
[USER_OPTION_CONFIG_END]
[CONFIG_END]
```

5.6 보안 기능

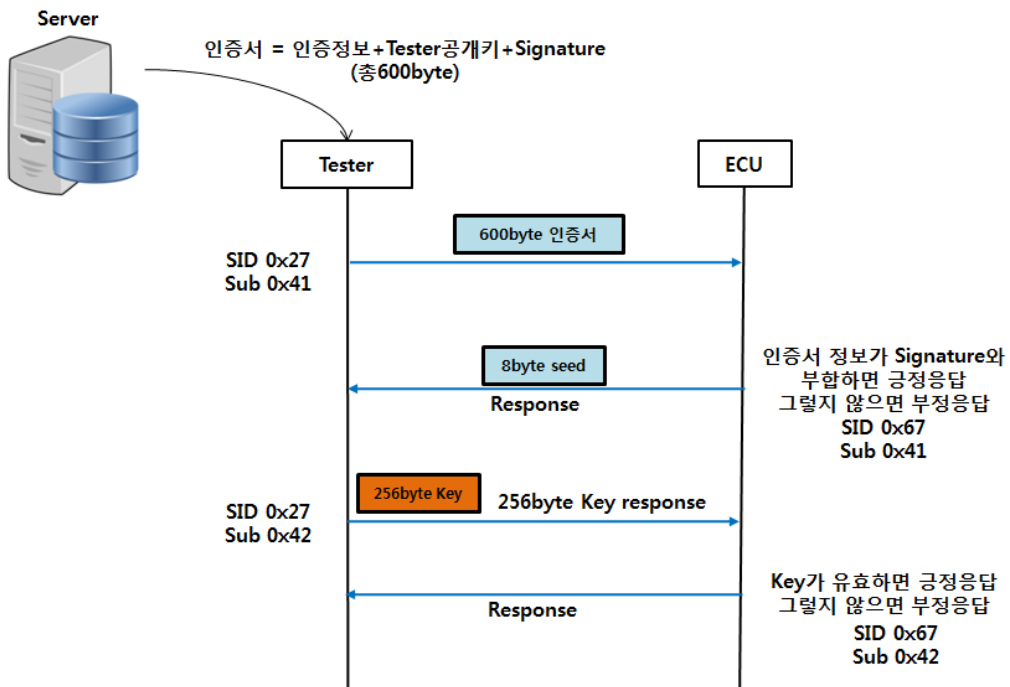
5.6.1 HSAC

HSAC(Hidden algorithm based Secure Access Control) 은 HKMC Advanced SeedKey 알고리즘 기반의 Security Access 인증 절차이며 none CGW unit을 위해 사용한다.



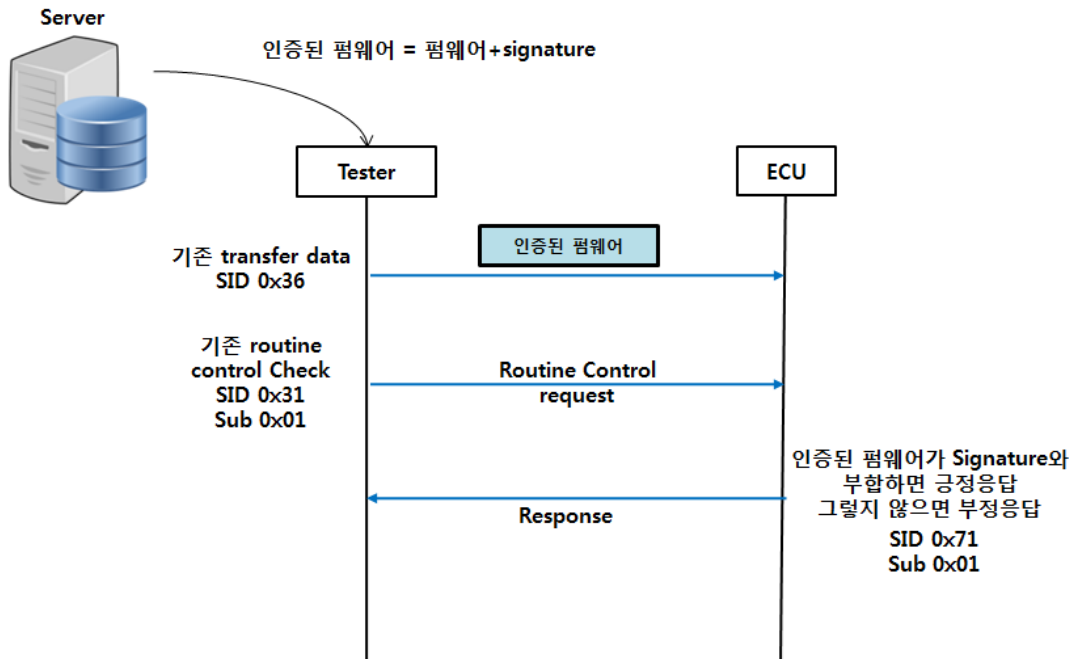
5.6.2 CSAC

CSAC(Certificate based Secure Access Control) 은 인증서 기반의 Security Access 인증 절차이며 일반적으로 CGW unit을 위해 사용한다.



5.6.3 Secure Flash

인증서 기반의 리프로그래밍 펌웨어 인증 절차로 HSAC or CSAC 사용시 적용된다.



5.7 SPC58X DIO External WDG

1) Include header file 추가(bbmcpuspc58x.h, io_port.h)

Header File Include(Integration_Code/usercode/Fbl_Appif.c)

```

/*****
**                               Include Section                               **
*****/
#include "Fbl_AppIf.h"
#include "Fbl_Cfg.h"
#include "bbmcpuspc58x.h"
#include "io_port.h"

```

2) Internal WDG 사용하지 않을 시에 Ecud_Fbl.arxml 의 FblIntWdgEnable false 설정

Container Details - FblWatchdogConfig

Short Name*:

Int Wdg Enable*: ☒ true ☐ false

Int Wdg Timeout Ms*:

3) External wdg 토글 샘플 코드는 아래와 같으며 206 은 PAD NUMBER 이며 PM_14 의 PAD NUMBER 는 206, PIN 변경에 따라 PAD NUMBER 만 변경 바람.

```

void Fbl_BeforeCommExtInit(void)
{
    /* Initialize external devices before communication device initialization. */
    SIUL2.MSCR_IO[206].R = (MSCR_ODC_PUSHPULL);
}

/* Since the code is executed in RAM, it should be implemented
without additional function call */
void Fbl_ExtPolling(void)
{
    /* Performed aperiodically */
    SIUL2.GPDO[206].R = ! SIUL2.GPDO[206].R;
}

```

Fbl_BeforeCommExtInit() 내부에서 io port 초기화

Fbl_ExtPolling() 내부에서 wdg toggling

5.8 Secure flash 적용 시 RTSW size의 ROM 의존성 제거 방법 가이드

- Secure Flash 기능을 사용할 때 Signature를 FLASH에 써야한다. Signature를 FLASH에 쓰기 위해서는 Signature Address 설정이 필요하며 주소영역은 아래의 예시처럼 사용 가능하다.

A. Signature를 Security Key 영역 바로 다음에 설정하여 RTSW의 ROM 의존성을 없애는 방법

B. Chorus의 예시를 참고하여 다른 MCU 영역 설정 필요

- RTSW의 LD파일에서 MAINSW_SIGNATURE(0x200 size 확보) 영역 설정

```
.VT_VECT_TBL : > flash_memory
MAINSW_MODULE_INFO 0x00FE0200 : > .
MAINSW_CRC 0x00FE0210 : > .
MAINSW_SECURITY_KEY 0x00FE0300 : > .
MAINSW_CPD_KEY 0x00FE03F0 : > .
MAINSW_SIGNATURE 0x00FE0400 : > .

.MEM_CHK_ID 0x00FE0600 : > . /* RomTst */
.MEM_CHK_CRC 0x00FE060C : > .
.MEM_CHECK 0x00FE0620 : > .
```

- SkipArea 에 SIGNATURE 영역 포함하여 설정

Index	Short Name	Index	Start Address	End Address
0	SwSigningSkipArea0	0	0x00FE0300	0x00FE05FF

- Signature Address 설정

Index	Short Name	Index	Start Address	End Address	Module Info A...	Special Data A...	Crc Info Address	Signature Add...
0	FBL	0	0x00FC0000	0x00FC7FFF	0x00FC0020	0x00FC0060	0x00FC0030	
1	MainSW	1	0x00FE0000	0x00FFFFFF	0x00FE0200	0x00FE0300	0x00FE0210	0x00FE0400

- S19 파일 생성 시 UTP에서 SIGNATURE 영역 제외

```
# This is the UTIP Configuration File

# Init Area
INIT -r=0xFE0000-0xFEFFFF

# Load file
LOAD C:\Users\1000804\Desktop\새 폴더\e_rtu2_SPC58x-19.10.1.sre

# Fill Area
FILL skip -r=0xFE0000-0xFEFFFF -op=0xFF

# Section Area
SECTION 'ASW' -r=0xFE0000-0xFE020B
SECTION 'ASW' -r=0xFE0600-0xFEFFFF
SECTION 'CRC' -r=0xFE020C-0xFE02FF
SECTION 'SK' -r=0xFE0300-0xFE03FF
SECTION 'SIGNATURE' -r=0xFE0400-0xFE05FF

# ENCRYPT Information

# CRCSUM Information
CRCSUM CRC16 M -r=ASW -w=0xFE020C -i=0xFE0203

# Save Information
save 'e_rtu2_SPC58x-19.10.1_Signature.s19' MOTO32 -r=ASW, CRC

exit
```

- aSIMS ini 설정 파일에서 SIGNATURE 영역 제외 하고 'Signature.1.StartOffset' 추가

```
[Firmware Signing Tool Configuration]
[CONFIG_START]
[MANDATORY_CONFIG_START]
Firmware_INFO.FileType=Srecord
Firmware_INFO.FirmwareFileName=e_rtu2_SPC58x-19.10.1_Signature.s19
[SIGNATURE_START]
SignatureCount=1
Signature.1=0xFE0000--0xFE02FF, 0xFE0600--0xFEFFFF
[SIGNATURE_END]
[ENCRYPTION_START]
AutoEncryptionSetting=NO
EncryptionCount=1
Encryption.1=0xFE0000--0xFE02FF, 0xFE0600--0xFEFFFF
[ENCRYPTION_END]
[OTA_FKMETADATA_START]
FK_META.StartOffset=0x00001000
FK_META.FirmwareId=37110-1EA07
FK_META.FirmwareVersion=0x00343132
FK_META.MemoryAddressingType=64
[OTA_FKMETADATA_END]
[MANDATORY_CONFIG_END]
[USER_OPTION_CONFIG_START]
ECU_INFO.Company=Autron
ECU_INFO.Name=Dev
ECU_INFO.SerialNumber=123456789
ECU_INFO.ModelNumber=123456789
ECU_INFO.HwVersion=123456789
ECU_INFO.SwVersion=123456789
DEVELOPER_INFO.DeveloperName=JaeHyun
DEVELOPER_INFO.Contact=JaeHyun.Lim@hyundai-autron.com
Firmware_INFO.UseAutronHeader=No
Firmware_INFO.AutronHeaderAddress=
Firmware_INFO.IncludeDelimiterAndLength=YES
SignatureStartOffsetCount = 1
Signature.1.StartOffset = 0x00FE0400
[USER_OPTION_CONFIG_END]
[CONFIG_END]
```

C. 기존처럼 끝주소를 사용하여 Signature를 FLASH에 쓰는 방법

i. Signature Address, End Address 설정

- Signature는 기존 RTSW의 끝주소로 설정
- End Address는 FLASH Erase를 위해서 Signature Address 영역을 포함하는 영역으로 설정

Container Details - SwModule									
Index	Short Name	Index	Start Address	End Address	Module Info A...	Special Data A...	Crc Info Address	Signature Add...	
0	FBL	0	0x00FC0000	0x00FC7FFF	0x00FC0020	0x00FC0060	0x00FC0030		
1	MainSW	1	0x00FE0000	0x01010000	0x00FE0200	0x00FE0300	0x00FE0210	0x01000000	

5.9 사용자 정의 사항

- FBL에서는 Engine Condition, Power Condition 등의 Application 상태 및 제어기 상태에 대한 판단을 포함하지 않는다. 따라서 필요 시 진단사양서, SWP 가이드에 따라 제어기 상태 등을 고려하여 각 명령 처리 여부를 판단하는 기능을 추가 하여야 한다. (Reprogramming Session 진입 시(FBL mode) Application에서 수행해야할 처리 방법은 DCM 매뉴얼의 가이드를 따른다)

5.10 Software Version Check (다운그레이드방지) 기능 정보

5.10.1 Software Version Check 기능이란

FBL은 업데이트 될 Software 버전이 현재 플래시되어있는 Software 버전보다 낮다면 Software 다운로드를 허용해서는 안된다(ES9549-21K 소프트웨어 다운그레이드 방지). FBL이 현재 펌웨어의 버전을 취득하기 위해서는 Software 버전 정보가 약속된 Software 주소 (Code Flash) 에 저장되어 있어야 한다. 특별한 점은, 당 사 FBL은 리프로그래밍 과정에서의 비정상 Reset을 대비하기 위해 버전 정보를 펌웨어의 앞뒤에 분산 하도록 구조를 제안한다. Flash Erase 시에 플래시 되어있는 버전 정보가 지워지는 경우 등 여러 다양한 상황을 대비해야되기 때문이다. 따라서, Application 유저는 새로운 펌웨어를 FBL이 제안하는 Header/Trailer 구조에 맞게 구성해야 한다. 이는 Application 컴파일 단계에서 어플리케이션 .c파일과 링커를 통해 반영되어야 한다. 하위 버전 펌웨어가 감지되면 RoutineControl Check 단계에서 NRC 를 응답하며 펌웨어 업데이트가 실패한다.

- Header Magic Number : 0x6D6F6269
- Trailer Magic Number : 0x6C67656E
- Current Version : App 유저 자유 입력 (4byte)

5.10.2 Structure of the Version Block

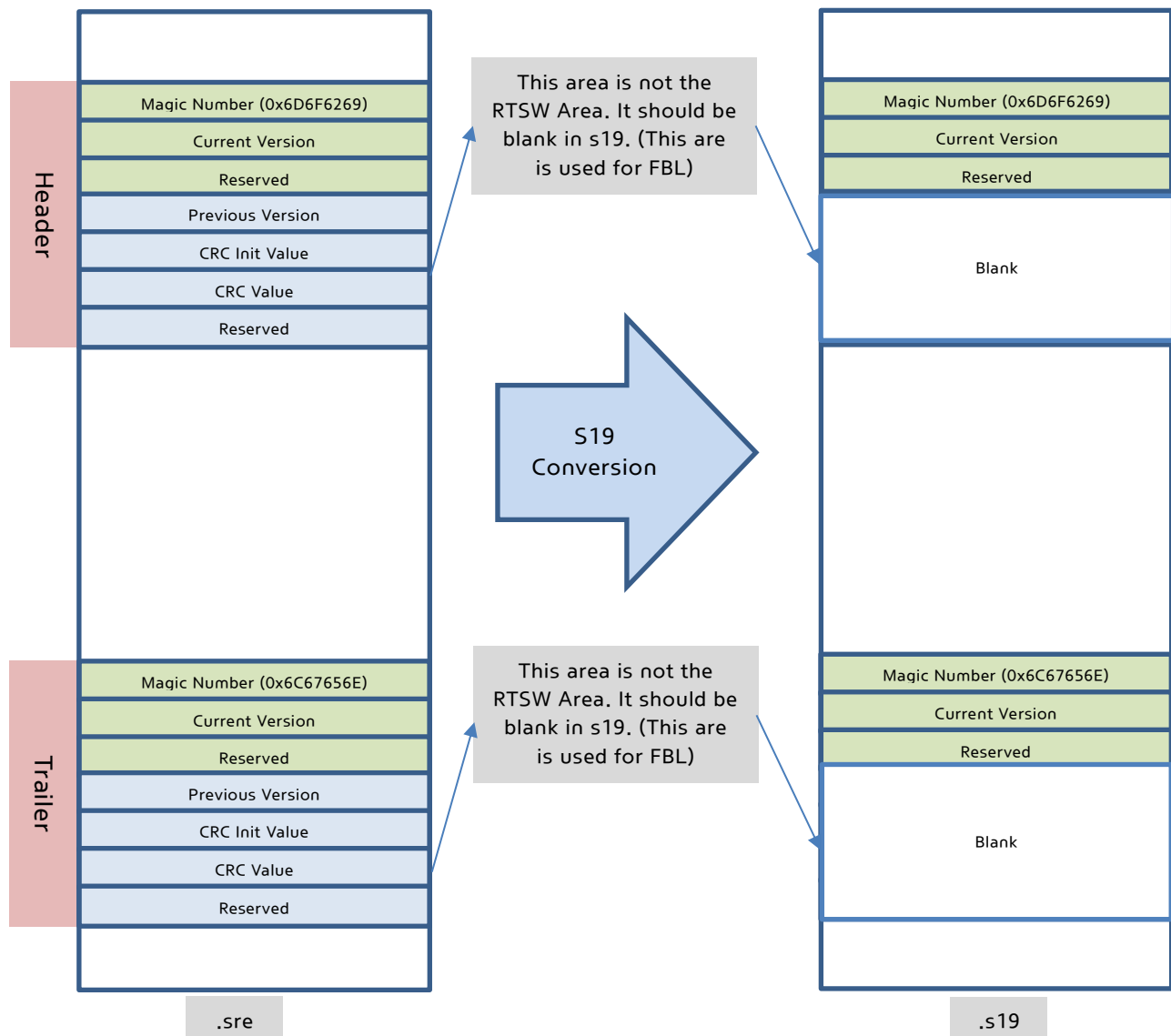
[Header Block]

Total	App Data (256 byte)			Blank (used in the FBL) 256 byte			
length	4 byte	4 byte	248 byte	4 byte	2 byte	2 byte	248 byte
Name	Magic Number	Current Version	Reserved	Previous Version	CRC InitValue	CRC Value	Reserved
Value	0x6D6F6269	0XXXXXXXXX	0XXX 0XX...	0XXXXXXXXX	0XXXXX	0XXXXX	0XXX 0XX...

[Trailer Block]

Total	App Data (256 byte)			Blank (used in the FBL) 256 byte			
length	4 byte	4 byte	248 byte	4 byte	2 byte	2 byte	248 byte
Name	Magic Number	Current Version	Reserved	Previous Version	CRC InitValue	CRC Value	Reserved
Value	0x6C67656E	0XXXXXXXXX	0XXX 0XX...	0XXXXXXXXX	0XXXXX	0XXXXX	0XXX 0XX...

[Firmware Structure]



- 여기에서 각 버전 블록의 뒷 Blank 영역 (0x100 byte) 은 FBL 만 사용하는 영역이기 때문에 Host Firmware에서 제외된다. Host Firmware는 이 영역을 Reserved 처리 후에 S19에서 제외한다.

5.10.3 어플리케이션 유저 Software Version Check 기능 사용 가이드 예시

5.10.3.1 App (RTSW) 펌웨어 구조 세팅

App_Version.c (RTSW) 에 하기와 같이 Header, Trailer 구성

첫번째로, RTSW 펌웨어에 Header 와 Trailer 를 구성하고 버전정보를 저장하여 코드영역에 배치될 수 있도록 준비하는 단계이다. FBL 이 제안하는 Block 구조를 만들기 위해 SW_BlkJFlashInfo (이름은 상관없음) 구조체를 생성하고 Sw_BlkJHeader, Sw_BlkJTrailer (이름은 상관없음) 구조체 변수를 만들어 각각 SW_START/STOP_SEC_BLK_HEADER, SW_START/STOP_SEC_BLK_TRAILER 심볼 (이 예시에는 메모리 Section 배치를 위해 Integration_MemMap 1.1.67.0 모듈을 활용했으며, 사용자가 어떠한 형태로 구현하여도 관계 없음) 을 통해 링커에서 코드 영역에 배치할 준비를 한다. 이때, Header 와 Trailer 의 Magic Number 는 약속된 고정값이 쓰이며 이 값이 잘못된 경우 버전 블록이 없는것으로 판단하여 리프로그래밍이 실패하게 됨을 유의한다.

아래 Magic Number 와 Current Version 만이 Firmware 데이터이고, PreviousVersion 및 Crc 정보는 FBL 이 사용하는 영역
이므로 아래와 같이 MagicNumber 과 CurrentVersion 정보만 정확히 입력하도록 한다. MagicNumber 와 Current Version
만이 App Data 임에도 불구하고 구조체를 아래와 같이 PreviousVersion 과 Crc, Reserved 까지 구성하는 이유는 FBL 이 사용
하게 될 영역을 예약(Reserved) 하므로써 다른 App Data 가 Current Version 뒤로 붙지 못하게 하기 위함이다. 이후 리프로그
래밍을 위한 s19 생성 단계에서는 FBL 이 사용하는 영역은 (PreviousVersion~ Reserved_2) Host 펌웨어에서 제거한다
(Partition Flag 와 동일한 방식으로 추후 FBL 이 사용할 영역으로 s19 영역에서는 제외함)

Source File (ex. Static_CodeWReference_CodeWApp_Version.c)

```

/*****
**                               **
**      Include Section          **
**                               **
*****/
#include "Std_Types.h"

/*****
**                               **
**      Macro Definition         **
**                               **
*****/

typedef struct
{
    /* offset of Address
    *
    * +-----+-----+-----+-----+-----+-----+-----+-----+
    * | magicNumber(4) | currentVersion(4) | reserved(248) |           Reserved for FBL           |
    * +-----+-----+-----+-----+-----+-----+-----+-----+
    * |<-----0x100 (App Data)----->|<-----0x100 (FBL Note)----->|
    */
    /* Magic Number 4-byte*/
    uint32 magicNumber;
    /* Current Version 4-byte */
    uint32 currentVersion;
    /* Reserved for write unit 248-byte */
    uint8 reserved_1[248];
    /* Magic Number 4-byte*/
    uint32 previousVersion; /* This is reserved area, not including the App*/
    /* crcinit 2-byte */
    uint16 crcInit; /* This is reserved area, not including the App*/
    /* crcVal 2-byte */
    uint16 crcVal; /* This is reserved area, not including the App*/
    /* Reserved for write unit 248-byte */
    uint8 reserved_2[248]; /* This is reserved area, not including the App*/
} SW_BlkJFlashInfo;

/*****
**                               **
**      Function Definitions      **
**                               **
*****/

/* Header Magic Number */
#define MAGIC_NUMBER_HEADER (0x6D6F6269U)
/* Trailer Magic Number */
#define MAGIC_NUMBER_TRAILER (0x6C67656EU)
/* Current Version Number */
#define PRODUCT_CURRENT_VERSION (0x00010201U)

#define VERSION_START_BLK_HEADER
#include "MemMap.h"

const SW_BlkJFlashInfo App_SoftwareVersionHeader =

```

```
{
    /* Magic Number */
    .magicNumber = (uint32)MAGIC_NUMBER_HEADER,
    /* Version Info */
    .currentVersion = (uint32)PRODUCT_CURRENT_VERSION,
};

#define VERSION_STOP_BLK_HEADER
#include "MemMap.h"

#define VERSION_START_BLK_TRAILER
#include "MemMap.h"

const SW_BlkJFlashInfo App_SoftwareVersionTrailer =
{
    /* Magic Number */
    .magicNumber = (uint32)MAGIC_NUMBER_TRAILER,
    /* Version Info */
    .currentVersion = (uint32)PRODUCT_CURRENT_VERSION,
};

#define VERSION_STOP_BLK_TRAILER
#include "MemMap.h"

/*****
**                               **
**                               **
*****/
```

5.10.3.2 App 링커 스크립트 세팅

- 생성한 Trailer 및 Header 블록이 실제 특정 코드 영역에 배치될 수 있도록 심볼을 사용하여 링커에서 주소를 지정한다. 세팅시에 할당될 주소값에 유의한다. 주소 결정 조건은 하기와 같다.

- ① Header 와 Trailer 는 전체 펌웨어의 앞쪽과 뒤쪽에 배치될 수 있도록 한다
- ② Header 와 Trailer 는 서로 다른 Code Flash Sector (Erase 단위) 에 위치해야 한다. 되도록이면 Code Flash Sector 의 시작 부분으로 결정한다.
* Code Flash Sector 단위는 MCU 마다 다르므로 주의해야 한다.
- ③ Header 와 Trailer 의 시작 위치는 Code Flash 의 Write Align 단위에 맞아야 한다 (대부분의 MCU 는 0x100 단위면 충분함)
- ④ Header 와 Trailer 각각은 오직 하나의 Code Flash Sector 에 포함되어야 한다.

Linker Script (ex. RTSW.ld)

```
.....

.pibase
MAINSW_MODULE_INFO          0x10048800    : > .
MAINSW_CRC                  0x10048810    : > .
MAINSW_SECURITY_KEY         0x10048900    : > .
MAINSW_CPD_KEY              0x100489F0    : > .
MAINSW_SIGNATURE            0x10048A00    : > .

.MEM_CHK_ID                 0x10048F00    : > .
.MEM_CHK_CRC                0x10048F0C    : > .
.MEM_CHECK                  0x10048F20    : > .

VERSION_BLK_HEADER          0x10050000    : > .

.text                       : > .
.ghwts.text                 : > .

.....

VERSION_BLK_TRAILER         0x10208000    : > .

.....
```

실제로 빌드 시에 아래와 같이 Map파일과 sre 파일이 올바르게 생성되었는지 반드시 확인 (map 파일, sre 파일)

.mem_chk_crc	10080c20	00000000	0	00000000
.MEM_CHECK	10084c20	00000000	0	00000000
VERSION_BLK_HEADER	10050000	00000200	512	00000824
.text	10050200	00029e88	170216	0000ea24
.ghwts.text	10079ae8	00000000	0	00000000
.intercall	10079ae8	00000000	0	00000000
.interfunc	10079ae8	00000000	0	00000000
.fixaddr	10079ae8	00000000	0	00000000
.fixtype	10079ae8	00000000	0	00000000
.secinfo	10079ae8	0000003c	60	002a30c
.robase	10079b24	00000000	0	00000000
.rodata	10079b24	00006af8	27384	002a348
.syscall	10080620	00000004	4	0030e40
HSM_DRIVER_SECTION_CODE	10080624	000006e6	1766	0030ee44
.ROM.data	10080d20	00000b5c	2908	0031540
.ROM.tls_cond.data	1008187c	00000000	0	00000000
.LTB_CODE	10082000	00000554	1364	0033000
.EcucPartition_Main_CODE	10082560	0000356c	218828	0033560
.EcucPartition_Main_CONST	100b7c40	00007ec4	32452	0068c40
VERSION_BLK_TRAILER	10208000	00000200	512	0070b04
.sram_base	0000a000	00000000	0	00000000
.stack	0000a000	00000400	1024	00000000
.....				

위 내용을 부연설명하자면, TV-II-B-E 시리즈 같은 경우는 0x8000 단위로 Erase가 가능하다. 따라서 0x10050000 도 섹터의 시작주소이고, 0x10208000 도 섹터의 시작 주소이다. 단위가 0x8000이므로 Header와 Trailer는 각각 서로 다른 하나의 섹터에 속한다. 또한 Header 와 Trailer의 시작주소는 Write 단위인 0x100으로 Align 되어 있다. TV-II-B-E의 경우 2M 제품으로 Single Bank 사용 시 RTSW 영역은 (FBL, HSM 영역 제외) 0x10048000—0x1020FFFF 이고 Header는 Interrupt Vector Table 및 Partition Flag 등의 정보 위치를 감안 한 다음 섹터에 위치시켰고, Trailer는 가장 끝쪽에서 0x8000 이전 섹터에 위치시켰다.

5.10.3.3 App s19 및 signed s19 생성

앞서 언급했듯, Header 와 Trailer 의 0x200 (512Byte) 중 앞 0x100 영역만이 Host Firmware 영역이고, 나머지 뒤 0x100 영역은 FBL이 블록 관리 용도로 사용하는 영역으로 Reprogramming 될 Firmware 영역이 아니다. 따라서 Host Firmware 영역인 0x100 만큼만 포함시키고 뒤 0x100만큼은 s19 생성 시 제거해주어야 한다 (Partition Flag 와 같은 개념)
(* Structure of the Version Block 참조)

.UTP (without RomTst)

```
# This is the UTIP Configuration File

# Init Area
INIT -r=0x10048000-0x1020FFFF

# Load file
LOAD E:\svn\.....

# Fill Area
FILL skip -r=0x10048000-0x1020FFFF -op=0xFF

# Section Area
SECTION 'ASW' -r=0x10048000-0x1004890F
SECTION 'CRC' -r=0x10048810-0x100488FF
SECTION 'SK' -r=0x10048900-0x100489FF
SECTION 'Signature' -r=0x10048A00-0x10048BFF
SECTION 'ASW' -r=0x10048C00-0x100500FF
SECTION 'BLK_HEADER_RESERVED' -r=0x10050100-0x100501FF
SECTION 'ASW' -r=0x10050200-0x102080FF
SECTION 'BLK_TRAILER_RESERVED' -r=0x10208100-0x102081FF
SECTION 'ASW' -r=0x10208200-0x1020FFFF

# ENCRYPT Information

# CRCSUM Information
CRCSUM CRC16 I -r=ASW -w=0x10048810 -i=0x10048807

# SIGINFO Information, Only FDT Enable support

# Save Information
```

```
save 'TestProject_RTU_FBL_common_cyt2b9xx_0x10048000_0x1020FFFF.s19' MOTO32 -r=ASW, CRC
```

```
exit
```

위 예시처럼, Host 펌웨어에 포함되지 말아야 할 SK, Signature 와 같은 맥락으로 Header 와 Trailer 의 Reserved 영역인 뒤 0x100 만크도 s19에서 제거한다. (초록 색 SECTION 만 SAVE)

.ini (ASIMS)

```
[Firmware Signing Tool Configuration]
[CONFIG_START]
[MANDATORY_CONFIG_START]
Firmware_INFO.FileType=Srecord
Firmware_INFO.FirmwareFileName=TestProject_RTU_FBL_common_cyt2b9xx_0x10048000_0x1020FFFF.s19
Firmware_INFO.IncludeDelimiterAndLength=YES
[SIGNATURE_START]
SignatureCount=1
Signature.1=0x10048000--0x100488FF, 0x10048C00--0x100500FF, 0x10050200--0x102080FF, 0x10208200--0x1020FFFF
[SIGNATURE_END]
[ENCRYPTION_START]
AutoEncryptionSetting=NO
EncryptionCount=1
Encryption.1=0x10048000--0x100488FF, 0x10048C00--0x100500FF, 0x10050200--0x102080FF, 0x10208200--0x1020FFFF
[ENCRYPTION_END]
[OTA_FKMETADATA_START]
FK_META.StartOffset=0x00001000
FK_META.FirmwareId=37110-1EA07
FK_META.FirmwareVersion=0x00343132
FK_META.MemoryAddressingType=64
[OTA_FKMETADATA_END]
[HASH_ALL_SECTION_START]
AllSection =0x10048000--0x100488FF, 0x10048C00--0x100500FF, 0x10050200--0x102080FF, 0x10208200--0x1020FFFF
[HASH_ALL_SECTION_END]
[MANDATORY_CONFIG_END]
[USER_OPTION_CONFIG_START]
ECU_INFO.Company=Autron
ECU_INFO.Name=Dev
ECU_INFO.SerialNumber=123456789
ECU_INFO.ModelNumber=123456789
ECU_INFO.HwVersion=123456789
ECU_INFO.SwVersion=123456789
DEVELOPER_INFO.DeveloperName=JHLim
DEVELOPER_INFO.Contact=jaehyun.lim@hyundai-autron.com
Firmware_INFO.UseAutronHeader=No
Firmware_INFO.AutronHeaderAddress=
ECU_INFO.Endian=LITTLE
SignatureStartOffsetCount = 1
Signature.1.StartOffset = 0x10048A00
[USER_OPTION_CONFIG_END]
[CONFIG_END]
```

위 예시처럼, 호스트 펌웨어 s-record 전체 영역을 시그니처 대상 영역으로 지정한다. 앞서 s-record 상에서 제외된 영역 (빨간 영역) 은 빈 공간이므로 이를 제외한 초록 영역을 시그니처 및 암호화 대상 영역으로 지정한다. 초록색 영역이 곧 Host Firmware 전체 영역이다.

5.10.3.4 Ecud_Fbl 설정

```
/AUTRON/Fbl/FblGeneral/FblSoftwareVersionCheck : True
/AUTRON/Fbl/FblFlashConfig/FblFlashEraseUnit : MCU 별 상이
/AUTRON/Fbl/ModuleInformation/SwModule/BlkHeaderAddress : MCU 별 상이
/AUTRON/Fbl/ModuleInformation/SwModule/BlkTrailerAddress : MCU 별 상이
```

5.10.3.5 Fbl_SvcCheckVersion 구성

다운그레이드 방지 기능을 사용하면 Fbl_Applf.c 의 Fbl_SvcCheckVersion 함수에서 다운그레이드 감지의 실패, 성공을 판

단한다. Fbl_SvcCheckVersion 함수는 input 인자로 currentVersion 과 newVer 을 받는데, currentVersion 은 리프로그래밍을 실시할 때 programming session 으로 천이하기 이전 RTSW 의 버전을 의미하고, newVer 은 FBL 을 통해 새로 리프로그래밍 할 이미지의 버전을 의미한다. 두 값 모두 SW_BlkFlashInfo 의 currentVersion 에서 값을 따온다.

다음과 같이 구성되어 있는 코드에서 주석 부분은 단순 대수 비교를 해서 version check 를 했던 기존 로직이고, user 의 판단에 따라 아래 함수를 구성해주어야 한다. Default 값으로 E_NOT_OK 를 return 하도록 구성되어 있고, E_OK 를 return 할 경우 버전 체크가 정상적으로 이루어졌다고 판단하고 리프로그래밍을 진행하게 된다.

Integration_FBL_F (Fbl_Applf.c)

```
Std_ReturnType Fbl_SvcCheckVersion(uint32 currentVersion, uint32 newVer)
{
    Std_ReturnType Ldt_RetValue;

    /* This is HAE test code, Do not use this. */
    /*
    if (newVer >= currentVersion)
    {
        Ldt_RetValue = E_OK;
    }
    else
    {
        Ldt_RetValue = E_NOT_OK;
    }
    */

    /* User should make own version check logic here. */

    Ldt_RetValue = E_NOT_OK;
    return Ldt_RetValue;
}
```

5.10.3.6 메모리 이중화 사용의 경우 (Psuedo Code)

아래코드는 단지 예시일 뿐이므로 사용자는 주의하도록 한다.

Integration_FBL_F (Fbl_Applf.c)

```
void Fbl_GetActivePartitionBlkAddress(uint32* headerAddr, uint32* trailerAddr)
{
    Uint32 Offset = 0U;

    IF B Partition is Active Partition,
    {
        Offset = Offset + 0x02000000 (It's depend on the MCU Spec and User's config)
    }
    Else
    {
        Offset = 0U;
    }

    *headerAddr = *headerAddr + Offset;
    *trailerAddr = *trailerAddr + Offset;
}
```

메모리 이중화의 경우 Active Partition 의 위치는 User가 판단 가능하고 Offset 또한 유저가 판단할 수 있으므로 위 함수를 반드시 채워넣어야 한다. 위 함수의 In-Out 파라미터 headerAddr, trailerAddr 는 현재 버전 정보를 취득할때만 사용된다.

5.10.3.7 테스트 (FBL 전역변수를 통한)

- 버전이 같거나 상위 버전으로 리프로그래밍 시
: 아래와 같이 VersionStatus 가 FBL_VERSION_OK 상태이고 리프로그래밍이 완료됨

```
Fbl_Gen_BlkState = FBL_BLK_NORMAL ≒ 0x4
Fbl_Gbl_VersionStatus = FBL_VERSION_OK ≒ 0x1
Fbl_Gu32_NewVersion = 0x00010201
Fbl_Gu32_CurrentVersion = 0x00010201
Fbl_Gu32_PreviousVersion = 0x00010201
Fbl_Gu32_BlkTrailerAddr = 0x10208000
Fbl_Gu32_BlkHeaderAddr = 0x10050000
```

<< 위 변수는 RoutineControlCheck 단계에서 확인 가능

- 다운그레이드 시 (리프로그래밍을 통해 버전 0x00010201 이 플래시되어있는 상태에서 버전 0x00000200 으로 다운그레이드)
: 아래와 같이 VersionStatus 가 FBL_VERSION_NOT_OK 상태가 되고 RoutineControl Check 에서 Fail 됨

```
Fbl_Gen_BlkState = FBL_BLK_HEADER_INVALID ≒ 0x2
Fbl_Gbl_VersionStatus = FBL_VERSION_NOT_OK ≒ 0x2
Fbl_Gu32_NewVersion = 0x0201
Fbl_Gu32_CurrentVersion = 0x00010201
Fbl_Gu32_PreviousVersion = 0x0
Fbl_Gu32_BlkTrailerAddr = 0x10208000
Fbl_Gu32_BlkHeaderAddr = 0x10050000
```

- **주의사항** : FBL 이 다운그레이드를 감지하기 시작하는 시점은 FBL 이 적어도 한번 RTSW 를 Flashing 했을 때 이다. 유저가 이 기능을 정확하게 테스트하기 위해서는 외부 Flash 장비로 FBL 과 HSM 만을 Flash 한 다음, 최초 RTSW Download 이 완료된 뒤 Downgrade Detection 기능이 동작한다.

※ 외부 Flash 장비는 FBL 이 내부적으로 운영하는 Header/Trailer 관리 블록을 업데이트 해주지 못함.

6. Open source copyright

6.1 lwIP (light weight IP)

Copyright (c) 2001-2004 Swedish Institute of Computer Science. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. The name of the author may not be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS AND ANY EXPRESS OR IMPLIED WARRANTIES,

INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.